



DATN ứng dụng tay thao tác di động trong y tế

Đồ án tốt nghiệp kỹ sư (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP THIẾT KẾ, CHẾ TẠO VÀ ĐIỀU KHIỂN TAY THAO TÁC DI ĐỘNG ỨNG DỤNG TRONG Y TẾ

NGUYỄN HUY TOÀN
toan.nh170941@sis.hust.edu.vn

VŨ ĐỨC VĂN
van.vd170993@sis.hust.edu.vn

NGUYỄN KHẮC HÙNG
hung.nk151799@sis.hust.edu.vn

Ngành Cơ điện tử

Giảng viên hướng dẫn: **TS Nguyễn Xuân Hạ**

Chữ ký của GVHD

Bộ môn:

Cơ sở thiết kế máy và Robot

Viện:

Cơ khí

Hà Nội, 8/2022

TRƯỜNG CƠ KHÍ

Bộ môn Cơ sở thiết kế máy và rôbốt

Học kỳ: 20212

Năm học: 2021-2022

ĐỒ ÁN TỐT NGHIỆP: NGÀNH CƠ ĐIỆN TỬ Mã HP: ME5115

Thời gian thực hiện: 15 tuần;

Ngày 22/04/2022

Ngày 22/04/2022

Ngày.../08/2022

ĐƠN VỊ CHUYÊN

NGƯỜI RA ĐỀ

CB Hướng dẫn

MÔN

(ký, ghi rõ họ tên)

(ký, ghi rõ họ tên)

(ký, ghi rõ họ tên)

TS Nguyễn Xuân Hạ

Ngày giao nhiệm vụ: 22/04/2022;

Ngày hoàn thành: 05/08/2022

I. Nhiệm vụ thiết kế: Thiết kế, chế tạo và điều khiển tay thao tác di động ứng dụng trong y tế

II. Số liệu cho trước

1. Tải trọng: 0.5 (kg);
2. Tầm với: 0.35 (m);
3. Vận tốc bàn tay gấp: 0.25 (m/s).

III. Nội dung thực hiện

1. Phân tích tổng quan và nhu cầu thực tiễn

- Tổng quan về các rôbốt có nhiệm vụ đặc biệt;
- Lựa chọn rôbốt cho đề án;

2. Thiết kế hệ thống cơ khí

- Xây dựng bản vẽ cơ khí bằng phần mềm Solidworks;
- Thực hiện xây dựng mô hình thực tế;
- Kiểm tra bền các khâu rôbốt và tính toán bài toán động học.

3. Thiết kế hệ thống điều khiển

- Xây dựng mạch nguyên lý và điều khiển bằng phần mềm Altium;
- Lựa chọn các thiết bị cho hệ thống điều khiển;
- Phân tích, đánh giá các chỉ tiêu kỹ thuật của hệ thống;

4. Lập trình điều khiển

- Lập trình điều khiển tay thao tác trên mô hình thực tế.
- Điều khiển rôbốt Kobuki và giao tiếp với tay thao tác.

Giảng viên hướng dẫn
(ký, ghi rõ họ tên)

LỜI CẢM ƠN

Trong chặng đường gần 5 năm học tập tại **Bộ môn Cơ Điện Tử, Trường Cơ Khí, Đại Học Bách Khoa Hà Nội**, và đến tận bây giờ vừa hoàn thành Đồ án tốt nghiệp.

Nhóm đồ án xin chân thành gửi lời cảm ơn đến:

Ban giám hiệu, quý thầy cô **Bộ môn Cơ Điện Tử, Trường Cơ Khí, Đại Học Bách Khoa Hà Nội** đã tạo điều kiện thuận lợi trong suốt quá trình học tập rèn luyện tại trường.

Quý Thầy/Cô trong **Trường Cơ Khí** đã tận tình giảng dạy và hết lòng truyền đạt kiến thức, kinh nghiệm cũng như những bài học quý báu của mình.

Đặc biệt, nhóm đồ án xin gửi lời cảm ơn đến **TS Nguyễn Xuân Hạ**, là giảng viên trực tiếp hướng dẫn trong suốt thời gian làm đồ án vừa qua. Cảm ơn thầy đã luôn hướng dẫn, giúp đỡ, bổ sung kiến thức chuyên môn và bổ ích để nhóm hoàn thành đề tài.

Cuối cùng, chân thành cảm ơn đến các bạn đã cùng học tập, giúp đỡ và ủng hộ trong suốt thời gian vừa qua.

Vì kiến thức của bản thân còn hạn chế nên trong quá trình hoàn thiện đồ án này không tránh khỏi những sai sót, kính mong nhận được ý kiến đóng góp từ quý thầy cô.

Một lần nữa chúng em kính chúc quý thầy cô đang công tác tại **Trường Cơ khí, Đại học Bách Khoa Hà Nội** lời chúc sức khỏe dồi dào – thành công – hạnh phúc!

Nhóm xin chân thành cảm ơn!

Hà Nội, ngày tháng 8 năm 2022
Sinh viên thực hiện

TÓM TẮT

Trong khuôn khổ **Đồ án tốt nghiệp** với đề tài **Thiết kế, chế tạo và điều khiển tay thao tác ứng dụng trong y tế**, nhóm đã đưa ra những vấn đề liên quan, các phương án giải quyết và đặc biệt là phương án về tính toán và thiết kế và điều khiển tay thao tác và mobile rôbot.

Với bố cục đồ án bao gồm 3 ý chính sau:

1- Tổng quan về các rôbot có nhiệm vụ đặc biệt

Phần này sẽ là cái nhìn sơ qua về sự phát triển của ngành chế tạo rôbot nói chung bao gồm lịch sử phát triển, phân loại và ứng dụng hiện nay.

2- Tính toán thiết kế tay thao tác di động

Bao gồm các bước tính toán thiết kế hệ thống cơ khí (phần mềm SolidWorks), hệ thống điều khiển cho tay thao tác (phần mềm Altium và) và điều khiển mobile rôbot (phần mềm) xuất phát từ yêu cầu đề ra.

3- Thực hiện thi công mô hình tay thao tác di động.

Nhóm làm rõ các quá trình chế tạo và lắp ráp các thiết bị linh kiện cơ khí, điện tử để lắp ráp thành tay thao tác di động. Và thiết kế giải thuật điều khiển để rôbot hoạt động đúng như yêu cầu.

Nhóm đạt được các kết quả tương đối chính xác về hoạt động của mô hình tay thao tác di động và có tiềm năng lớn trong việc ứng dụng vào ngành y tế thực tiễn. Định hướng phát triển của đề tài hướng đến những hoạt động có tính chất đặc thù như pha chế, kiểm thử các chất hóa học ứng dụng trong y tế nói riêng và ngành hóa học nói chung.

Ngoài ra, trong đồ án còn các phần đi kèm như bảng chú thích, danh mục hình ảnh, các kết luận và phụ lục code điều khiển của mô hình.

MỤC LỤC

DANH MỤC	Trang
LỜI CẢM ƠN	1
TÓM TẮT	2
MỤC LỤC	3
DANH MỤC HÌNH VẼ	5
DANH MỤC BẢNG BIỂU	8
MỞ ĐẦU	9
CHƯƠNG 1- TỔNG QUAN VỀ RÔBỐT THÔNG MINH	10
1. Giới thiệu về Robot và Robot tự hành thông minh	10
1.1 Sự phát triển của ngành công nghiệp Rôbốt	10
1.2 Phân loại Robot	11
1.2.1 Robot công nghiệp	11
1.2.2 MicroRobot	12
1.2.3 Robot tự hành	13
1.3 Robot tự hành thông minh	15
1.3.1 Định nghĩa và đặc điểm chung	15
1.3.2 Phân loại rôbốt tự hành thông minh	16
1.4 Tay thao tác	20
1.4.1 Định nghĩa và phân loại	20
1.4.2 Ứng dụng của tay thao tác di động	24
1.5 Kết hợp rôbốt tự hành thông minh và tay thao tác	25
1.6 Phương pháp thực hiện đề tài	25
1.6.1 Mục tiêu của đề tài	25
1.6.2 Yêu cầu đề bài	25
1.6.3 Phương pháp thực hiện đề tài	26
CHƯƠNG 2- THIẾT KẾ, CHẾ TẠO VÀ TÍNH TOÁN TAY THAO TÁC	28
2.1 Tính toán, lựa chọn động cơ và bộ truyền cho các khâu	28
2.1.1 Cụm bàn tay kẹp	28
2.1.2 Tính toán động cơ và bộ truyền khớp quay thứ 3	29
2.1.3 Tính toán động cơ và bộ truyền khớp quay thứ 2	32
2.1.4 Tính toán động cơ và bộ truyền khớp quay thứ 1	33
2.2 Thiết kế xây dựng mô hình 3D bằng phần mềm SolidWorks	35
2.2.1 Cụm truyền động cho khớp quay thứ nhất	35
2.2.2 Cụm truyền động cho khớp quay thứ hai	36
2.2.3 Cụm truyền động cho khớp quay thứ ba	38
2.2.4 Cụm tay gấp	38
2.3 Kiểm tra bền các khâu	39
2.3.1 Kiểm tra bền cánh tay khâu 3	39
2.3.2 Kiểm tra bền cánh tay khâu 2	41
2.4 Thiết kế mạch điều khiển tay thao tác bằng phần mềm Altium	44

2.4.1 Thiết kế mạch nguyên lí	44
2.4.2 Quá trình thiết kế mạch PCB	55
2.4.3 Tính toán bài toán động học rôbốt	56
2.5 Không gian làm việc tay thao tác	61
CHƯƠNG 3- ĐIỀU KHIỂN ROBOT TỰ HÀNH THÔNG MINH KOBUKI	64
3.1 Tổng quan về hệ điều hành ROS và bài toán SLAM	64
3.1.1 Hệ điều hành ROS	64
3.1.2 Bài toán SLAM	67
3.1.3 Gói SLAM trong ROS	76
3.2 Điều khiển và xây dựng giao diện ứng dụng trên Rôbốt Kobuki	77
3.2.1 Giới thiệu phần cứng	77
3.2.2 Điều khiển Kobuki robot trên hệ điều hành ROS	82
CHƯƠNG 4- KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN	88
4.1 Điều khiển tay thao tác và mobile Robot Kobuki	88
4.1.1 Điều khiển và đánh giá kết quả tay thao tác	88
4.1.2 Điều khiển và đánh giá kết quả mobile robot Kobuki	94
4.1.3 Kết hợp tay thao tác và robot Kobuki	102
4.2 Hướng phát triển	106
KẾT LUẬN	108
DANH MỤC TÀI LIỆU THAM KHẢO	109

DANH MỤC HÌNH VẼ

- Hình 1.1 Các giai đoạn phát triển của rôbốt
- Hình 1.2 Ứng dụng rôbốt công nghiệp trong các lĩnh vực (đơn vị 1000 rôbốt)
- Hình 1.3 Microrobot ruồi
- Hình 1.4 Hệ thống AGV trong nhà máy
- Hình 1.5 Rôbốt AIRSTAR tại sân bay quốc tế Incheon Hàn Quốc
- Hình 1.6 Rôbốt tự hành khử trùng tại bệnh viện Vũ Hán – Trung Quốc
- Hình 1.7 Robot Mar Rover 2020 khám phá sao Hỏa
- Hình 1.8 Rôbốt Talon IV rà phá bom mìn
- Hình 1.9 Rôbốt Kuri đang chơi với trẻ
- Hình 1.10 Robot 3 bậc tự do
- Hình 1.11 Robot song song
- Hình 1.12 Robot hàn chuỗi hở
- Hình 1.13 Cấu tạo động cơ bước (Step Motor)
- Hình 1.14 Động cơ khí nén
- Hình 1.15 Bộ truyền bánh răng – đai răng
- Hình 1.16 Rôbốt hỗ trợ thực hiện phẫu thuật
- Hình 1.17 Rôbốt Swatbot lấy mẫu xét nghiệm COVID-19
- Hình 2.1 Gá cụm tay gấp dự kiến
- Hình 2.2 Gá động cơ servo dự kiến
- Hình 2.3 Tính khối lượng dự kiến 1 chi tiết khâu 3
- Hình 2.4 Động cơ Giảm tốc GA25 Encoder
- Hình 2.5 Thông số động cơ giảm tốc GA25 Encoder
- Hình 2.6 Chiều rộng đai răng
- Hình 2.7 Bảng tra số răng
- Hình 2.8 Tính khối lượng khâu 2 dự kiến
- Hình 2.9 Tính khối lượng tám xoay khâu 1
- Hình 2.10 Động cơ bước Step57
- Hình 2.11 Tổng thể hệ thống tay thao tác 4 bậc tự do
- Hình 2.12 Cụm truyền động khớp quay thứ nhất
- Hình 2.13 Dẫn động trực tiếp từ động cơ
- Hình 2.14 Dẫn động gián tiếp qua hệ bánh răng, đai răng
- Hình 2.15 Tám xoay khâu 1
- Hình 2.16 Cụm truyền động cho khớp quay thứ hai
- Hình 2.17 Cánh tay khâu 2
- Hình 2.18 Khớp nối trục mặt bích
- Hình 2.19 Gối đỡ trục P08
- Hình 2.20 Cụm truyền động cho khớp quay thứ ba
- Hình 2.21 Cánh tay khâu 3
- Hình 2.22 Cụm tay gấp của robot
- Hình 2.23 Add vật liệu như PEEK cho khâu 3
- Hình 2.24 Chia phần tử trên khâu 3
- Hình 2.25 Phân bố ứng suất uốn trên cánh tay khâu 3
- Hình 2.26 Chuyển vị trên khâu 3

Hình 2.27 Add vật liệu như PEEK cho khâu 2
 Hình 2.28 Chia phần tử trên khâu 2
 Hình 2.29 Phân bố ứng suất uốn trên cánh tay khâu 2
 Hình 2.30 Chuyển vị trên khâu 2
 Hình 2.31 Mạch hạ áp 5V
 Hình 2.32 IC LM1117 và sơ đồ chân
 Hình 2.33 Mạch hạ áp 3.3V
 Hình 2.34 IC STM32
 Hình 2.35 Sơ đồ chân RS485 2 dây
 Hình 2.36 Sơ đồ nguyên lý mạch RS232 và RS485
 Hình 2.37 Module Wifi ESP32
 Hình 2.38 Mạch nguyên lý khối Wifi
 Hình 2.39 Mạch chuyển đổi HX711
 Hình 2.40 Mạch nguyên lý khối Analog Converter
 Hình 2.41 Sơ đồ nguyên lý tín hiệu Input
 Hình 2.42 Module điều khiển động cơ DC BTS 7960 43A
 Hình 2.43 BTS 7960 43A Pinout
 Hình 2.44 Mô hình mạch cầu H
 Hình 2.45 Động cơ quay thuận
 Hình 2.46 Động cơ quay nghịch
 Hình 2.47 Mạch nguyên lý chính của tay thao tác
 Hình 2.48 Hình ảnh 3D mạch PCB
 Hình 2.50 Sơ đồ vị trí các khối nguyên lý
 Hình 2.51 Chiều dài và góc xoắn khâu thứ I
 Hình 2.52 Các thông số D-H của khâu
 Hình 2.53 Các trục tọa độ trên tay thao tác
 Hình 2.54 Biểu diễn vùng làm việc của rôbot
 Hình 2.55 Tọa độ điểm tác động cuối E trong cơ cấu tay thao tác
 Hình 2.56 Không gian làm việc của tay thao tác trong hệ $O0x0y0z0$
 Hình 3.1 ROS là một Meta-Operating System
 Hình 3.1 ROS hỗ trợ giao tiếp đa nền tảng
 Hình 3.3 Các thành phần trong HĐH ROS
 Hình 3.4 Phương thức giao tiếp giữa các Node
 Hình 3.5 Màu của các ô lưới khi quét bằng cảm biến siêu âm
 Hình 3.6 Biểu đồ xác suất có vật cản khi sử dụng cảm biến siêu âm
 Hình 3.7 Biểu đồ trên thể hiện xác suất theo khoảng cách đối với laser
 Hình 3.8 Giá trị của các ô lưới trong Costmap
 Hình 3.9 Sơ đồ nguyên lý tìm đường và tránh vật cản
 Hình 3.10 Bản đồ Costmap với các vùng có màu tương ứng
 Hình 3.11 Các tham số của hàm tìm quỹ đạo Global_planner
 Hình 3.12 Quỹ đạo di chuyển của rôbot khi đặt các tham số theo mặc định
 Hình 3.13 Quỹ đạo di chuyển của Robot khi sử dụng thuật toán Dijkstra
 Hình 3.14 Quỹ đạo di chuyển của rôbot khi sử dụng thuật toán A^*
 Hình 3.15 Quỹ đạo di chuyển của Robot khi sử dụng thuật toán Grid Path

Hình 3.16 Các không gian vận tốc trong thuật toán DWA
 Hình 3.17 Lựa chọn quỹ đạo tối ưu tới điểm đích
 Hình 3.18 Sơ đồ hoạt động của gói Navigation
 Hình 3.19 Cấu tạo của Robot
 Hình 3.20 Sơ đồ kết nối các phần cứng của Robot
 Hình 3.21 Đế Kobuki robot
 Hình 3.22 Intel PC NUC
 Hình 3.23 Rplidar A1M8
 Hình 3.24 nguyên tắc đo khoảng cách của Rplidar A1M8
 Hình 3.25 Thông số điện áp tín hiệu giao tiếp Serial UART
 Hình 3.26 Thông số cấp nguồn cho động cơ Rplidar
 Hình 3.27 Thông số nguồn Rplidar
 Hình 3.28 Màn hình cảm ứng
 Hình 3.29 Kết quả chạy lệnh roscore ở terminal trong HĐH Ubuntu
 Hình 3.30 Quá trình quét bản đồ tĩnh
 Hình 3.31 Rviz
 Hình 3.32 Robot di chuyển sử dụng tính năng 2D Nav Goal của Rviz
 Hình 3.33 Lỗi va chạm tại góc rẽ
 Hình 3.34 Vật cản trong bản đồ costmap
 Hình 4.1 Tay thao tác 4 bậc tự do
 Hình 4.2 Hệ tọa độ đánh giá kết quả của tay thao tác
 Hình 4.3 Sơ đồ vị trí các điểm so với điểm gốc (màu đỏ)
 Hình 4.4 Sơ đồ vị trí các điểm so với điểm gốc TH2 (màu đỏ)
 Hình 4.5 Sơ đồ vị trí các điểm so với điểm gốc TH3 (màu đỏ)
 Hình 4.6 Sơ đồ vị trí các điểm so với điểm gốc TH4 (màu đỏ)
 Hình 4.7 Hệ tọa độ quan sát sai lệch vị trí của robot ở điểm đích
 Hình 4.8 Hình ảnh quan sát từ trên xuống khi tâm robot bị lệch xuống dưới điểm gốc tọa độ
 Hình 4.9 Sự di chuyển của robot kiểm tra độ chính xác robot đến điểm đích
 Hình 4.10 Vị trí tâm của robot trên hệ tọa độ robot dừng lại tại điểm đích khi không có vật cản
 Hình 4.11 Quỹ đạo di chuyển của robot khi không có vật cản
 Hình 4.12 Tâm của robot trên hệ tọa độ robot dừng lại tại điểm đích khi có vật cản
 Hình 4.13 Quỹ đạo di chuyển của robot khi có vật cản
 Hình 4.14 Sơ đồ đánh giá khả năng tránh vật cản động của Robot
 Hình 4.15 Vật cản mà cảm biến laser phát hiện nhưng chưa được cập nhật lên bản đồ bản đồ cục bộ
 Hình 4.16 Sơ đồ nguyên lý của giao thức MQTT
 Hình 4.17 Cơ chế hoạt động của giao thức MQTT
 Hình 4.18 Giao diện Server
 Hình 4.19 Giao diện xây dựng app điều khiển tay thao tác

DANH MỤC BẢNG BIỂU

- Bảng 1.1 Bảng so sánh các công nghệ dẫn đường hiện nay
- Bảng 2.1 Thông số nhựa in PLA
- Bảng 2.2 Thông số kỹ thuật STM32F205VET6
- Bảng 2.3 Thông số D-H
- Bảng 2.4 Bảng D-H tay thao tác 4 bậc tự do
- Bảng 3.1 Các phương thức giao tiếp trong ROS
- Bảng 3.2 Các tham số điều chỉnh thông qua chạy thực tế
- Bảng 4.1 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH1
- Bảng 4.2 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH2
- Bảng 4.3 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH3
- Bảng 4.4 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH4
- Bảng 4.5 Bảng giá trị x, y tại điểm đích khi không có vật cản
- Bảng 4.6 Bảng giá trị x, y quan sát được tại điểm đích khi có vật cản
- Bảng 4.7 Bảng kết quả khi người di chuyển theo quỹ đạo vuông góc với đường thẳng AB với tốc độ khoảng 100cm/s
- Bảng 4.8 Bảng kết quả khi người di chuyển theo quỹ đạo vuông góc với đường thẳng AB với tốc độ khoảng 30cm/s
- Bảng 4.9 Bảng kết quả khi người di chuyển theo quỹ đạo song song với đường thẳng AB (chiều từ B đến A) với tốc độ đi bộ bình thường khoảng 100cm/s

MỞ ĐẦU

Trong sự nghiệp công nghiệp hóa, hiện đại hóa đất nước, vấn đề tự động hóa sản xuất có vai trò đặc biệt quan trọng, cùng với sự phát triển trong các lĩnh vực cơ khí, điện tử, tin học thì sự tích hợp hữu cơ của 3 lĩnh vực đó là cơ điện tử cũng phát triển không ngừng và được coi là một trong những ngành mũi nhọn. Sản phẩm đặc trưng của ngành cơ điện tử là rôbốt. Ngày nay rôbốt xuất hiện ngày càng nhiều và thay thế dần vai trò của con người. Rôbốt có khả năng làm việc với độ chính xác cao, trong môi trường độc hại, trong các công việc mà đòi hỏi phải cẩn thận không được nhầm lẫn, thao tác nhẹ nhàng, tinh tế.

Để đạt được mục tiêu đề tài là **Thiết kế, chế tạo và điều khiển tay thao tác di động ứng dụng trong y tế**, nhằm sớm đưa sản phẩm vào ứng dụng trong các công việc đặc thù của ngành y tế thì nhóm đã hoàn thành sản phẩm của mình dưới dạng sản phẩm thực tế và bản báo cáo thuyết minh này.

Với bố cục gồm 3 chương:

Chương 1- Thông số kỹ thuật yêu cầu; tổng quan về các công việc đặc thù cần sự trợ giúp của rôbốt;

Chương 2- Thiết kế hệ thống cơ khí, điều khiển tay thao tác;

Chương 3- Điều khiển mobile rôbốt Kobuki;

Chương 4- Lắp ráp, lập trình điều khiển mô hình và kiểm thử.

Em xin gửi lời cảm ơn tới TS Nguyễn Xuân Hạ, cảm ơn thầy vì những đóng góp qua những bài giảng và những hướng dẫn trong quá trình trao đổi ở các buổi gặp mặt. Những góp ý, sửa chữa của thầy sẽ phần nào giúp em tự tin hơn trong cách thức tiếp cận với nền công nghiệp hiện nay bởi mặc dù đã có những sự chuẩn bị của em hoặc cũng có thể kiến thức em mang đến trong bài báo cáo này còn sai sót và chưa đúng. Em rất mong có được sự bổ sung, sửa chữa đó.

Em chân thành cảm ơn và chúc thầy sức khỏe!

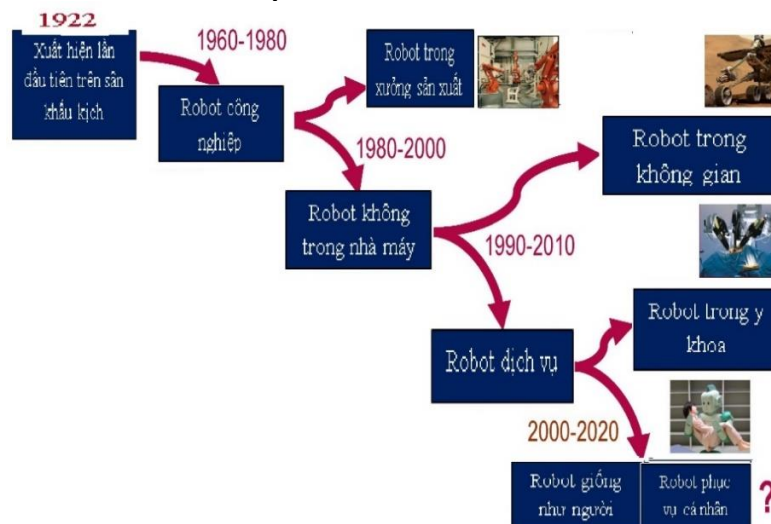
CHƯƠNG 1- TỔNG QUAN VỀ RÔBÔT THÔNG MINH

1. Giới thiệu về Robot và Robot tự hành thông minh

1.1 Sự phát triển của ngành công nghiệp Rôbốt

Theo các dự đoán, trong thời điểm hiện nay cũng như trong thời gian tới, nhu cầu sử dụng rôbốt trong cuộc sống hằng ngày sẽ tăng lên đáng kể và rôbốt sẽ trở thành tâm điểm của cuộc cách mạng công nghệ lớn sau “Internet”. Với xu thế này, với các ứng dụng truyền thống khác của rôbốt trong công nghiệp, y tế, giáo dục đào tạo, giải trí và đặc biệt trong an ninh quốc phòng thì thị trường rôbốt và các dịch vụ ăn theo rôbốt sẽ vô cùng lớn. Rôbốt là thiết bị mà trong đó tổng hợp cả khoa học và công nghệ. Để thiết kế và chế tạo được rôbốt, người thiết kế cần có các tri thức của toán học, cơ học, vật lý, điện tử, lý thuyết điều khiển, khoa học tính toán và nhiều tri thức khác. Để có thể ứng dụng được rôbốt, ta cần biết rõ về đối tượng ứng dụng. Tùy vào các điều kiện áp dụng và môi trường sử dụng khác nhau mà lựa chọn loại rôbốt có kết cấu và tính năng đáp ứng được yêu cầu đề ra.

Rôbốt đã có những tiến bộ đáng kể trong hơn nửa thế kỷ qua. Rôbốt đầu tiên được ứng dụng trong công nghiệp vào những năm 60 để thay thế con người làm các công việc nặng nhọc, nguy hiểm trong môi trường độc hại. Do nhu cầu cần sử dụng ngày càng nhiều trong các quá trình sản xuất phức tạp nên rôbốt công nghiệp cần có những khả năng thích ứng linh hoạt và thông minh hơn. Ngày nay, ngoài ứng dụng sơ khai ban đầu của rôbốt trong chế tạo máy và phục vụ sản xuất công nghiệp thì các ứng dụng khác như trong y tế, chăm sóc sức khỏe, nông nghiệp, đóng tàu, xây dựng, an ninh quốc phòng và gia đình đang có nhu cầu gia tăng đang là động lực cho các rôbốt địa hình và rôbốt dịch vụ phát triển.



Hình 1.1 Các giai đoạn phát triển của rôbốt

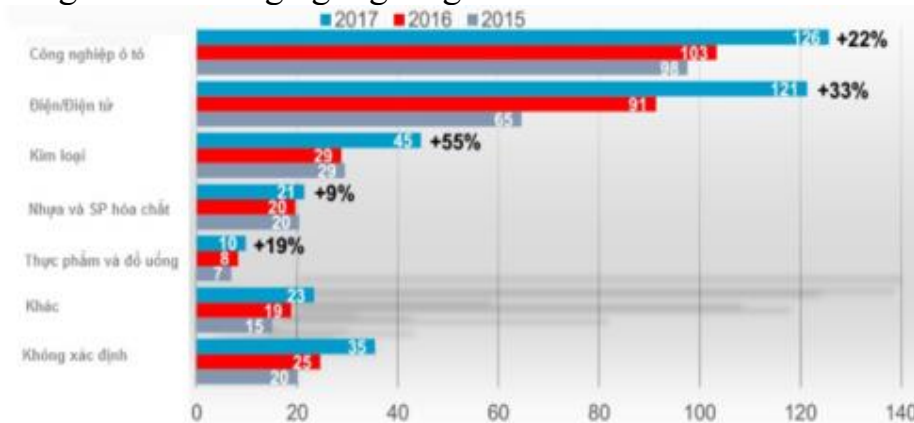
Có thể kể đến một số loại rôbốt được quan tâm nhiều trong thời gian qua là: tay thao tác rôbốt (Robot Manipulators), rôbốt di động (Mobile Robots), rôbốt phỏng sinh học (Bio Inspired Robots) và rôbốt cá nhân (Personal Robots). Tay máy rôbốt bao gồm các loại rôbốt công nghiệp (Industrial Robot), rôbốt y tế (Medical Robot) và rôbốt trợ giúp người tàn tật (Rehabilitation robot). Rôbốt di động được nghiên cứu nhiều như xe tự hành

trên mặt đất AGV (Autonomous Guided Vehicles), rôbot tự hành dưới nước AUV (Autonomous Underwater Vehicles), rôbot tự hành trên không UAV (Unmanned Arial Vehicles) và rôbot vũ trụ (Space robots). Đối với rôbot phỏng sinh học, các nghiên cứu thời gian qua tập trung vào 2 loại chính là rôbot đi (Walking robots) và robot dạng người (Humanoid Robots). Bên cạnh đó, các loại rôbot phỏng sinh học dưới nước như rôbot cá, các cấu trúc chuyển động phỏng theo sinh vật biển cũng được nhiều nhóm nghiên cứu phát triển.

1.2 Phân loại Robot

1.2.1 Robot công nghiệp

Robot công nghiệp là một loại máy có thể thực hiện công việc một cách tự động bằng sự điều khiển của máy tính hoặc các vi mạch điện tử được lập trình. Robot công nghiệp được lập trình sẵn theo một trình tự nhất định và sử dụng mục đích phục vụ công việc lắp ráp, sản xuất hoặc chế biến sản phẩm. Tuy đã trải qua một thời gian phát triển nhưng nhu cầu Robot công nghiệp trên thế giới vẫn không ngừng tăng lên.



Hình 1.2 Ứng dụng rôbot công nghiệp trong các lĩnh vực (đơn vị 1000 rôbot [5])

Như H1.2 đã thể hiện, các rôbot công nghiệp ứng dụng vô cùng rộng rãi trong các dây chuyền sản xuất và ở đa dạng các lĩnh vực. Đặc biệt là trong ngành sản xuất, chế tạo công nghiệp ô tô và điện điện tử. Số lượng các rôbot công nghiệp tăng nhanh chóng, trong ngành ô tô là từ 98 000 rôbot (2015) lên 103 000 rôbot (2016) và lên 126 000 rôbot (2017); tăng 28% chỉ trong vòng 3 năm. Nổi bật với sự tăng trưởng số lượng rôbot áp dụng vào dây chuyền sản xuất là ngành điện - điện tử, với mức tăng 86% từ 65 000 rôbot (2015) lên 121 000 rôbot (2017). Với mức độ tăng trưởng như trên thì rôbot ngày càng thay thế con người trong các hoạt động sản xuất công nghiệp.

Hiện nay trên thế giới, do nhu cầu sử dụng robot ngày càng nhiều trong các quá trình sản xuất phức tạp với mục đích góp phần nâng cao năng suất dây chuyền công nghệ, giảm giá thành, nâng cao chất lượng, và nâng cao khả năng cạnh tranh của sản phẩm. Đồng thời cải thiện điều kiện lao động, nên robot công nghiệp cần có những khả năng thích ứng tốt và thông minh hơn với những cấu trúc đơn giản và linh hoạt.

Có thể kể đến một số ứng dụng điển hình của robot công nghiệp trên thế giới như:

- Robot song song dùng trong phân loại và đóng gói sản phẩm.
- Robot sơn và phun sơn: Được ứng dụng khá nhiều trong sơn gỗ, sơn thép.
- Robot dùng trong công nghệ ép phun nhựa.
- Robot gấp hàng, Robot bốc xếp hàng hóa hay Robot bốc dỡ hàng hóa.
- Robot vận chuyển và đóng gói sản phẩm: Robot đóng gói và vận chuyển trong phạm vi rộng các sản phẩm khác nhau.
- Robot hàn công nghiệp hay Robot hàn tự động: phục vụ cho ngành ô tô xe máy.

1.2.2 MicroRobot

Ngày nay, nhờ những công nghệ chế tạo siêu chính xác, kích thước các thiết bị ngày càng giảm. Đó cũng chính là bước đệm lớn để MicroRobot phát triển mạnh mẽ. Cũng nhờ đó mà con người đã có thêm khả năng khám phá những môi trường mà trước đây chưa từng được khám phá, thế giới siêu nhỏ.

MicroRobot được ứng dụng rất nhiều trong y tế, khám phá môi trường gần gũi với con người nhất, chính là môi trường bên trong cơ thể người. Đặc biệt trong y tế, các MicroRobot có thể làm giảm các tổn thương gây cho người bệnh trong quá trình điều trị. Do đặc điểm siêu nhỏ của MicroRobot nên MicroRobot thường làm bằng vật liệu đặc biệt cũng như điều khiển khác biệt so với các Robot thông thường. MicroRobot sẽ không thể dùng các động cơ để di chuyển, hay không thể dùng bất cứ mạch điện tử thông thường nào để điều khiển. Do vậy ngành MicroRobot là một ngành rất khó và cần rất nhiều đầu tư và thời gian để phát triển.



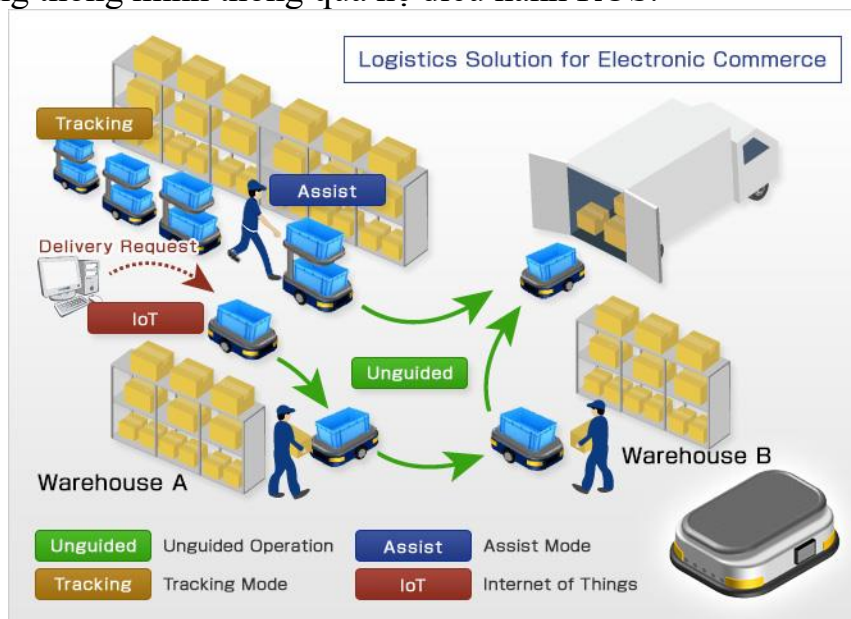
Hình 1.3 Microrobot ruồi [13]

H1.3 là hình ảnh chụp của một microrobot ruồi được đặt cạnh một đồng xu. Với kích thước nhỏ gọn như trên thì nó có khả năng hoạt động quân sự một cách kín đáo và khó có thể bị phát hiện. Có thể thấy các MicroRobot là những robot siêu nhỏ. Ngoài ra, MicroRobot cũng thường được dùng để chỉ những robot có khả năng thao tác với những vật có kích thước cỡ micromet.

1.2.3 Robot tự hành thông minh

Robot tự hành thông minh là những rôbot có khả năng hoạt động độc lập, không cần tới sự can thiệp của con người – không cần giám sát, không cần sự điều khiển trực tiếp của con người. Robot tự hành thông minh có tiềm năng phát triển rất lớn do sự phát triển của các công nghệ AI, IOT và sự phát triển của phần cứng điển hình là sự phát triển của các cảm biến. Đồng thời với cộng đồng phát triển không ngừng tăng lên, việc phát triển một robot tự hành thông minh thực sự đơn giản hơn rất nhiều so với trước đây.

Các rôbot tự hành cũng được phân loại thành nhiều dạng sản phẩm khác nhau, tùy vào mục đích sử dụng của người yêu cầu: xe kéo (Towing Vehicle); xe chở (Unit Load Vehicle); xe đẩy (Cart Vehicle); xe nâng (Fork Vehicle). Hoặc có thể phân loại nhờ cách thức hay phương án dẫn đường như B1.1. Ngày nay, các rôbot tự hành được ứng dụng trong đa dạng các lĩnh vực khác nhau. Có thể kể đến một số ứng dụng nổi bật dưới đây: các mobile rôbot (hay còn được gọi là các AGV - Autonomous Guided Vehicles) là các rôbot tự hành hoạt động trong các nhà máy sản xuất, với nhiệm vụ vận chuyển hàng hóa, sản phẩm từ vị trí này đến vị trí khác, nó có thể hoạt động theo các đường line dẫn hướng được thiết lập sẵn hoặc có thể di chuyển nhờ hệ thống dẫn đường thông minh thông qua hệ điều hành ROS.



Hình 1.4 Hệ thống AGV trong nhà máy

H1.4 thể hiện nhiệm vụ của các AGV trong hệ thống vận chuyển hàng hóa. Các AGV giúp vận chuyển giữa các giá để hàng, giữa giá hàng và xe vận chuyển bên ngoài. Với sự phát triển của công nghệ ngày nay, các công nghệ phân tích dẫn hướng đường đi cho các mobile rôbot ngày càng được cải thiện và đạt độ chính xác cao hơn. Tùy vào mục đích sử dụng và phạm vi áp dụng các AGV mà các doanh nghiệp có thể lựa chọn đa dạng các hình thức dẫn đường cho hệ thống xe tự hành thông minh hoạt động trong nhà máy. Có thể kể đến các phương án dẫn đường như: dẫn đường bằng laser, dẫn đường bằng băng từ, hạt từ và dẫn đường tự do. Dưới đây là B1.1 so sánh giữa các phương án dẫn đường cho xe tự hành. Bảng so sánh này chủ

yếu nêu lên các ưu và nhược điểm của từng hệ thống dẫn đường, từ đó giúp người dùng lựa chọn một cách chính xác và phù hợp nhất với yêu cầu bài toán đề ra.

Bảng 1.1 Bảng so sánh các công nghệ dẫn đường hiện nay

Công nghệ dẫn đường	Ưu điểm	Nhược điểm
Dẫn đường bằng laser	- Không thay đổi hiện trạng khi lắp đặt - Độ chính xác cao - Tốc độ nhanh - Không tốn chi phí bảo trì	- Khi cần thay đổi, khách hàng phải nhờ sự trợ giúp từ nhà cung cấp - Giá thành cao
Dẫn đường bằng băng từ	- Dễ thay đổi theo nhu cầu - Không thay đổi hiện trạng khi lắp đặt - Độ chính xác cao - Di chuyển ổn định, tin cậy - Chi phí rẻ	- Tốn chi phí bảo trì băng từ; - Chi phí băng từ cao khi sử dụng số lượng lớn; - Không phù hợp với dự án phức tạp.
Dẫn đường tự do	- Linh hoạt cao. Dễ dàng thêm lộ trình di chuyển - Chi phí lắp đặt thấp - Không thay đổi hiện trạng khi lắp đặt - Không tốn chi phí bảo trì	- Hoạt động ổn định trong môi trường thay đổi; - Hiệu quả cao, tuy nhiên giá thành cũng cao; - Độ chính xác thấp
Dẫn đường bằng hạt từ	- Độ chính xác cao - Không tốn chi phí bảo trì	- Tác động đến hiện trạng - Lắp đặt phức tạp hơn so với công nghệ băng từ

1.3 Robot tự hành thông minh

1.3.1 Định nghĩa và đặc điểm chung

Robot tự hành thông minh là một loại rôbot đặc biệt, là loại rôbot có thể hoạt động độc lập. Con người có thể cùng làm việc ngay cạnh robot mà không cần lo lắng về vấn đề an toàn hay xảy ra va chạm với rôbot. Điều này hoàn toàn khác với những rôbot công nghiệp trước đây – cần có một khu vực riêng để làm việc. Robot tự hành thông minh không đi theo một quỹ đạo cố định nào mà nó tự tìm quỹ đạo di chuyển phù hợp với sự thay đổi của môi trường, điều này đã đem rôbot ra ngoài các môi trường cố định, làm cho rôbot tự hành thông minh có nhiều ứng dụng đa dạng. Để có những khả năng này rôbot tự hành thông minh có các đặc điểm sau:

- Thu thập thông tin về môi trường xung quanh.
- Có thể tự di chuyển mà không cần sự điều khiển của con người.
- Có thể xử lý với các sự thay đổi của môi trường

a) Thu thập thông tin môi trường xung quanh

Khi rôbot tự hành thông minh hoạt động, nó tự hành liên tục thu thập các thông tin về môi trường xung quanh. Rôbot nhờ vào các cảm biến như: cảm biến laser, cảm biến hồng ngoại, cảm biến siêu âm... để thu thập các thông tin của môi trường xung quanh. Mỗi môi trường khác nhau sẽ có những yếu tố khác nhau, làm cho một rôbot phù với môi trường này không thể hoạt động trong môi trường khác. Chẳng hạn như một robot đang hoạt động tốt trong nhà, sử dụng Lidar có bán kính quét khoảng 5-10m, nhưng lại không thể đem ra ngoài trời để vận hành. Bởi vì ở ngoài trời, không gian mở rộng nên cảm biến laser sẽ không có tín hiệu phản hồi về, dẫn đến việc rôbot không thể xây dựng được bản đồ. Thậm chí trong cùng một môi trường, có thể có một số sự thay đổi cũng có thể làm cho rôbot khó hoạt động đúng như ban đầu đặt ra. Khi di chuyển trong một căn phòng có nhiều người, việc những vật cản động xuất hiện 1 lúc rồi biến mất là không thể tránh khỏi. Vấn đề này đã có thể giải quyết nhờ vào sự phát triển của các cảm biến cũng như các thuật toán xử lý dữ liệu và thuật toán xây dựng bản đồ. Khi phát hiện một vật cản, rôbot sẽ đưa các vật cản đó vào bản đồ, sau đó rôbot sẽ xóa vật cản đó khỏi bản đồ khi vật cản đó biến mất.

b) Có thể tự di chuyển mà không cần sự điều khiển của con người

Hiện nay, cùng với việc các cảm biến đã chính xác hơn, các thuật toán xử lý dữ liệu cũng được tối ưu hơn trước thì các thuật toán điều khiển di chuyển của rôbot cũng đã có những phát triển vượt bậc. Rôbot tự hành giờ đã di chuyển ổn định, chính xác hơn trước đây và hơn nữa trong quá trình di chuyển con người không cần phải lo lắng về việc có vật cản nào đó xuất hiện trên quỹ đạo di chuyển của rôbot như những rôbot công nghiệp trước đây.

c) Có thể xử lý với các sự thay đổi của môi trường

Các rôbot tự hành thông minh có thể tránh được những vật cản xuất hiện bất ngờ trong môi trường hoạt động thay đổi liên tục. Không chỉ vậy, khi tích hợp công nghệ AI và công nghệ xử lý tiếng nói lên rôbot, và có thể biết được bạn đang vui hay buồn, có thể biết được bạn đang nói gì với rôbot từ đó có

thể tự hoạt động sao cho thích hợp. Bằng việc đưa các Opensource vào việc lập trình ứng dụng cho rôbot sẽ giúp cho chúng ngày càng thông minh hơn, hỗ trợ con người nhiều hơn trong cuộc sống.

1.3.2 Phân loại rôbot tự hành thông minh

a) Ứng dụng trong ngành dịch vụ

Ứng dụng trong kho hàng thông minh

Cùng với sự phát triển và mở rộng của sản xuất công nghiệp, các nhà kho cũng dần dần được tự động hóa. Việc ứng dụng rôbot công nghiệp trong việc bốc xếp hàng hóa dần trở nên phổ biến. Nhưng Robot công nghiệp cũng có một số hạn chế:

- Khi rôbot hoạt động có thể gây nguy hiểm cho con người.
- Rôbot công nghiệp chỉ hoạt động theo một hoặc một vài quỹ đạo cũ thể, khi thay đổi cần phải lập trình lại.
- Mỗi rôbot thường chỉ phụ trách trong một khu vực nhất định.

Đó chính là lí do dẫn đến sự ra đời của nhà kho thông minh với các robot tự hành thông minh thực hiện chuyển đồ trong đó. Rôbot tự hành thông minh hay các xe tự hành có khả năng di chuyển linh hoạt, không gây nguy hiểm cho con người làm việc gần đó. Không những thế, rôbot tự hành có thể tránh được các vật cản động như con người, hay những vật di chuyển khác trong nhà kho. Đồng thời rôbot tự hành thông minh cũng có thể di chuyển khắp nhà kho mà không cần quan tâm đến vấn đề lắp đặt như rôbot công nghiệp thông thường. Ở Việt Nam hiện tại cũng đã ứng dụng rôbot tự hành thông minh trong các nhà kho thông minh. Xu hướng này đang chứng minh được ưu điểm và ngày càng có nhiều công ty sử dụng hơn. Xe tự hành của công ty STI Việt Nam là một loại rôbot tự hành được ứng dụng trong các kho hàng thông minh tại Việt Nam.

Ứng dụng trong việc hỗ trợ và hướng dẫn khách hàng

Một xu hướng phát triển mạnh mẽ của rôbot tự hành thông minh hiện nay là rôbot phục vụ các ngành dịch vụ. Con người hiện nay có đời sống ngày càng cao, ngày càng có nhiều đòi hỏi hơn trong đời sống hằng ngày. Vì thế, những rôbot tự hành thông minh dần dần được đưa vào các ngành dịch vụ nhằm nâng cao tiện ích cũng như tăng sự thu hút đối với khách hàng. Những rôbot hướng dẫn tại sân bay, trong các khách sạn hay trong các bảo tàng. Hay những rôbot bồi bàn trong các nhà hàng hỗ trợ gọi món ăn. Trong lĩnh vực dịch vụ này, rôbot tự hành thông minh gần như chiếm ưu thế tuyệt đối so với các rôbot khác. Nhờ vào khả năng di chuyển tránh vật cản linh hoạt mà robot tự hành thông minh có thể dễ dàng di chuyển trong một môi trường có nhiều người. Cùng với việc tích hợp các công nghệ tiên tiến hiện nay như công nghệ xử lý tiếng nói, chatbox, nhận diện con người, đồ vật bằng AI đã làm cho Robot tự hành thông minh ngày càng tiến xa hơn trong lĩnh vực này.

Rôbot AIRSTAR (H1.5) là một rôbot tự hành thông minh được sử dụng tại sân bay quốc tế Incheon Hàn Quốc. Robot này có thể giúp hành khách

tìm đường tới quầy làm thủ tục và nơi tiếp nhận hành lý. Ngoài ra nó cũng có thể chụp ảnh hay hướng dẫn tìm kiếm các quán ăn hay quán cà phê.



Hình 1.5 Rôbot AIRSTAR tại sân bay quốc tế Incheon Hàn Quốc [5]

Ứng dụng trong bệnh viện

Ngày nay, với sự phát triển của y học, các loại virus cũng biến đổi khó lường hơn, chúng có khả năng lây lan nhanh qua nhiều con đường khác nhau. Điển hình gần đây là dịch COVID-19. Vì thế, các đại dịch trong tương lai có thể vô cùng nghiêm trọng, việc cách ly các bệnh nhân sẽ cần phải nghiêm ngặt hơn. Và để hạn chế sự lây lan cho đội ngũ y bác sĩ, việc đưa các rôbot vào các khu cách ly để chuyển đồ và theo dõi sức khỏe bệnh nhân là một vấn đề cấp thiết. Cũng có nhiều rôbot được sử dụng vì mục đích này nhưng rôbot tự hành thông minh là loại rôbot hiệu quả nhất. Đầu tiên là rôbot tự hành thông minh có khả năng di chuyển ưu việt so với các loại rôbot khác hiện nay. Thứ hai là rôbot tự hành thông minh có thể tự tìm hiểu môi trường. Thứ ba là rôbot tự hành thông minh có thể ứng dụng AI để theo dõi sức khỏe bệnh nhân hằng ngày qua camera, ngay cả ban đêm.

Hơn 2 năm qua, tại Vũ Hán Trung Quốc đã bùng phát dịch COVID-19 và lây lan ra khắp thế giới. Vấn đề lây nhiễm dịch bệnh trong các bệnh viện trở thành một vấn đề nan giải. Chính vì vậy Trung Quốc đã bắt đầu sử dụng dần các rôbot tự hành thông minh để giảm tiếp xúc giữa người với người. Hay là sử dụng rôbot tự hành để đi khử trùng thay cho con người (H1.6).



Hình 1.6 Rôbot tự hành khử trùng tại bệnh viện Vũ Hán – Trung Quốc

Rôbot tự hành không chỉ được sử dụng trong bệnh viện tại các khu vực cách ly, mà còn được sử dụng trong việc hỗ trợ giảm stress cho các bệnh nhân có bệnh nan y, cần có thời gian điều trị lâu dài. Rôbot tự hành giúp bệnh nhân có thể gọi video call với người nhà, giúp bệnh nhân thư giãn bằng cách trò chuyện. Hay thậm chí việc có 1 con rôbot bên cạnh cũng giúp nhiều bệnh nhân khá lên. Rôbot tự hành cũng hỗ trợ trong việc vận chuyển các đồ vật trong bệnh viện.

b) Ứng dụng trong các môi trường đặc biệt

Ứng dụng trong việc thăm dò môi trường mới

Các công nghệ tự tìm quỹ đạo di chuyển và tránh vật cản trên Robot đã phát triển, nên các rôbot tự hành thông minh đang bắt đầu được ứng dụng để di chuyển ngoài trời. Robot tự hành thông minh có thể di chuyển trong các môi trường không biết trước, điều mà các rôbot trước đây khó có thể thực hiện được. Vì vậy, rôbot tự hành đã bắt đầu được ứng dụng để khám phá các môi trường mới, nơi có điều kiện khác nghiệt với con người như ngoài vũ trụ, dưới đáy biển...



Hình 1.7 Robot Mar Rover 2020 khám phá sao Hỏa [14]

Robot Mar Rover 2020 (H1.7) là Robot được sử dụng cho dự án khám phá Sao Hỏa năm nay của NASA. Robot này sẽ thu thập các thông tin trên mặt đất nhằm tìm kiếm dấu vết của sự sống trên Sao Hỏa. Mars 2020 là một rôbot tự hành chạy bằng năng lượng hạt nhân với nhiệm vụ thăm dò Sao Hỏa bởi Chương trình thám hiểm Sao Hỏa của NASA đã phóng vào tháng 7 năm 2020.

Ứng dụng trong các công việc trong môi trường độc hại

Khi làm việc trong các điều kiện độc hại như phòng hóa chất, khu vực nhiễm phóng xạ... con người thường dùng các rôbot để thay thế. Làm giảm các tác động tiêu cực đến sức khỏe công nhân. Trước đây những cánh tay rôbot với khả năng di chuyển chính xác theo quỹ đạo được lập trình trước là giải pháp hữu hiệu. Những cánh tay rôbot này cũng cho thấy nhiều ưu điểm. Nhưng nó vẫn gặp phải vấn đề cơ bản của rôbot công nghiệp cũ là chỉ có thể di chuyển theo quỹ đạo được lập trình sẵn, không có tính linh hoạt. Và các cánh tay này cũng cần phải lắp đặt tương đối phức tạp. Do đó, rôbot tự hành cũng được sử dụng trong việc này, tuy còn hạn chế trong một số khả năng

nhưng có lẽ trong tương lai robot tự hành thông minh sẽ thay thế các cánh tay robot trong lĩnh vực này. Có thể kể tên một số dòng robot tự hành kết hợp tay thao tác nổi bật được sử dụng trong việc rà phá bom mìn cực kỳ nguy hiểm như: Talon IV (H1.8) của Iraq, Caliber MK4 của Mỹ hay Uran-6 của Nga.



Hình 1.8 Robot Talon IV rà phá bom mìn [14]

Ứng dụng trong gia đình

Cùng với sự đi lên của đời sống con người thì các vấn đề liên quan đến gia đình cũng ngày càng nhiều. Cuộc sống hiện đại luôn kèm với công việc bận rộn, do đó các bậc cha mẹ ngày càng dành ít thời gian cho con cái. Những người cao tuổi hoặc vào trung tâm dưỡng lão, hoặc ở nhà với rất ít thời gian cùng con cái. Với sự phát triển của máy tính và điện thoại thông minh cũng giúp con người giải trí, nhưng lại tạo ra vấn đề nghiện game, sống ảo... Đó là lý do mà robot tự hành thông minh được đưa vào trong các gia đình. Robot giúp quan sát người con cái, chăm sóc người già thay các bậc cha mẹ. Chúng là người bạn trò chuyện cho những người cao tuổi giúp họ vui bớt sự cô đơn. Và đặc biệt robot là người bạn của những đứa trẻ, nguồn tạo ra sự thích thú với khoa học. Là người bạn chơi cùng, học cùng những đứa trẻ. Robot Kuri (H1.9) là một robot điển hình trong gia đình. Nó có hình dáng đáng yêu, có camera cùng với khả năng di chuyển thông minh của một robot tự hành.



Hình 1.9 Robot Kuri đang chơi với trẻ [9]

Kuri là một rôbot tự hành thông minh với kích thước nhỏ và ngoại hình đáng yêu. Nó hỗ trợ cho việc dạy học cho con cái và một số chức năng giải trí khác trong gia đình.

1.4 Tay thao tác

1.4.1 Định nghĩa và phân loại

a) Định nghĩa

Có nhiều định nghĩa tay máy (tay thao tác) cùng tồn tại, chúng ta hãy cùng tham khảo một số định nghĩa như sau:

Định nghĩa theo từ điển New World College

“Tay máy là một kết cấu cơ khí có hình dạng bất kì, được xây dựng để thực hiện những công việc bằng tay của con người”.

Các định nghĩa sau này bao gồm các cánh tay cơ khí, các máy móc điều khiển số, các máy móc di chuyển theo kiểu bước đi và mô phỏng hình dáng con người. Các robot công nghiệp ngày nay chỉ thực hiện một phần công việc của con người. Các tay máy ban đầu thường được gọi là các tay máy (Manipulator).

Định nghĩa theo hiệp hội rôbot công nghiệp Nhật Bản

Định nghĩa này mang tính khái quát nhất của tất cả các định nghĩa được sử dụng. Nó bao gồm tất cả các thiết bị tay máy và có thể xem khi định nghĩa một robot sau này.

“Tay máy là một máy, cơ cấu thường gồm một số bộ phận phân đoạn được nối với phân đoạn khác bằng khớp quay hay khớp trượt nhằm mục đích để gấp hay di chuyển các đối tượng, thường có một số bậc tự do. Nó có thể điều khiển bởi một nguồn kích hoạt, một hệ thống điện tử có thể lập trình được hay một hệ thống logic nào đó”.

Định nghĩa theo tiêu chuẩn AFNOR (Pháp)

“Tay máy là một cơ cấu chuyển đổi tự động có thể chương trình hóa, lập lại các chương trình, tổng hợp các chương trình đặt ra trên sườn các trục tọa độ, có khả năng định vị, định hướng, di chuyển các đối tượng vật chất (chi tiết, dụng cụ gá lắp.v.v.) theo những hành trình thay đổi đã chương trình hóa nhằm thực hiện các nhiệm vụ công nghệ khác nhau”.

Định nghĩa theo hiệp hội rôbot công nghiệp Hoa Kỳ

“Tay máy là một tay máy có nhiều chức năng có thể lập trình, được thiết kế để di chuyển vật liệu, các phần tử, linh kiện, các dụng cụ và thiết bị đặc biệt thông qua việc thay đổi các chương trình hoạt động đã được lập để thực hiện các nhiệm vụ khác nhau”.

Định nghĩa theo hiệp hội rôbot Anh

“Tay máy công nghiệp là một thiết bị có thể được lập trình lại, được thiết kế để thực hiện hai nhiệm vụ cầm nắm và vận chuyển các phần tử, linh kiện, các dụng cụ hoặc các công cụ chế tạo đặc biệt thông qua việc thay đổi các chương trình hoạt động đã được thiết lập để thực hiện các tác vụ gia công khác nhau”.

Định nghĩa GOST (Nga)

“Tay máy là một máy tự động liên kết giữa một tay máy và một cụm điều khiển chương trình hóa, thực hiện một chu trình công nghệ một cách chủ động với sự điều khiển có thể thay thế những chức năng tương tự con người”. Như vậy, qua các định nghĩa trên, ta có thể hiểu đơn giản tay máy công nghiệp là một loại thiết bị được thiết kế và chế tạo để phục vụ một số hoạt động trong quá trình sản xuất, thông qua các chương trình được thiết lập sẵn, và có thể thay đổi tùy vào mục đích sử dụng.

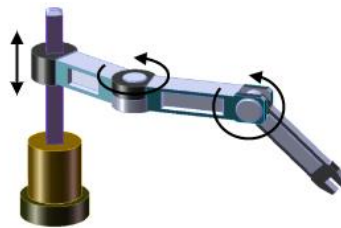
b) Phân loại

Việc phân nhóm, phân loại rôbot có thể dựa trên những yếu tố kỹ thuật khác nhau. Dưới đây là một số cách phân loại chủ yếu:

Phân loại theo số bậc tự do

Định nghĩa

Bậc tự do của cơ cấu là số thông số độc lập cần cho trước để xác định hoàn toàn vị trí của các khâu trong cơ cấu khi cơ cấu hoạt động. Điều đó có nghĩa mỗi thông số độc lập sẽ là một quy luật cho trước để xác định quy luật chuyển động của cơ cấu. Khâu có quy luật chuyển động cho trước được gọi là khâu dẫn, khâu dẫn được nối với giá bằng một khớp loại 5 vì khớp loại 5 chỉ có một thông số xác định.



Hình 1.10 Robot 3 bậc tự do [2]

Trong H1.10 là mô hình rôbot 3 bậc tự do gồm 1 bậc tự do tịnh tiến và 2 bậc tự do quay. Công thức tính số bậc tự do của cơ cấu được trình bày như ví dụ sau.

Đối với rôbot là dạng chuỗi động hở như rôbot H1.10 đang xét thì số bậc tự do có thể tính theo công thức sau:

$$W = 6n - \sum_{i=1}^5 ip_i$$

Trong đó: n là số khâu động

p_i là số khớp loại i ($i=1,2,...,5$: số bậc tự do bị hạn chế)

Áp dụng vào rôbot H1.10: Có 4 khâu động, 3 khớp quay loại 5 và 1 khớp trượt loại 5.

$$W = 6*4 - (4*5) = 4 \text{ (bậc tự do)}$$

Một rôbot sẽ linh hoạt hơn khi di chuyển và hoạt động trong một không gian bị hạn chế. Mặt khác, trong một số ứng dụng đặc biệt như trong việc lắp ráp trong một mặt phẳng thì chỉ cần bốn bậc tự do.

Phân loại theo cấu trúc động học

Một rôbot được gọi là rôbot tuần tự hay rôbot chuỗi hở nếu cấu trúc động học của chúng có dạng một chuỗi động học hở, gọi là rôbot song song nếu cấu trúc động học của chúng có dạng một chuỗi đóng và gọi là rôbot hỗn hợp nếu nó bao gồm hai loại chuỗi hở và chuỗi đóng.



Hình 1.11 Robot song song [4]

Nhìn nhận một cách tổng quát thì rôbot song song H1.11 có nhiều ưu điểm vì chúng có độ cứng vững cao hơn, khả năng tải cao hơn, nhưng không gian làm việc nhỏ hơn và cấu trúc phức tạp hơn. Tuy nhiên trong các đơn vị công nghiệp lắp ráp, hàn xì, đúc, .v.v. thường sử dụng các rôbot chuỗi động hờ vì khả năng vận hành linh hoạt và thiết kế đơn giản.



Hình 1.12 Robot hàn chuỗi hờ [5]

Như trong hình mô tả về rôbot hàn chuỗi hờ, nó có thể tùy ý thao tác các hoạt động trong một không gian làm việc rộng và điều chỉnh được hướng và vị trí của mũi hàn.

Phân loại theo nguồn động lực sử dụng

Nguồn năng lượng điện (động cơ điện)

Thường sử dụng các động cơ điện một chiều, các động cơ bước (H1.4) hay động cơ Servo.

Các loại động cơ này có đặc điểm là hoạt động chính xác, tin cậy, đạt công suất cao và có tính tuyến tính cao để điều khiển. Hệ này cũng đảm bảo kết cấu gọn, truyền dẫn năng lượng trực tiếp. Ngoài ra nó còn đảm bảo an toàn vệ sinh môi trường.



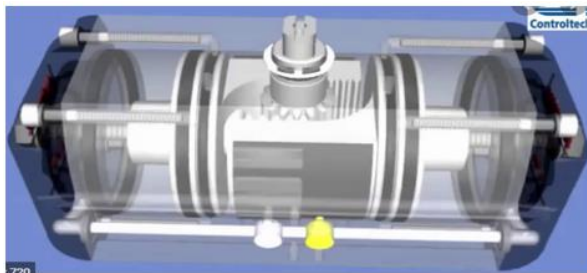
Hình 1.13 Cấu tạo động cơ bước (Step Motor) [3]

Nhằm nâng cao chất lượng là hiệu quả sử dụng đối với hệ này cần tuân theo những nguyên tắc cơ bản sau:

- Sử dụng các công nghệ mới, các loại vật liệu mới ít chịu ảnh hưởng bởi từ trường Trái Đất.
- Tiếp tục nâng cao công suất và hiệu suất công tác.
- Xử lý tốt các cụm ghép nối trong mạch nguồn, mạch điều khiển và hiệu chỉnh nâng cao hơn nữa độ tin cậy.

Nguồn năng lượng thủy, khí (động cơ thủy lực, khí nén)

Động cơ thủy lực có thể đạt đến công suất cao, đáp ứng những yêu cầu làm việc nặng. Tuy nhiên, hệ thống thủy lực thường cồng kềnh, yêu cầu dòng dầu, chất lượng dầu cao, hơn nữa vận tốc phải có độ phi tuyến lớn, khó đảm bảo độ chính xác cao khi điều khiển.



Hình 1.14 Động cơ khí nén [3]

Hệ khí nén làm việc với công suất trung bình và nhỏ có kết cấu đơn giản. Điều khiển phải gắn liền với trung tâm khí nén, kém chính xác. Thích hợp cho các loại robot hoạt động theo chương trình định sẵn với các thao tác đơn giản kiểu nâng lên hạ xuống.

Phân loại theo hệ thống truyền động

Hệ truyền động gián tiếp

Các cơ cấu chấp hành được nối với nguồn động lực thông qua các bộ truyền động cơ khí thường gặp như hệ bánh răng thường, hệ bánh răng hành tinh, hệ bánh răng song, dây đai răng (H1.15) bộ truyền xích hay là bộ truyền vít me đai ốc bi.v.v.



Hình 1.15 Bộ truyền bánh răng – đai răng [7]

Nhược điểm của hệ này là tạo mòn khe hở động học dẫn đến phi tuyến và hiệu ứng trễ ngày càng cao hơn. Mặt khác hiệu suất sẽ giảm do tiêu hao công suất trên bộ truyền. Do quá trình hoạt động với cường độ làm việc liên tục và ảnh hưởng của lực ma sát gây nên những hư hỏng liên quan đến bộ truyền trung gian như mẻ răng, mòn răng, dẫn đai, v.v. làm ảnh hưởng tới hiệu suất và chất lượng hoạt động của các robot công nghiệp.

Hệ truyền động trực tiếp

Các cơ cấu chấp hành được nối trực tiếp với nguồn động lực, do đó kết cấu sẽ gọn nhẹ và hạn chế, loại bỏ được những nhược điểm khi truyền động gián tiếp. Mặt khác khó khăn đặt ra là phải thiết kế chế tạo các động cơ có số vòng quay thích hợp cho phép điều khiển vô cấp trên một dải rộng.

1.4.2 Ứng dụng của tay thao tác

Như đã nêu trong mục 1.2a, ứng dụng của các tay máy rôbốt rất đa dạng và phổ biến trong nhiều lĩnh vực khác nhau. Đặc biệt là trong những mảng sản xuất công nghiệp. Các tay máy rôbốt được sử dụng bởi những tính năng vô cùng vượt trội mà bàn tay con người có thể chưa đạt tới mức độ như vậy. Dưới đây là một số lợi ích mà tay rôbốt (tay thao tác) đem lại trong sản xuất công nghiệp nói riêng.

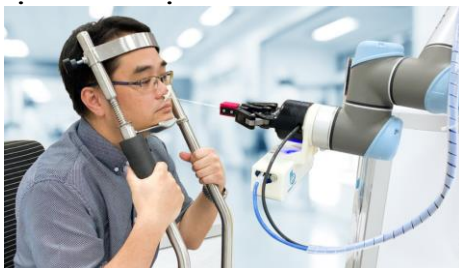
- Thao tác nhanh, tiết kiệm thời gian;
- Năng suất, độ an toàn cao;
- Chuyển động chính xác.

Không chỉ được sử dụng trong các ngành sản xuất công nghiệp, hiện nay các tay thao tác càng được sử dụng nhiều trong đời sống. Trong lĩnh vực y tế, việc áp dụng tay thao tác giúp sức cho các y bác sĩ rất nhiều: tay thao tác thực hiện phẫu thuật cho bệnh nhân (H1.16); rôbốt lấy mẫu xét nghiệm COVID-19 (H1.17),... Ngoài việc hỗ trợ các bác sĩ thực hiện những ca phẫu thuật khó, việc áp dụng các rôbốt vào y khoa giúp giảm thiểu tối đa việc lây nhiễm bệnh dịch từ bệnh nhân sang người khác.



Hình 1.16 Rôbốt hỗ trợ thực hiện phẫu thuật [11]

Đặc biệt, phẫu thuật tuyến tiền liệt (H1.16) là một trong những chỉ định phổ biến nhất của phẫu thuật rôbốt. Bởi vùng phẫu thuật khá sâu và nhiều góc khuất. Dưới sự giúp sức của camera và 3 cánh tay rôbốt có thể xoay 580 độ, người phẫu thuật viên có thể thực hiện các thao tác dễ dàng hơn mổ nội soi hay mổ hở. Các bác sĩ có thể loại bỏ khối u một cách triệt để, kể cả ở những góc khuất, góc hẹp nhất. Đặc biệt, với phẫu thuật cắt khối u, rôbốt có thể giúp lấy hết khối u một cách triệt để.



Hình 1.17 Rôbốt Swatbot lấy mẫu xét nghiệm COVID-19

Trên H1.17 là hình ảnh rôbot SwatBot được các nhà nghiên cứu Singapo chế tạo. Các bác sĩ lâm sàng tham gia chế tạo robot SwabBot cho biết quy trình lấy mẫu do rôbot này tiến hành có độ an toàn cao hơn, nhanh hơn và thoải mái hơn so với phương pháp lấy mẫu xét nghiệm dịch mũi bằng tay. Mặc dù các quốc gia khác đã phát triển các loại rôbot tương tự, nhưng SwabBot cho phép người được lấy mẫu kiểm soát hoàn toàn quá trình lấy mẫu để họ cảm thấy thoải mái hơn. Theo đó, người được lấy mẫu xét nghiệm có thể khởi động hoặc kết thúc máy theo ý muốn.

Theo các chuyên gia, SwabBot có thể giải quyết những hạn chế và rủi ro của việc lấy mẫu dịch nhầy ở mũi bằng tay, làm giảm nguy cơ người lấy mẫu tiếp xúc với virus SARS-CoV-2, giảm sức ép phải tăng cường đào tạo nhân viên lấy mẫu xét nghiệm, tiêu chuẩn hóa sự đồng bộ của mẫu xét nghiệm cũng như tăng hiệu quả của việc tiến hành xét nghiệm.

1.5 Kết hợp rôbot tự hành thông minh và tay thao tác

Với những đặc điểm cũng như những lợi ích mà rôbot tự hành và các tay thao tác mang đến là vô cùng hữu dụng. Khi kết hợp hai thiết bị này với nhau thì ta sẽ được một sản phẩm mà ứng dụng của nó càng rộng rãi và độ khó của yêu cầu hoạt động thực thi sẽ tăng lên một cách đáng kể. Nó có thể đảm nhận những công việc có yếu tố đặc thù mà con người không thể làm được. Có thể nói đến là những rôbot tự hành mang theo tay thao tác thực hiện nhiệm vụ trong môi trường độc hại và nguy hiểm như: nhà máy hóa chất, nhà máy phóng xạ, khu rà phá bom mìn,...

Việc áp dụng cả hai thiết bị này đã được các quốc gia hàng đầu thế giới triển khai trong những năm gần đây, Và quả thật những lợi ích mà nó mang đến vô cùng to lớn. Đây chính là mẫu thiết bị mà nhóm đồ án đặt mục tiêu hướng đến trong đồ án tốt nghiệp này.

1.6 Phương pháp thực hiện đề tài

1.6.1 Mục tiêu của đề tài

Xuất phát từ khoảng thời gian diễn ra dịch COVID-19 2 năm trở lại đây, nhóm nhận thấy đây là một loại dịch bệnh lây nhiễm thông qua tiếp xúc giữa người với người, người với các mẫu vật phẩm hoặc bệnh phẩm nhiễm virus. Việc đó không chỉ gây lây lan giữa các người bệnh mà còn lây nhiễm cho các y bác sĩ chữa trị. Chính vì vậy, nhóm đặt ra nhiệm vụ cho đồ án là thiết kế chế tạo một sản phẩm có thể thu gọn, gom các rác thải y tế nhằm hạn chế sự lây lan có thể xảy ra của các dịch bệnh trong tương lai.

Mục tiêu của đề tài chính là nghiên cứu, thiết kế, chế tạo và điều khiển tay thao tác di động làm việc trong một không gian rộng (nhà máy, nhà kho, bệnh viện...) với nhiệm vụ của rôbot là di chuyển giữa các vị trí khác nhau trong không gian đó và thu nhặt các rác thải y tế ở địa điểm đích mà rôbot hướng tới làm việc.

1.6.2 Yêu cầu đề bài

Với mục tiêu đề tài của nhóm là thiết kế tay thao tác di động phục vụ trong y tế; và sản phẩm bao gồm 2 phần chính: tay thao tác và rôbot tự hành

thông minh (rôbot Kobuki). Dưới đây là các yêu cầu và nhiệm vụ chính của nhóm khi thiết kế, chế tạo mô hình tay thao tác.

Yêu cầu thông số kỹ thuật của cánh tay thao tác:

- Tải trọng: 500 (g);
- Tầm với: 350 (mm).
- Vận tốc bàn tay gấp: 0.25 (m/s).

Yêu cầu đạt được của tay thao tác:

- Tìm hiểu về lịch sử phát triển của tay thao tác;
- Thiết kế, xây dựng mô hình 3D tay thao tác bằng phần mềm SolidWorks;
- Tính toán, lựa chọn động cơ và bộ truyền động cho các khâu của cánh tay rôbot;
- Tính toán bài toán động học của tay thao tác (bao gồm bài toán động học; lập bảng D-H);
- Kiểm tra độ bền các khâu;
- Tìm hiểu và lựa chọn các chi tiết khác cho mô hình;
- Xây dựng mô hình thực tế tay thao tác.
- Thiết kế, xây dựng mạch nguyên lý và mạch điều khiển cho mô hình tay thao tác;
- Chế tạo và dán các linh kiện lên mạch PCB;
- Thực hiện đi dây các linh kiện khác vào mạch điều khiển.

Sau khi đạt được các yêu cầu trên, nhóm cần thực hiện quá trình kiểm thử các tính năng hoạt động riêng lẻ của tay thao tác bằng các kiểm tra hoạt động của từng khâu (kiểm tra góc xoay và tải trọng trên đó). Sau đó, thực hiện quá trình kết nối giữa tay thao tác và rôbot tự hành thông minh Kobuki.

1.6.3 Phương pháp thực hiện đề tài

Phương pháp quan sát thực tế:

Nhóm chúng em đã có thời gian tiếp cận với những sản phẩm có tính chất tương tự như: các AGV trong các nhà máy; các tay máy công nghiệp (tay máy Nachi, ABB, Kuka) ; các rôbot tự hành (rôbot Kobuki)

Phương pháp nghiên cứu lý thuyết:

Trong thời gian nghiên cứu, nhóm đồ án đã tìm hiểu qua các sách tài liệu, các đồ án sinh viên khóa trước, các trang web, ... liên quan đến đề tài rôbot phục vụ trong ngành y tế. Từ quá trình bên trên, nhóm đồ án đã quyết định tiến hành xây dựng cấu trúc một tay thao tác để thực hiện gấp nhả các sản phẩm. Để đạt được việc thiết kế, chế tạo và điều khiển được một tay thao tác di động, thì những vấn đề sau cần được tìm hiểu và nghiên cứu:

- Bài toán dò đường sử dụng Lidar thông qua hệ điều hành ROS của rôbot Kobuki;
- Bài toán thiết kế tay thao tác, tính toán bài toán độ bền bằng phần mềm SolidWorks;
- Tính toán bài toán động học xác định mối liên hệ giữa các chuyển động và tìm quy luật thay đổi vị trí, quy luật chuyển hướng của tay thao tác;

- Sự liên kết giữa tay thao tác và rôbốt tự hành thông minh (Kobuki);
- Bài toán điều khiển động cơ giảm tốc và điều khiển động cơ bước;
- Bài toán thiết kế mạch điều khiển cho tay thao tác bằng phần mềm Altium.

Sau khi đã tìm hiểu và nghiên cứu, việc tiến hành làm ra sản phẩm cũng rất quan trọng. Khi sản phẩm được hoàn thành thì sản phẩm phải đạt được những tiêu chí sau:

- Thiết kế được mô hình cơ khí và kiểm tra bền của cánh tay thao tác;
- Lắp ráp và hiệu chỉnh tay thao tác;
- Rôbốt Kobuki hoạt động theo đường thiết lập trên bản đồ đã quét;
- Thiết kế mạch điều khiển, hàn các linh kiện và kết nối dây với các thiết bị;
- Điều khiển rôbốt hoạt động chính xác.

CHƯƠNG 2- THIẾT KẾ, CHẾ TẠO VÀ TÍNH TOÁN TAY THAO TẮC

2.1 Tính toán, lựa chọn động cơ và bộ truyền cho các khâu

Dựa vào yêu cầu bài toán được giao như:

- Tải trọng: 0.5 (kg);
- Tầm với: 0.35 (m);
- Vận tốc bàn tay kẹp: 0.25 (m/s).

Nhóm đã đưa ra một số thông số dự kiến về kích thước các khâu như:

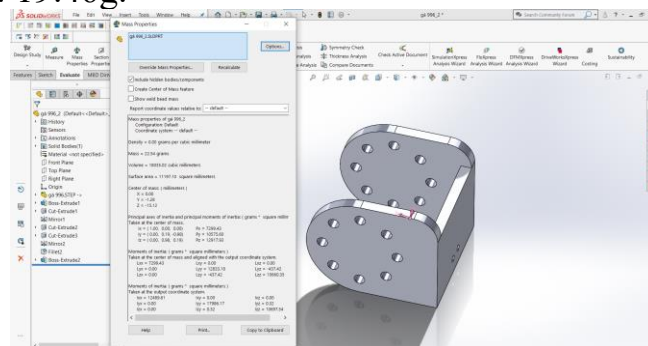
- Tay thao tác: 4 bậc tự do (RRRR);
- Chiều dài cánh tay khâu 2: 200 (mm);
- Chiều dài cánh tay khâu 3: 150 (mm);
- Góc quay giới hạn của khớp quay thứ 1: $-3\pi/4 < \theta_1 < 3\pi/4$;
- Góc quay giới hạn của khớp quay thứ 2: $-\pi/4 < \theta_2 < \pi/3$;
- Góc quay giới hạn của khớp quay thứ 3: $0 < \theta_3 < 3\pi/4$;
- Góc quay giới hạn của khớp quay thứ 4: $-\pi/2 < \theta_4 < \pi/2$.

Dựa vào các kích thước dự kiến như trên và các thông số của một số chi tiết như động cơ, các bộ truyền, nhóm sẽ đưa ra các tính toán và kiểm tra phù hợp để lựa chọn các chi tiết cho tay thao tác.

2.1.1 Cụm bàn tay kẹp

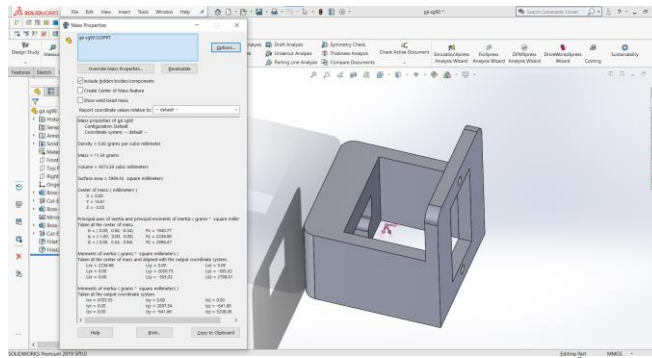
Như đã nêu ở mục 2.2, cụm tay gấp bao gồm gá tay gấp, gá động cơ servo và tay gấp (đều được in bằng nhựa PLA có khối lượng riêng 1.25g/cm^3) và 2 động cơ servo SG90. Theo tính toán trên phần mềm SolidWorks thì khối lượng cụm tay gấp này bao gồm:

- Động cơ servo SG90: 9g/cái;
- Gá cụm tay gấp (H2.1): 22.54g;
- Gá động cơ servo (H2.2): 11.34g;
- Bàn tay kẹp: 19.46g.



Hình 2.1 Gá cụm tay gấp dự kiến

Gá tay gấp được thiết kế đối xứng với các lỗ bố trí hình tròn để có thể xoay các góc so với cánh tay khâu 3, ngoài ra phía đầu gá còn được khoét 2 lỗ để cố định cảm biến cân nặng Loadcell 100g.



Hình 2.2 Giá động cơ servo dự kiến

Như vậy, tổng khối lượng bàn tay gấp là:

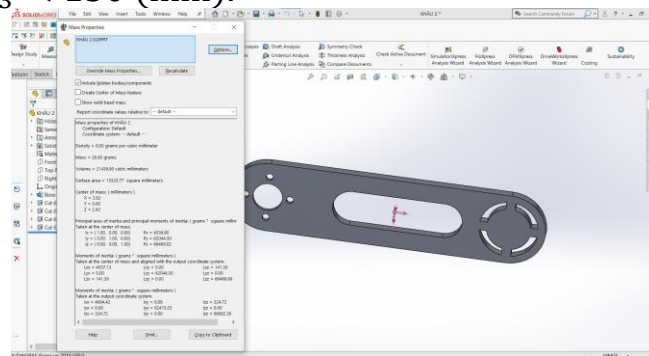
$$m_4 = 9 \times 2 + 22.54 + 11.34 + 19.46 = 71.34 \text{ (g)}$$

2.1.2 Tính toán động cơ và bộ truyền khớp quay thứ 3

a) Tính chọn động cơ

Thông số đầu vào:

- Khâu 4: 71.34 (g) (bao gồm bàn tay kẹp, 2 động cơ SG90, giá bàn tay kẹp);
- Tải trọng vật cần nâng: 500 (g);
- Khối lượng khâu 3: $26.8 \times 2 = 53.6$ (g) (theo tính toán của phần mềm SolidWorks H2.3)
- Khoảng cách trọng tâm tổng khối lượng khâu 3 và bàn tay kẹp đến động cơ: $d_3 < 150$ (mm).



Hình 2.3 Tính khối lượng dự kiến 1 chi tiết khâu 3

Như vậy tổng khối lượng của khâu 3 và bàn tay kẹp 4 là:

$$m_{34} = m_3 + m_4 + \text{tải} = 53.6 + 71.34 + 500 = 624.94 \text{ (g)} \\ = 0,625 \text{ (kg)}$$

Momen nhỏ nhất của động cơ khâu 3 là:

$$M_3 = m_{34} \times g \times d_{34} = 0,625 \times 10 \times 0,15 = 0,94 \text{ (Nm)}.$$

Với hệ số an toàn lựa chọn là $k = 2$, như vậy mômen cần thiết là:

$$M'_3 = M_3 \cdot k = 0,94 \cdot 2 = 1,88 \text{ (Nm)}$$

Cần chọn loại động cơ có momen lớn hơn hoặc bằng M'_3 . Ngoài ra loại động cơ này cần có khối lượng nhỏ để giảm trọng lượng cánh tay robot.

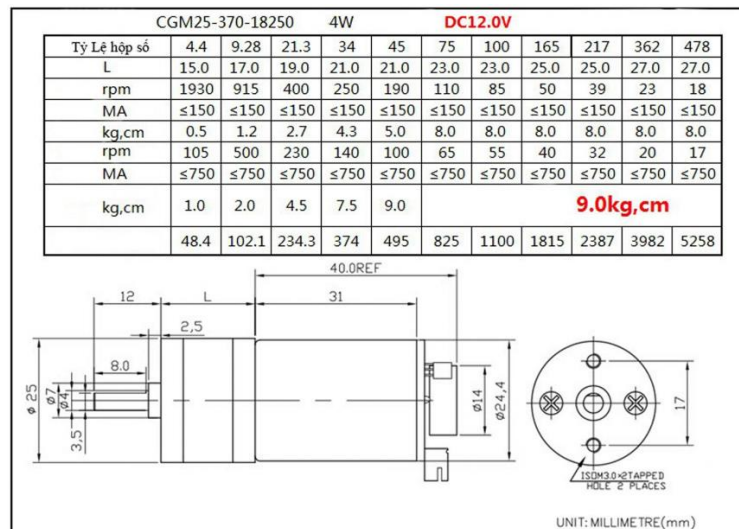


Hình 2.4 Động cơ Giảm tốc GA25 Encoder [4]

Vì vậy nhóm lựa chọn 1 động cơ Giảm tốc GA25 Encoder để thực hiện nhiệm vụ đặt ra, với một số ưu điểm và thông số như sau:

Giảm tốc GA25 Encoder kết hợp với bộ truyền bánh răng đai răng giúp tăng momen xoắn lên rất lớn phù hợp cho các loại rôbốt công nghiệp, cơ cấu máy, các bộ phận nhỏ những cần lực lớn. Đối với motor bước thường xuyên phải đảo chiều quay, tăng giảm tốc độ hoặc dừng đột ngột nên bộ truyền hoặc hộp số phải là loại chính xác và chịu được momen lớn. Trong một số trường hợp đặc biệt khi cần dừng đột ngột thì phải sử dụng thêm phanh từ.

Và lựa chọn của nhóm là bộ truyền bánh răng đai răng có tỷ số truyền 1:3. Như vậy momen kéo của động cơ kết hợp bộ truyền là $0.9 \cdot 3 = 2.7 \text{ Nm}$.



Hình 2.5 Thông số động cơ giảm tốc GA25 Encoder [4]

b, Tính toán bộ truyền bánh răng – đai răng khớp quay thứ 3

Ta sử dụng bộ truyền đai răng - bánh răng cho việc truyền động cho khâu 3.

Vận tốc bàn tay kẹp theo yêu cầu là: $v = 0.25 \text{ (m/s)}$, với cánh tay đòn dự kiến là 200 (mm), như vậy vận tốc góc khâu 3 là:

$$\omega_3 = \frac{v}{R} = 1.3 \text{ (rad/s)} = 12 \text{ (vòng/phút)}.$$

Với: n_d – Hiệu suất bộ truyền đai răng, $n_d = 0,97$;

n_{ol} – Hiệu suất 1 cặp ổ lăn, $n_{ol} = 0,99$;

Hiệu suất thực tế: $\mu = n_d \cdot n_{ol} = 0,99 \cdot 0,97 = 0,96$.

Công suất trên trục 3: $W_{t3} = M_3 \frac{\omega_3}{\mu} = 0,94 \cdot \frac{12}{0,96} = 11,75(W)$.

Vì vậy nhóm lựa chọn động cơ Giảm tốc GA25 Encoder và bộ truyền đai răng bánh răng với tỷ số truyền 1:3 để thực hiện nhiệm vụ đặt ra.

Công suất trên động cơ khâu 3 là: $W_{đc3} = W_{t3} \div 3 = 4 (W)$.

Bước răng xác định bằng công thức:

$$p \geq 35 \sqrt[3]{\frac{P_3}{n_3}} = 35 \sqrt[3]{\frac{0,004}{12}} = 2,4 (mm)$$

Trong đó:

P_3 – công suất trên bánh đai chủ động

$$(P_3 = 4(W) = 0,004 (kW))$$

n_3 – tốc độ quay của bánh đai chủ động ($n_1 = 12$ (vòng/phút))

Chọn $p = 3 (mm)$, suy ra $m = p/\pi = 0,96$

Chọn $m = 1$.

Xác định chiều rộng đai

Theo [2], chiều rộng đai xác định như sau:

$$b = \psi_d m$$

Trong đó: ψ_d – Hệ số chiều rộng đai. ψ_d dao động từ 6 đến 9.

Bảng 4.28. Chiều rộng đai răng b

Môđun của đai m, mm	Chiều rộng đai răng b, mm
1	3; 4; 5; 8; 10; 12,5
1,5	3; 4; 5; 8; 10; 12,5; 16; 20
2	5; 8; 10; 12,5; 16; 20
3	12,5; 16; 20; 25; 32; 40; 50
4	20; 25; 32; 40; 50; 63; 80; 100
5	25; 32; 40; 50; 63; 80; 100
7	40; 50; 63; 80; 100; 125
10	50; 63; 80; 100; 125; 160; 200

Hình 2.6 Chiều rộng đai răng [7]

Dựa vào bảng (H2.6), [7]; Chiều rộng đai $b = 5 (mm)$.

Chiều rộng bánh đai: $B = b + m = 5 + 1 = 6 (mm)$.

Xác định các thông số của bộ truyền

Bảng 4.29. Số răng của bánh đai răng nhỏ z_1

Môđun của đai m, mm	Số vòng quay bánh đai nhỏ n, vg/ph	z_1 không nhỏ hơn	Môđun của đai m, mm	Số vòng quay bánh đai nhỏ n, vg/ph	z_1 không nhỏ hơn
2	500 ... 3000	12	5	500	16
	3500 ... 4500	14		1000 ... 1500	18
	5000 ... 6500	16		2000 ... 3000	20
	7000 ... 7500	18		3500 ... 4000	22
3	500 ... 1000	12	6	500	20
	1500 ... 2000	14		1000	22
	2500 ... 3500	16		1500	24
	4000 ... 5000	18		2000	26
4	500	12* ; 14	7	500	20
	1000	14* ; 16		1000	22
	1500 ... 2000	16* ; 18		1500	24
	2500 ... 3500	18* ; 20		2000	26 ... 28

Chú thích : * Đối với đai dùng dây kim loại.

Hình 2.7 Bảng tra số răng [7]

Số răng z_1 của bánh đai nhỏ được chọn theo bảng (H2.7) nhằm đảm bảo tuổi thọ cho đai. Số răng của bánh đai lớn $z_2 = u \cdot z_1$ với $u = n_1/n_2 = z_2/z_1$. Với tỷ số truyền $u = 3$.

Chọn $z_1 = 20$ (răng). $z_2 = u \cdot z_1 = 3 \cdot 20 = 60$ (răng).

Khoảng cách trục a được chọn theo điều kiện:

$$a_{min} \leq a \leq a_{max}$$

Trong đó:

$$a_{min} = 0,5m(z_1 + z_2) + 2m = 42(mm);$$

$$a_{max} = 2m(z_1 + z_2) = 160(mm).$$

Số răng đai

$$z_d = \frac{2a}{p} + \frac{z_1 + z_2}{2} + \frac{(z_2 - z_1)^2 p}{40a} = \frac{2a}{p} + \frac{z_1 + z_2}{2} + \frac{(z_2 - z_1)^2 p}{40a}$$

$$= 70 \div 147$$

Chọn $z_d = 108$ (răng);

Thay $z_d = 108$ (răng) ta có: $a = 100$ (mm).

Chiều dài đai:

$$l_d = p \cdot z_d = 2.108 = 216 \text{ (mm)}$$

Góc ôm đai:

$$\alpha_1 = 180^\circ - \left[\frac{m(z_2 - z_1)}{a} \right] \cdot 57,3^\circ = 157,1^\circ$$

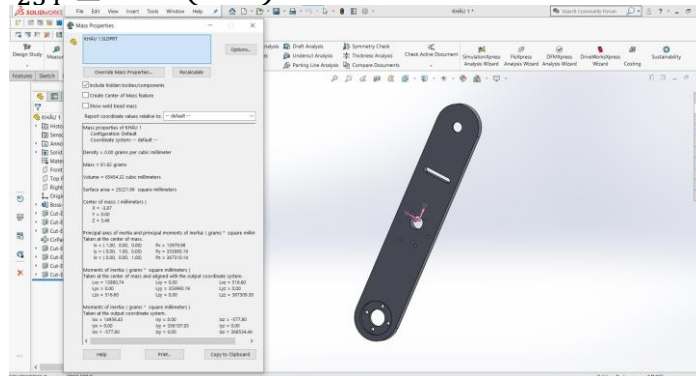
Số răng đồng thời ăn khớp trên bánh đai nhỏ:

$$z_{03} = \frac{z_1 \alpha_1}{360} = 5,9 \text{ (răng)}$$

2.1.3 Tính toán động cơ và bộ truyền khớp quay thứ 2

Thông số đầu vào:

- Khối lượng khâu 2: $81.82 \times 2 = 163.64$ (g) (theo tính toán của phần mềm SolidWorks H2.8);
- Khối lượng khâu 3,4 là 625 (g);
- Khối lượng động cơ khớp quay thứ 3 và các chi tiết của bộ truyền đai: 200g;
- Khoảng cách trọng tâm tổng khối lượng khâu 2,3, tay kẹp đến trục momen: $d_{234} \leq 200$ (mm).



Hình 2.8 Tính khối lượng khâu 2 dự kiến

Như vậy tổng khối lượng của khâu 2 và khâu 3, bàn tay kẹp là:

$$m_{234} = 163.64 + 625 + 200 = 988.64 \text{ (g)} = 0,989(kg)$$

Momen nhỏ nhất của động cơ khâu 2 là:

$$M_2 = m_{234} \times g \times d_{234} = 0,989 \times 10 \times 0,20 = 1.977(Nm).$$

Cần chọn loại động cơ có momen lớn hơn hoặc bằng M_2 .

Tương tự cách lựa chọn như cho động cơ khớp quay thứ 3. Nhóm lựa chọn Động cơ giảm tốc hộp số vuông góc JGY370 18rpm với một số thông số kỹ thuật dưới đây.

- Điện áp sử dụng: 12VDC;

- Tốc độ quay không tải: 18 rpm (vòng/phút);
- Đường kính trục: 6 mm;
- Kích thước: 77 x 32 x 21.5 mm;
- Momen: 25 kg.cm (tương đương 2.5Nm).

(thỏa mãn điều kiện về momen xoắn để xoay cánh tay khâu 2, 3, tay gấp).

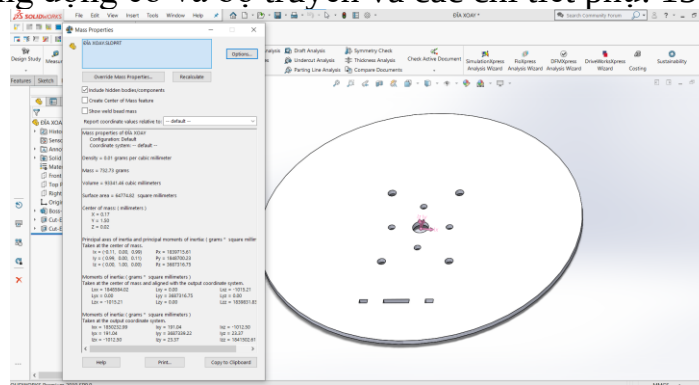
Đây là loại động cơ thích hợp với những ứng dụng cần tốc độ chậm khỏe, cần khóa trục khi không cấp điện, các ứng dụng như cơ cấu nâng hạ cơ cấu kẹp. Do cơ cấu bánh răng trục vít nên khi không cấp điện trục quay sẽ bị khóa, nghĩa là không thể quay trục bằng tay.

2.1.4 Tính toán động cơ và bộ truyền khớp quay thứ 1

a) Tính chọn động cơ

Thông số đầu vào:

- Khâu 2, 3, tay gấp: 989 (g)
- Khối lượng tâm xoay 3mm khâu 1: 0,733 (kg) = 732.73 (g) (theo tính toán của phần mềm SolidWorks H2.9 khối lượng riêng 7.85g/cm³);
- Khoảng cách trọng tâm tổng khối lượng khâu 1,2,3,4 đến trục momen: $d_{1234} < 150$ (mm);
- Khối lượng động cơ và bộ truyền và các chi tiết phụ: 150 (g);



Hình 2.9 Tính khối lượng tâm xoay khâu 1

Như vậy tổng khối lượng của khâu 2 và khâu 3,4 là:

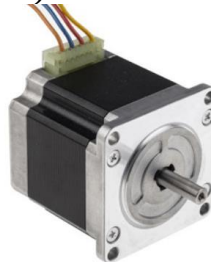
$$m_{1234} = m_{234} + m_{đc2} + m_{1'} = 989 + 733 + 150 = 1872 \text{ (g)} \\ = 1.872 \text{ (kg)}$$

Momen nhỏ nhất trên trục quay khâu 1 là:

$$M_1 = m_{234} \times g \times d_{234} = 1.872 \times 10 \times 0,15 = 2,808 \text{ (Nm)}.$$

Cần chọn loại động cơ có momen lớn hơn hoặc bằng M_1 .

Vì vậy nhóm lựa chọn động cơ Step 57 (H2.10) kết hợp bộ truyền bánh răng đai răng 4:1 để thực hiện nhiệm vụ đặt ra. Chọn loại động cơ bước step57 có momen xoắn là 1.8 (Nm).



Hình 2.10 Động cơ bước Step57

Với momen động cơ là $1.8 * 4 = 7.2 \text{ Nm}$. Vậy momen dùng để xoay cánh tay khâu 2,3, tay gấp là:

$$M_{1'} = 7.2 \text{ (Nm)} > 2.8 \text{ (Nm)}$$

(thỏa mãn điều kiện momen xoắn để xoay cánh tay).

Cách tính bộ truyền bánh răng đai răng tương tự như bộ truyền bánh răng đai răng khâu 3.

Vận tốc góc mong muốn của khâu 1: $\omega_1 = 1.9 \text{ (rad/s)} = 18 \text{ (vòng/phút)}$.

Với: n_d – Hiệu suất bộ truyền đai răng, $n_d = 0,97$;

n_{ol} – Hiệu suất 1 cặp ổ lăn, $n_{ol} = 0,99$;

Hiệu suất thực tế: $\mu = n_d \cdot n_{ol} = 0,99 \cdot 0,97 = 0,96$.

Công suất trên trục 1: $W_{t1} = M_1 \frac{\omega_1}{\mu} = 2.8 \times \frac{18}{0,96} = 52.5 \text{ (W)}$.

Vì vậy nhóm lựa chọn động cơ NEMA17 và bộ truyền đai răng bánh răng với tỷ số truyền 1:4 để thực hiện nhiệm vụ đặt ra.

Công suất trên động cơ khâu 1 là: $W_{dc1} = W_{t1} \div 4 = 13.125 \text{ (W)}$.

b, Tính toán bộ truyền bánh răng – đai răng khớp quay thứ 1

Bước răng xác định bằng công thức:

$$p \geq 35 \sqrt[3]{\frac{P_1}{n_1}} = 35 \sqrt[3]{\frac{0,013}{18}} = 3.0 \text{ (mm)}$$

Trong đó:

P_1 – công suất trên bánh đai chủ động

$$(P_1 = 13.125 \text{ (W)} = 0,013 \text{ (kW)})$$

n_1 – tốc độ quay của bánh đai chủ động ($n_1 = 18 \text{ (vòng/phút)}$)

Chọn $p = 3 \text{ (mm)}$, suy ra $m = p/\pi = 0.96$

Chọn $m = 1$.

Xác định chiều rộng đai

Theo [2], chiều rộng đai xác định như sau: $b = \psi_d m$

Trong đó: ψ_d – Hệ số chiều rộng đai. ψ_d dao động từ 6 đến 9.

Dựa vào bảng (H2.6), [2]; Chiều rộng đai $b = 10 \text{ (mm)}$.

Chiều rộng bánh đai: $B = b + m = 10 + 1 = 11 \text{ (mm)}$.

Xác định các thông số của bộ truyền

Số răng z_1 của bánh đai nhỏ được chọn theo bảng (H2.7) nhằm đảm bảo tuổi thọ cho đai. Số răng của bánh đai lớn $z_2 = u \cdot z_1$ với $u = n_1/n_2 = z_2/z_1$. Với tỷ số truyền $u = 4$.

Chọn $z_1 = 20$ (răng). $z_2 = u \cdot z_1 = 4 \cdot 20 = 80$ (răng).

Khoảng cách trục a được chọn theo điều kiện:

$$a_{min} \leq a \leq a_{max}$$

Trong đó:

$$a_{min} = 0,5m(z_1 + z_2) + 2m = 52 \text{ (mm)};$$

$$a_{max} = 2m(z_1 + z_2) = 200 \text{ (mm)}.$$

Số răng đai

$$z_d = \frac{2a}{p} + \frac{z_1 + z_2}{2} + \frac{(z_2 - z_1)^2 p}{40a} = \frac{2a}{p} + \frac{z_1 + z_2}{2} + \frac{(z_2 - z_1)^2 p}{40a}$$

$$= 90 \div 184$$

Chọn $z_d = 152$ (mm);

Thay $z_d = 152$ (mm) ta có: $a = 150$ (mm).

Chiều dài đai:

$$l_d = p \cdot z_d = 2.152 = 304 \text{ (mm)}$$

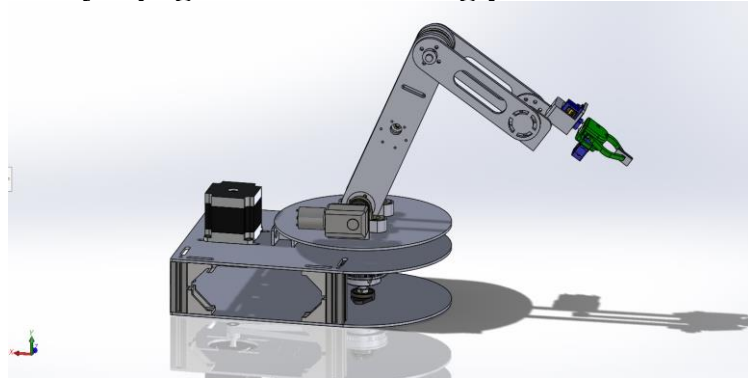
Góc ôm đai:

$$\alpha_1 = 180^\circ - \left[\frac{m(z_2 - z_1)}{a} \right] \cdot 57,3^\circ = 157,08^\circ$$

Số răng đồng thời ăn khớp trên bánh đai nhỏ:

$$z_{01} = \frac{z_1 \alpha_1}{360} = 8,7 \text{ (răng)}$$

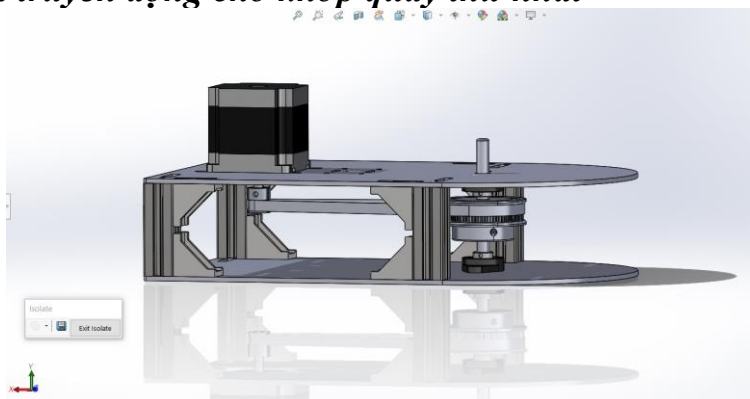
2.2 Thiết kế xây dựng mô hình 3D bằng phần mềm SolidWorks



Hình 2.11 Tổng thể hệ thống tay thao tác 4 bậc tự do

Cánh tay rôbot bao 4 cụm truyền động ghép nối với nhau thông qua một số chi tiết cơ khí. Sau đây, ta sẽ đi lần lượt từng cụm chi tiết qua các khâu dưới đây.

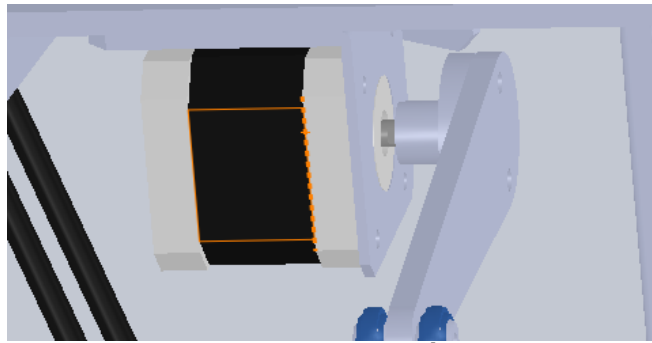
2.2.1 Cụm truyền động cho khớp quay thứ nhất



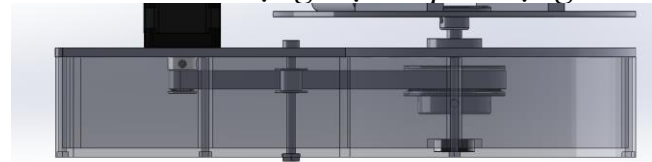
Hình 2.12 Cụm truyền động khớp quay thứ nhất

Để thực hiện chuyển động quay của khâu 1 so với khâu cố định, nhóm lựa chọn sử dụng hệ truyền động bánh răng đai răng gắn khâu làm việc với trục động cơ. Nhóm lựa chọn truyền động trực tiếp bởi một số ưu, nhược điểm sau đây:

Ở đây ta đưa ra hai phương án là dẫn động trực tiếp từ động cơ (H2.13) và dẫn động gián tiếp qua hệ truyền bánh răng đai răng (H2.14).



Hình 2.11 Dẫn động trực tiếp từ động cơ



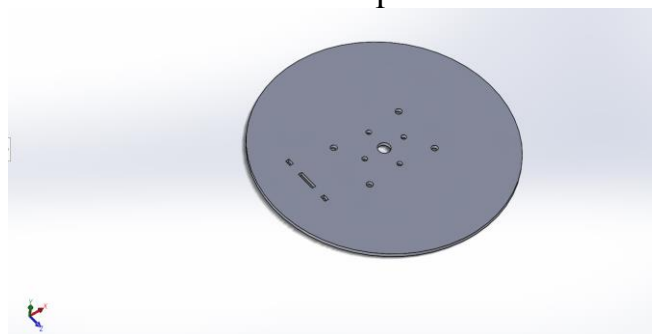
Hình 2.12 Dẫn động gián tiếp qua hệ bánh răng, đai răng

Trong 2 phương án thiết kế thì phương án bộ truyền bánh răng đai răng tạo lực nâng đảm bảo cho cơ cấu tuy nhiên sẽ giảm tốc độ của robot. Tỉ số truyền bánh răng có hiệu suất truyền cao, có tỉ số kích thước trên tỉ số truyền lớn, độ chính xác cao, tỉ số truyền ổn định, trong các robot có yêu cầu kết cấu nhỏ, độ chính xác cao thường hay sử dụng. Tuy nhiên với ưu điểm kết cấu cơ khí đơn giản, giá thành rẻ và phổ biến trên thị trường có khả năng làm việc với phạm vi vận tốc và tải trọng rộng nên bộ truyền bánh răng đai răng vẫn được dùng phổ biến cho các tay máy công nghiệp. Nhưng để robot hoạt động với tốc độ cao thì ta chọn phương án 2.

Tiếp theo đây, nhóm sẽ trình bày về đặc điểm, vật liệu gia công và bản vẽ của một số chi tiết trên.

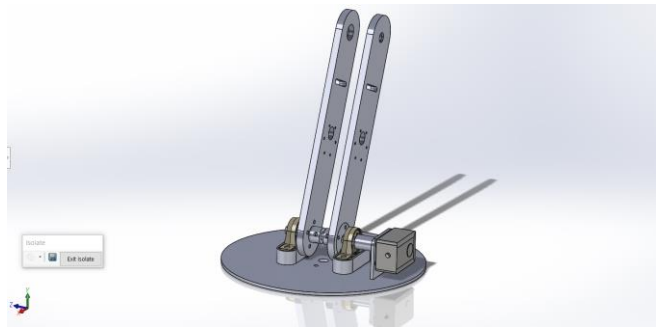
Tấm xoay khâu 1

Tấm xoay khâu 1 (H2.15) đóng vai trò là nơi cố định của các chi tiết khác thuộc khâu 2,3,4. Tấm này có vai trò làm xoay cánh tay robot theo trục Z ở khớp xoay thứ nhất. Nhóm lựa chọn vật liệu làm tấm xoay bằng thép tấm CT3, độ dày 3mm. Được gia công chế tạo bằng phương pháp cắt laser có độ chính xác cao theo bản vẽ đã thiết kế trên phần mềm SolidWorks.



Hình 2.15 Tấm xoay khâu 1

2.2.2 Cụm truyền động cho khớp quay thứ hai



Hình 2.16 Cụm truyền động cho khớp quay thứ hai

Để thực hiện chuyển động quay cho khớp thứ 2 (H2.16), ta cũng dùng động cơ truyền động trực tiếp cho khâu (sử dụng cách truyền động trực tiếp đòi hỏi động cơ phải có mômen kéo lớn hơn so với động cơ nếu sử dụng bộ truyền động), gồm các bộ phận sau: cánh tay khâu 2, động cơ bước bộ truyền đai đã được tính toán. Có một số chi tiết đã nhắc đến ở mục 3.1.2 nên ở đây nhóm sẽ chỉ nói tới các chi tiết mới.

Cánh tay khâu 2

Cánh tay khâu 2 (H2.17) đóng vai trò là thân khâu 2. Tấm này có vai trò làm xoay cánh tay robot theo trục Z (nằm ngang) ở khớp xoay thứ hai.

Nhóm lựa chọn vật liệu làm cánh tay nhựa in 3D, độ dày 7mm vừa nhỏ gọn và khối lượng tương đối nhẹ. Được gia công chế tạo bằng phương pháp in 3D có độ chính xác cao theo bản vẽ đã thiết kế trên phần mềm SolidWorks.



Hình 2.17 Cánh tay khâu 2

Khớp nối trục mặt bích

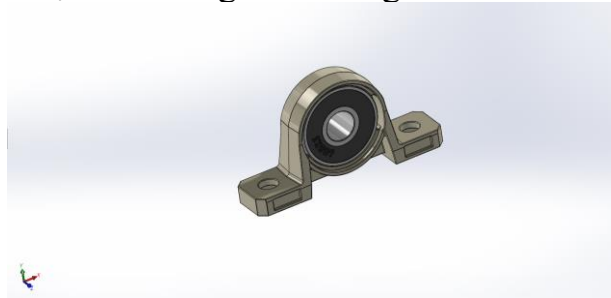
Khớp nối trục mặt bích (H2.18) là chi tiết truyền chuyển động từ trục quay động cơ sang cánh tay khâu 2. Bởi những kích thước tương đối đặc biệt, nhóm lựa chọn phương án gia công tiện trục và khoan các lỗ bắt bu – lông đai ốc để có thể chế tạo ra chi tiết này.



Hình 2.18 Khớp nối trục mặt bích

Gối đỡ trục ngang P08

Đây là chi tiết giúp trục chuyển động của khớp thứ 2 quay quanh trục cố định, các gối trục này được cố định với vị trí định hình nhờ các lỗ M5 có vị trí xác định trên tấm xoay khâu 1. Với kích thước trục đường kính 8mm nên lựa chọn gối đỡ trục có đường kính trong 8mm.



Hình 2.19 Gối đỡ trục P08

2.2.3 Cụm truyền động cho khớp quay thứ ba



Hình 2.20 Cụm truyền động cho khớp quay thứ ba

Cụm truyền động cho khớp quay thứ 3 của cánh robot cũng có nguyên lý giống cụm khâu 1 trước đó. Tuy nhiên để giảm khối lượng cho cánh tay robot và đơn giản hóa cấu trúc chuyển động, nhóm lựa chọn sử dụng phương pháp dẫn động đai răng bánh răng từ động cơ. Ở đây gồm: 1 động cơ giảm tốc có gắn encoder kèm bộ truyền đai răng; khớp nối mặt bích; cánh tay khâu 3 và bộ gá cánh tay làm việc.

Cánh tay khâu 3

Cánh tay khâu 3 (H2.21) đóng vai trò là thân khâu 3. Tấm này có vai trò làm xoay cánh tay robot theo trục Z (nằm ngang) ở khớp xoay thứ hai.

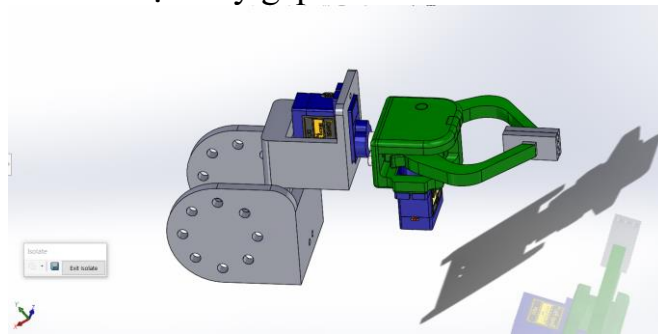
Nhóm lựa chọn vật liệu làm cánh tay nhựa in 3D, độ dày 5mm vừa nhỏ gọn và khối lượng tương đối nhẹ. Được gia công chế tạo bằng phương pháp in 3D có độ chính xác cao theo bản vẽ đã thiết kế trên phần mềm SolidWorks.



Hình 2.21 Cánh tay khâu 3

2.2.4 Cụm tay gắp

Bao gồm 2 động cơ servo SG90 và một bàn tay kẹp bằng nhựa PLA. Dưới đây là hình ảnh của cụm tay gấp nói trên.



Hình 2.22 Cụm tay gấp của robot

2.3 Kiểm tra bền các khâu

Với các cấu trúc có dạng thanh dầm chịu tải trọng, đặc biệt là trong ngành nghiên cứu, chế tạo tay rôbốt. Vì vậy, sau quá trình thiết kế mô hình cánh tay thao tác bằng phần mềm SolidWorks, nhóm cần kiểm tra lại các lực và độ bền các khâu rôbốt. Đối với mô hình tay thao tác mà nhóm sử dụng, cánh tay khâu 2 và 3 là những chi tiết chịu tải lớn nhất, chính những tải này có thể gây ra sai số cho mô hình. Mục 2.3.1 và 2.3.2 dưới đây sẽ là những kết quả mà nhóm thu được khi kiểm tra tính bền của chúng. Dựa vào những kết quả này, nhóm sẽ đưa ra những giải pháp xử lý.

2.3.1 Kiểm tra bền cánh tay khâu 3

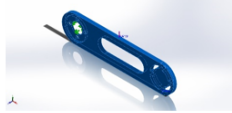
Cánh tay khâu 3 có hình dạng và kết cấu như H2.3. Nhóm sẽ kiểm tra ứng suất uốn và chuyển vị khi cánh tay này chịu tải. Cánh tay khâu 3 chịu tải từ hai yếu tố chính. Thứ nhất là trọng lượng của chính nó (26.8g). Thông số này đã được tính toán và kiểm tra bằng cách đưa vật liệu in 3D là nhựa PLA khi sử dụng phần mềm SolidWorks. Vật liệu này cho một vài thông số kỹ thuật dưới đây.

Bảng 2.1 Thông số nhựa in PLA

Tính chất	PLA
Độ bền kéo	50 MPa
Độ kéo dài	6 %
Mô đun uốn	4 GPa
Khối lượng riêng	1.3g/cm
Điểm hóa lỏng	173 độ C
Phân hủy sinh học	Ở điều kiện thích hợp
Nhiệt độ đóng rắn	60 độ C

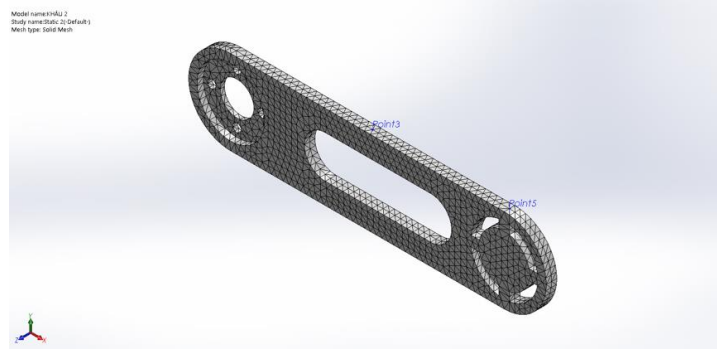
Một lực nữa tác động vào khâu này là trọng lượng của bàn tay gấp và các chi tiết phụ ở đầu tay thao tác (571g). Tuy nhiên, khâu 3 được cấu thành bởi 2 cánh tay như vậy, nên mỗi bên sẽ chịu tải 1 nửa (285.5g). Dưới đây là trình tự các thông số thực hiện mà nhóm kiểm tra bền cho cánh tay khâu 3. Tuy vậy, do không có thông số vật liệu nhựa PLA chi tiết và đầy đủ nên nhóm lựa chọn một loại nhựa có các thông số kỹ thuật gần giống vậy để thay thế. Và vật liệu đó là nhựa PEEK có thông số như H2.23.

Material Properties

Model Reference	Properties	Components
	Name: Polyetheretherketone (PEEK) Model type: Linear Elastic Isotropic Default failure criterion: Unknown Tensile strength: $9.5e+07 \text{ N/m}^2$ Compressive strength: $1.25e+08 \text{ N/m}^2$ Elastic modulus: $3.9e+09 \text{ N/m}^2$ Poisson's ratio: 0.4 Mass density: $1,310 \text{ kg/m}^3$	SolidBody 1(Cut-Extrude4)(KHẤU 3)
Curve Data:N/A		

Hình 2.23 Add vật liệu như PEEK cho khâu 3

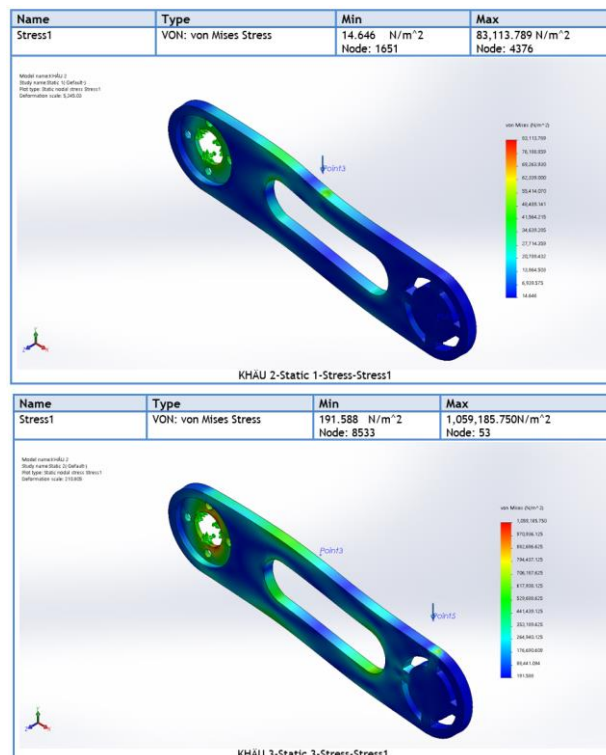
Sau khi chọn vật liệu cho chi tiết, nhóm lựa chọn chia cánh tay khâu 3 thành các phần tử tam giác 3 nút (H2.24) để thực hiện quá trình kiểm tra.



Hình 2.24 Chia phần tử trên khâu 3

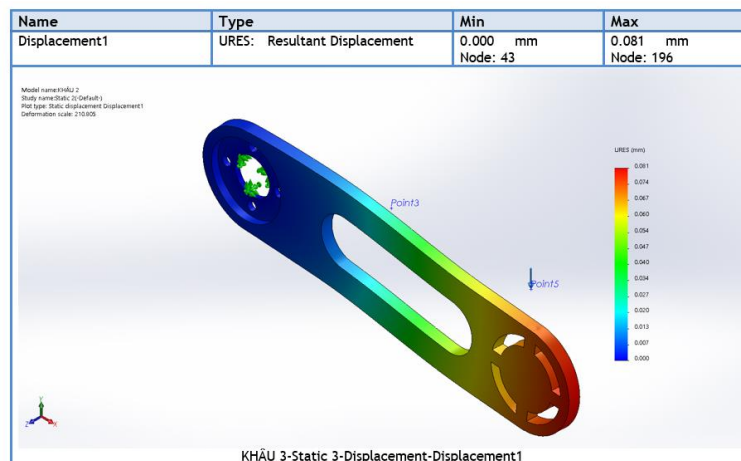
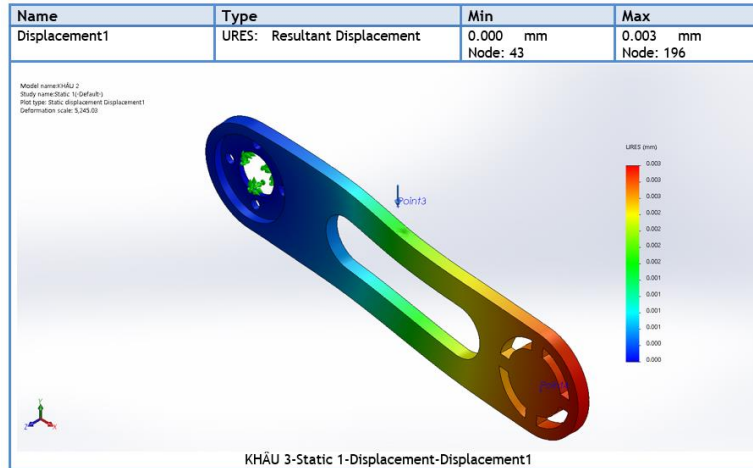
Điểm đặt lực cho khâu 3 như sau: đối với trọng lực của chính nó (đặt ngay tâm khối của khâu, như điểm Point 3 trên H2.24), còn lực tác động của các chi tiết khác đặt ở Point 5.

Dưới đây là thông số một vài kết quả đã đạt được: ứng suất uốn, chuyển vị của khâu 3.



Hình 2.25 Phân bố ứng suất uốn trên cánh tay khâu 3

H2.25 thể hiện ứng suất uốn sinh ra khi thanh khâu 3 chịu 2 lực tác động như đã nêu ở trên (lớn nhất ở node 4376 và 53, với thông số lần lượt là 83 113.789 N/m² và 1 059 185.750 N/m²). Ta thấy rằng ứng suất uốn sinh ra lớn nhất là ở vòng quanh lỗ bên trái. Vì đây là điểm tâm quay của khâu. Để nâng cao độ chắc chắn, nhóm đã thiết kế chi tiết khớp nối mặt bích từ thép CT45 gắn cố định vào phần lỗ ở trên nhằm giảm áp lực cho phần này.



Hình 2.26 Chuyển vị trên khâu 3

Vì tâm lỗ bên trái là tâm quay của khâu, nên chuyển vị lớn nhất của khâu sẽ nằm bên đầu xa tâm quay nhất. Với 2 lực tác động như đã nêu, chuyển vị lớn nhất lần lượt là 0.003 và 0.081mm (H2.26). Như vậy, khi tổng hợp 2 lực tác dụng, chuyển vị lớn nhất là 0.084 mm. Đây là một sai số tương đối nhỏ mà ta có thể bỏ qua.

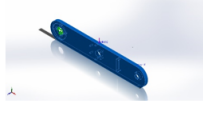
2.3.2 Kiểm tra bền cánh tay khâu 2

Cánh tay khâu 2 có hình dạng và kết cấu như H2.8. Nhóm sẽ kiểm tra ứng suất uốn và chuyển vị khi cánh tay này chịu tải. Cánh tay khâu 2 chịu tải từ hai yếu tố chính. Thứ nhất là trọng lượng của chính nó (81.82g). Thông số này đã được tính toán và kiểm tra bằng cách đưa vật liệu in 3D là nhựa PLA khi sử dụng phần mềm SolidWorks. Vật liệu này cho một vài thông số kỹ thuật dưới đây.

Một lực nữa tác động vào khâu này là trọng lượng của khâu 2, động cơ khâu 2, bàn tay gấp và các chi tiết phụ ở đầu tay thao tác (825g). Tuy nhiên, khâu 2 được cấu thành bởi 2 cánh tay như vậy, nên mỗi bên sẽ chịu tải 1 nửa

(412.5g). Dưới đây là trình tự các thông số thực hiện mà nhóm kiểm tra bên cho cánh tay khâu 2. Tuy vậy, do không có thông số vật liệu nhựa PLA chi tiết và đầy đủ nên nhóm lựa chọn một loại nhựa có các thông số kỹ thuật gần giống vậy để thay thế. Và vật liệu đó là nhựa PEEK có thông số như H2.27.

Material Properties

Model Reference	Properties	Components
	Name: Polyetheretherketone (PEEK) Model type: Linear Elastic Isotropic Default failure criterion: Unknown Tensile strength: $9.5e+07 \text{ N/m}^2$ Compressive strength: $1.25e+08 \text{ N/m}^2$ Elastic modulus: $3.9e+09 \text{ N/m}^2$ Poisson's ratio: 0.4 Mass density: $1,310 \text{ kg/m}^3$	SolidBody 1 (Cut-Extrude9) (KHẤU 2)
Curve Data: N/A		

Hình 2.27 Add vật liệu như PEEK cho khâu 2

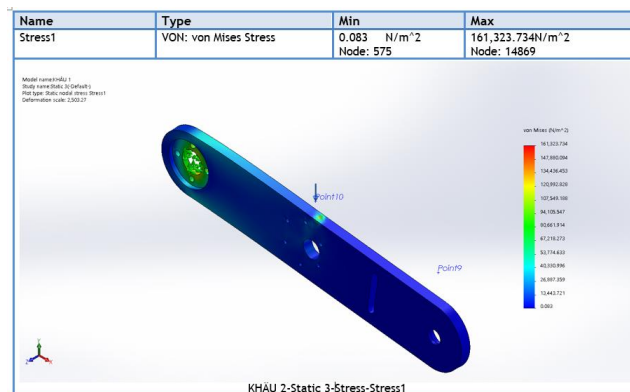
Sau khi chọn vật liệu cho chi tiết, nhóm lựa chọn chia cánh tay khâu 2 thành các phần tử tam giác 3 nút (H2.28) để thực hiện quá trình kiểm tra.

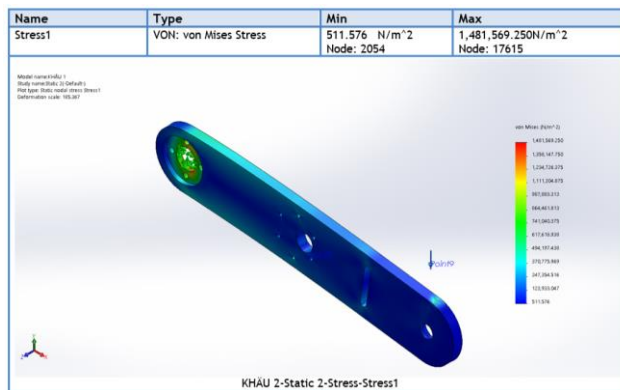


Hình 2.28 Chia phần tử trên khâu 2

Điểm đặt lực cho khâu 2 như sau: đối với trọng lực của chính nó (đặt ngay tâm khối của khâu, như điểm Point 10 trên H2.28), còn lực tác động của các chi tiết khác đặt ở Point 9.

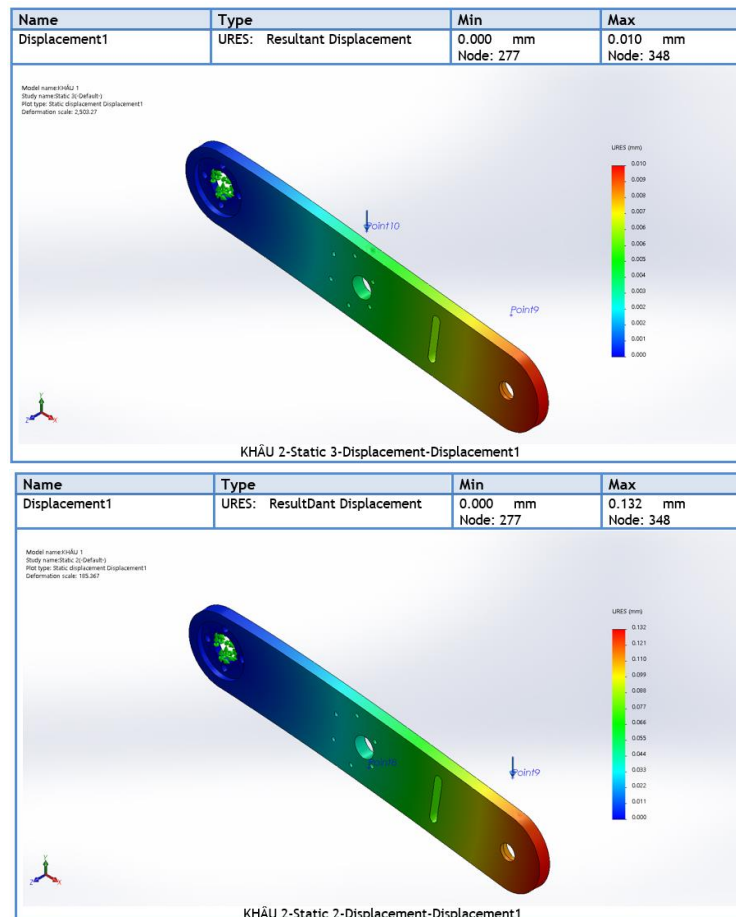
Dưới đây là thông số một vài kết quả đã đạt được: ứng suất uốn, chuyển vị của khâu 2.





Hình 2.29 Phân bố ứng suất uốn trên cánh tay khâu 2

H2.29 thể hiện ứng suất uốn sinh ra khi thanh khâu 2 chịu hai lực tác động như đã nêu ở trên (lớn nhất ở node 14869 và 17615, với thông số lần lượt là 161 363.734 N/m² và 1 481 569.250 N/m²). Ta thấy rằng ứng suất uốn sinh ra lớn nhất là ở vòng quanh lỗ bên trái. Vì đây là điểm tâm quay của khâu. Để nâng cao độ chắc chắn, nhóm đã thiết kế chi tiết khớp nối mặt bích từ thép CT45 gắn cố định vào phần lỗ ở trên nhằm giảm áp lực cho phần này.



Hình 2.30 Chuyển vị trên khâu 2

Vì tâm lỗ bên trái là tâm quay của khâu, nên chuyển vị lớn nhất của khâu sẽ nằm bên đầu xa tâm quay nhất. Với 2 lực tác động như đã nêu, chuyển vị lớn nhất lần lượt là 0.010 và 0.132 mm (H2.30). Như vậy, khi tổng hợp 2 lực

tác dụng, chuyển vị lớn nhất là 0.142 mm. Đây là một sai số tương đối nhỏ mà ta có thể bỏ qua.

2.4 Thiết kế mạch điều khiển tay thao tác bằng phần mềm Altium

2.4.1 Thiết kế mạch nguyên lí

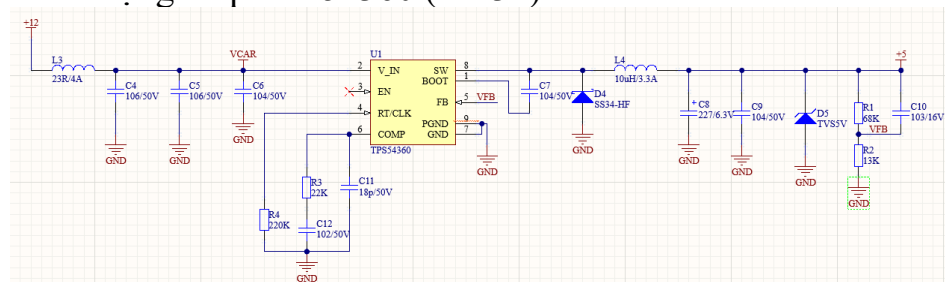
Để thiết kế được bộ điều khiển cho tay thao tác và kết nối các linh kiện điện tử sử dụng với nhau, nhóm cần vẽ được sơ đồ mạch nguyên lí. Trong sơ đồ này bao gồm các linh kiện điện tử: vi điều khiển, driver điều khiển động cơ, các cảm biến, động cơ các khâu, module kết nối,... Dựa vào yêu cầu bài toán của nhóm đặt ra ban đầu là tay thao tác di động thực hiện một số công việc y tế, nhóm đã chia mạch nguyên lí chung thành 6 khối nhỏ bao gồm: khối nguồn (Power); khối điều khiển (MCU), khối kết nối Wifi, khối Analog Converter, khối thiết bị ngoại vi (Peripheral), khối Driver. Sau đây nhóm sẽ đi chi tiết vào từng khối để làm rõ cấu tạo, thành phần và chức năng của các khối này.

a) Khối nguồn (Power)

Khối nguồn nhóm thiết kế là bộ phận cung cấp toàn bộ điện năng cho tay thao tác hoạt động, bao gồm cả mạch điều khiển và cơ cấu chấp hành. Vì các linh kiện khác nhau, mỗi loại lại có một thông số điện áp điều khiển khác nhau, nên nhóm lựa chọn dựa vào thông số điện áp của linh kiện có điện áp hoạt động lớn nhất làm nguồn đầu vào. Sau đó, để có thể cấp điện cho các chi tiết nhỏ hơn thì sẽ sử dụng các module hạ áp nhằm đáp ứng nhiệm vụ đó. Trong mô hình tay thao tác, điện áp sử dụng lớn nhất là 12V của hai động cơ giảm tốc Dc. Vì vậy, nguồn đầu vào nhóm sử dụng là 12V.

Ở khối này, từ nguồn 12V, nhóm sẽ thiết kế bộ hạ áp, đưa giá trị đầu vào là 12V xuống thành hai giá trị là 5V và 3.3V để sử dụng cấp nguồn cho các linh kiện khác.

Thứ nhất là hạ áp xuống 5V (sử dụng cho các động cơ Servo, Driver động cơ bước, cấp nguồn cho bộ hạ áp 3.3V). Để có thể thực hiện nhiệm vụ này, nhóm sử dụng chip TPS54360 (H2.31).

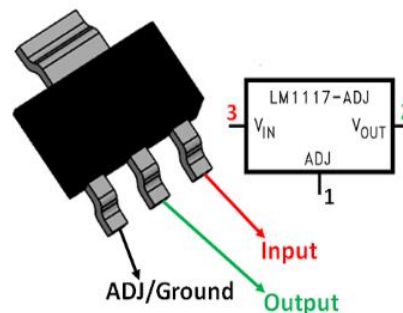


Hình 2.31 Mạch hạ áp 5V

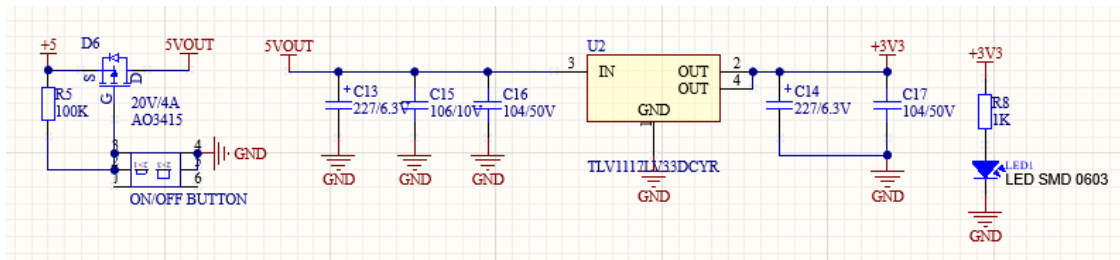
Tuy nhiên, 5V không phải là điện áp hoạt động của một số linh kiện khác, có thể kể đến như IC điều khiển STM32, module Wifi ESP32, bộ khuếch đại tín hiệu Loadcell HX711,.. Như vậy, ta tiếp tục cần thiết kế một mạch nguyên lí hạ áp từ 5V xuống 3.3V (H2.33) để sử dụng những linh kiện nêu trên. Nhóm lựa chọn bộ điều chỉnh LDO làm nguyên lí cho mạch hạ áp này, với IC LM1117 (H2.32) là IC chính. Mạch tích hợp điều chỉnh tuyến tính (IC) giảm điện áp từ mức cao hơn xuống mức thấp hơn mà không cần cuộn cảm.

Bộ điều chỉnh tuyến tính có độ sụt thấp (LDO) là một loại bộ điều chỉnh tuyến tính đặc biệt có điện áp rơi (chênh lệch giữa điện áp đầu vào và đầu ra cần được điều chỉnh) thường nhỏ hơn 400 mV. Các thiết kế bộ điều chỉnh tuyến tính ban đầu cung cấp điện áp giảm khoảng 1,3 V, có nghĩa là với điện áp đầu vào 5 V, thiết bị có thể được điều chỉnh để đạt được công suất tối đa chỉ khoảng 3,7 V.

Ngoài ra, trong khi các bộ điều chỉnh LDO thường là một trong những thành phần có chi phí thấp nhất trong bất kỳ hệ thống nhất định nào, chúng thường là một trong những thành phần có giá trị nhất về chi phí / lợi ích. Ngoài việc điều chỉnh điện áp đầu ra, một nhiệm vụ quan trọng khác của bộ điều chỉnh LDO là bảo vệ các phụ tải đắt tiền khỏi các điều kiện môi trường khắc nghiệt như quá điện áp, nhiễu cung cấp điện, điện áp ngược và tăng dòng điện. Nói tóm lại, thiết kế phải chắc chắn, bao gồm tất cả các tính năng bảo vệ để triệt tiêu tác động hiệu suất của môi trường trong khi bảo vệ tải. Nhiều bộ điều chỉnh tuyến tính LDO chi phí thấp thất bại vì chúng không có sự bảo vệ cần thiết, gây ra thiệt hại cho chính bộ điều chỉnh và làm hỏng tải phía sau.



Hình 2.32 IC LM1117 và sơ đồ chân



Hình 2.33 Mạch hạ áp 3.3V

Như vậy, nhóm đã xử lý nguồn đầu vào 12V thành các điện áp khác nhau (12V, 5V, 3.3V) để có thể sử dụng các linh kiện.

b) Khối điều khiển (MCU)

Khối điều khiển là bộ điều khiển trung tâm của tay thao tác, có nhiệm vụ nhận tín hiệu input và xuất các tín hiệu điều khiển output cho các cơ cấu chấp hành. Với IC chính là STM32F205VET6 (H2.34).

Dưới đây là một số thông số kỹ thuật của IC STM32F205VET6.

Bảng 2.2 Thông số kỹ thuật STM32F205VET6

STM32F205VET6 IC MCU 32BIT 512KB FLASH 100LQFP	
Đặc điểm	Thông số
Nguồn	1.8 V ~ 3.6 V
Tốc độ	120MHz
Loạt	STM32F2
RAM	132K x 8
Loại bộ nhớ chương trình	FLASH
Kích thước bộ nhớ chương trình	512KB (512K x 8)
Thiết bị ngoại vi	Brown-out Detect/Reset, DMA, I ² S, LCD, POR, PWM, WDT
Loại Oscillator	Internal
Nhiệt độ hoạt động	-40°C ~ 85°C (TA)
Số I / O	82
Độ nhạy độ ẩm	3 (168 Hours)
Chuyển đổi dữ liệu	A/D 16x12b; D/A 2x12b
Lỗi	32-Bit
Core Processor	ARM® Cortex®-M3
Giao thức kết nối	CANbus, I ² C, IrDA, LINbus, MMC, SPI, UART/USART, USB OTG



Hình 2.34 IC STM32

Để thực hiện giao tiếp giữa máy tính và thiết bị ngoại vi (ở đây là vi điều khiển STM32), nhóm sử dụng hai giao thức giao tiếp là RS232 và RS485.

Giao thức RS232

RS232 là một chuẩn giao tiếp nối tiếp dùng định dạng không đồng bộ, kết nối nhiều nhất là 2 thiết bị, chiều dài kết nối lớn nhất cho phép để đảm bảo dữ liệu là 12.5 đến 25.4m, tốc độ 20kbit/s đôi khi là tốc độ 115kbit/s với một số thiết bị đặc biệt. Ý nghĩa của chuẩn truyền thông nối tiếp nghĩa là trong một thời điểm chỉ có một bit được gửi đi dọc theo đường truyền.

Ưu điểm của giao diện nối tiếp RS232:

- Khả năng chống nhiễu của các cổng nối tiếp cao

- Thiết bị ngoại vi có thể tháo lắp ngay cả khi máy tính đang được cấp điện.
- Các mạch điện đơn giản có thể nhận được điện áp nguồn nuôi qua công nối tiếp;

Những đặc điểm cần lưu ý trong chuẩn RS232:

- Trong chuẩn RS232 có mức giới hạn trên và dưới (logic 0 và 1) là $\pm 12V$. Hiện nay đang được cố định trở kháng tải trong phạm vi từ 3000 Ω - 7000 Ω .
- Mức logic 1 có điện áp nằm trong khoảng -3V đến -12V, mức logic 0 từ +3V đến 12V.
- Tốc độ truyền nhận dữ liệu cực đại là 100kbps (ngày nay có thể lớn hơn).
- Các lõi vào phải có điện dung nhỏ hơn 2500pF.
- Trở kháng tải phải lớn hơn 3000 Ω nhưng phải nhỏ hơn 7000 Ω .
- Độ dài của cáp nối giữa máy tính và thiết bị ngoại vi ghép nối qua cổng nối tiếp RS232 không vượt qua 15m.
- Các giá trị tốc độ truyền dữ liệu chuẩn hay dùng : 9600, 19200, 28800, 38400.... 56600, 115200 bps.

Các mức điện áp đường truyền.

RS232 sử dụng phương thức truyền thông không đối xứng, tức là sử dụng tín hiệu điện áp chênh lệch giữa một dây dẫn và đất. Do đó ngay từ đầu tiên ra đời nó đã mang về lỗi thời của chuẩn TTL, nó vẫn sử dụng các mức điện áp tương thích TTL để mô tả các mức logic 0 và 1. Ngoài mức điện áp tiêu chuẩn cũng cố định các giá trị trở kháng tải được đấu vào bus của bộ phận và các trở kháng ra của bộ phát.

Mức điện áp của tiêu chuẩn RS232C (chuẩn thường dùng bây giờ) được mô tả như sau:

- Mức logic 0 : +3V , +12V
- Mức logic 1 : -12V, -3V

Các mức điện áp trong phạm vi từ -3V đến 3V là trạng thái chuyển tuyến. Chính vì từ - 3V tới 3V là phạm vi không được định nghĩa, trong trường hợp thay đổi giá trị logic từ thấp lên cao hoặc từ cao xuống thấp, một tín hiệu phải vượt qua quãng quá độ trong một thời gian ngắn hợp lý. Điều này dẫn đến việc phải hạn chế về điện dung của các thiết bị tham gia và của cả đường truyền. Tốc độ truyền dẫn tối đa phụ thuộc vào chiều dài của dây dẫn. Đa số các hệ thống hiện nay chỉ hỗ trợ với tốc độ 19,2 kBd.

Giao thức RS485

RS485 hay được biết đến với tên gọi đầy đủ là chuẩn giao tiếp RS485 hay cáp RS485, đây là phương thức giao tiếp kết nối với máy tính và các thiết bị khác. RS485 không chỉ đơn thuần là giao diện đơn lẻ mà nó chính là tổ hợp truyền thông có khả năng tạo ra các mạng đơn giản của nhiều thiết bị.

Chuẩn giao tiếp RS485 có thể kết nối max lên đến 32 thiết bị trên một cặp dây đơn và một hệ thống dây nối đất ở khoảng cách lên đến 1200m.

Cấu tạo của RS485:

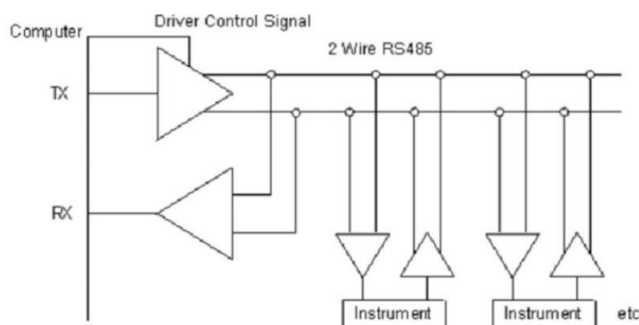
Cáp RS485 được cấu tạo rất đơn giản, chỉ từ các sợi dây được xoắn lại với nhau theo từng cặp. Tuy nhiên, chính cấu tạo này lại sinh ra một nhược điểm nghiêm trọng, khi hiện tượng nhiễu xuất hiện ở 1 cặp dây thì ngay lập tức cặp dây khác cũng sẽ bị. Điều này dẫn đến điện áp hoạt động giữa 2 dây sẽ không có quá nhiều sự chênh lệch, bộ phận thu của RS485 vẫn có thể nhận được tín hiệu vì bộ thu đã loại bỏ hết được hiện tượng nhiễu.

Nguyên lý hoạt động của RS485:

Nguyên lý hoạt động của RS485 khá đơn giản, dữ liệu sẽ được truyền qua 2 dây khi xoắn lại với nhau, dây này được gọi là cặp xoắn. Khi dây được xoắn lại sẽ tạo cho RS485 khả năng chống nhiễu cao và khả năng truyền tín hiệu đường dài tốt hơn. RS485 được chia làm 2 loại cấu hình, hiện đang được sử dụng nhiều nhất hiện nay là cấu hình 2 dây và cấu hình 4 dây.

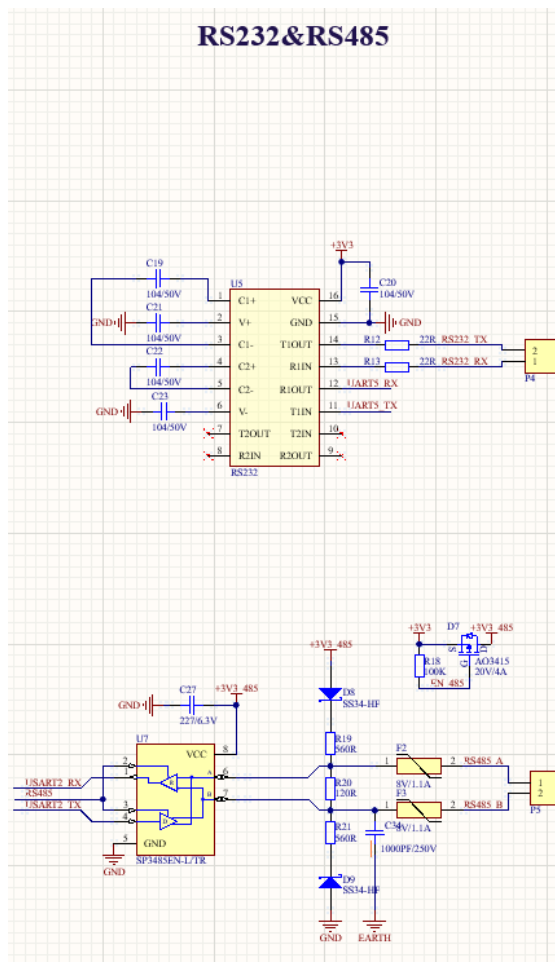
Ở đây, nhóm sử dụng RS485 2 dây. Đối với cấu hình 2 dây, dữ liệu sẽ được truyền đi theo một hướng tại một thời điểm nhất định. Với kiểu thiết lập này, tín hiệu TX và RX sẽ cùng nhau dùng chung một cặp dây duy nhất giúp người sử dụng tiết kiệm được chi phí cài đặt.

Nhìn vào H2.35 trên, ta thấy được rằng hệ thống phát và hệ thống thu sẽ được kết nối với nhau tại mỗi nút của một cặp xoắn. Tuy nhiên, cấu hình 2 dây lại làm giới hạn các nút tại cặp xoắn nên người sử dụng phải chú ý đến độ trễ quay vòng.



Hình 2.35 Sơ đồ chân RS485 2 dây

H2.36 thể hiện sơ đồ nguyên lý của kết nối 2 giao thức RS232 và RS485. Nguồn cấp cho 2 khối này là 3.3V mà ta đã hạ áp để sử dụng ở mục a.

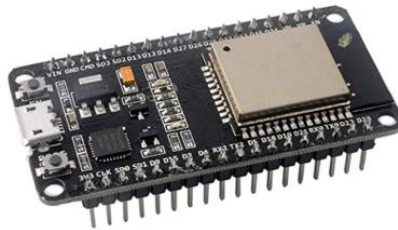


Hình 2.36 Sơ đồ nguyên lý mạch RS232 và RS485

c) Khởi kết nối Wifi

Để có thể kết nối và trao đổi tín hiệu giữa tay thao tác và mobile rôbot Kobuki. Nhóm lựa chọn sử dụng phương thức qua Wifi để thực hiện yêu cầu trên. Dữ liệu được đẩy lên một Web Server. Khi đó, các thông số và quá trình hoạt động của một trong hai (tay thao tác hoặc rôbot Kobuki) sẽ được truyền tới phần còn lại. Hiện nay có rất nhiều module đảm nhiệm nhiệm vụ thu phát Wifi như ESP32, ESP8266,... Tuy nhiên nhóm đã lựa chọn ESP32 làm bộ thu phát Wifi cho đề tài. Đây là một vi điều khiển lõi kép của Espressif Systems với Wi-Fi và Bluetooth tích hợp. ESP32 (H4.1) là một hệ thống vi điều khiển trên chip (SoC) giá rẻ của Espressif Systems, nhà phát triển của ESP8266 SoC. Nó là sự kế thừa của SoC ESP8266 và có cả hai biến thể lõi đơn và lõi kép của bộ vi xử lý 32-bit Xtensa LX6 của Tensilica với Wi-Fi và Bluetooth tích hợp. Điểm tốt về ESP32, giống như ESP8266 là các thành phần RF tích hợp của nó như bộ khuếch đại công suất, bộ khuếch đại nhận tiếng ồn thấp, công tắc ăng-ten, bộ lọc và Balun RF.

Kit RF thu phát Wifi ESP32 được dùng cho các ứng dụng cần kết nối, thu thập dữ liệu và điều khiển qua sóng Wifi, đặc biệt là các ứng dụng liên quan đến IoT. Kit RF thu phát Wifi ESP32 sử dụng chip nạp và giao tiếp UART mới và ổn định nhất là CP2102 có khả năng tự nhận Driver trên tất cả các hệ điều hành Window và Linux, đây là phiên bản nâng cấp từ các phiên bản sử dụng IC nạp giá rẻ CH340.

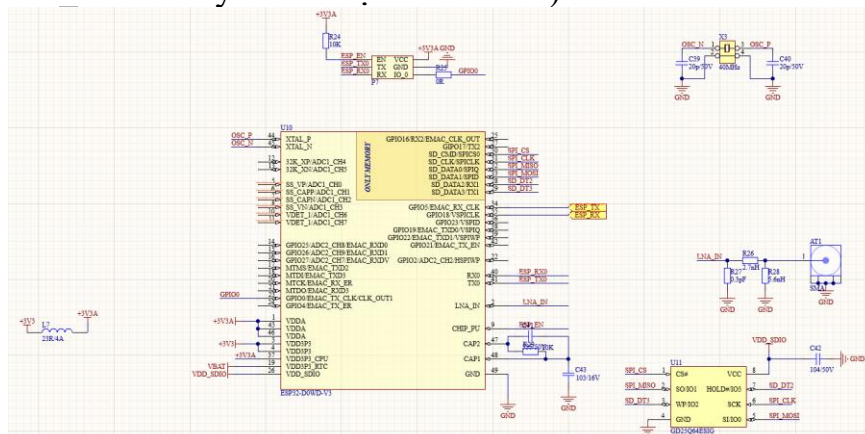


Hình 2.37 Module Wifi ESP32

Thông số kỹ thuật của ESP32:

- ESP32 có nhiều tính năng hơn ESP8266.
- Bộ vi xử lý LX6 32-bit lõi đơn hoặc lõi kép với xung nhịp lên đến 240 MHz.
- 520 KB SRAM, 448 KB ROM và 16 KB SRAM RTC.
- Hỗ trợ kết nối Wi-Fi 802.11 b / g / n với tốc độ lên đến 150 Mbps.
- Hỗ trợ cho cả thông số kỹ thuật Bluetooth v4.2 và BLE cổ điển.
- 34 GPIO có thể lập trình.
- 18 kênh SAR ADC 12 bit và 2 kênh DAC 8 bit
- Kết nối nối tiếp bao gồm 4 x SPI, 2 x I2C, 2 x I2S, 3 x UART.
- Ethernet MAC cho giao tiếp mạng LAN vật lý (yêu cầu PHY bên ngoài).
- 1 bộ điều khiển host cho SD / SDIO / MMC và 1 bộ điều khiển slave cho SDIO / SPI.
- Động cơ PWM và 16 kênh LED PWM.
- Khởi động an toàn và mã hóa Flash.
- Tăng tốc phần cứng mật mã cho AES, Hash (SHA-2), RSA, ECC và RNG.

Mạch nguyên lí (H2.38) thể hiện các kết nối của ESP32. Sử dụng nguồn 3.3V. Thực hiện quá trình truyền nhận với khối MCU thông qua 2 chân ESP_RX và ESP_TX (trong đó ESP_RX là chân ESP32 nhận tín hiệu từ MCU, và ESP_TX là truyền tín hiệu cho MCU).

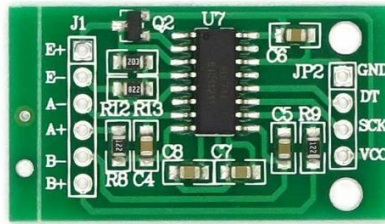


Hình 2.38 Mạch nguyên lí khối Wifi

d) Khối Analog Converter

Khi cảm biến Loadcell cảm nhận được lực nén sinh ra khi bàn tay gõ chạm vào một vật, thì nó sinh ra một điện áp rất nhỏ. Bởi điện áp này nhỏ

nên cần khuếch đại nó và chuyển nó từ tín hiệu Analog sang Digital để xử lý. Chính vì vậy, tên gọi của khối này là Analog Converter. Nhóm sử dụng mạch chuyển đổi HX711 (H2.39) thực hiện nhiệm vụ trên.



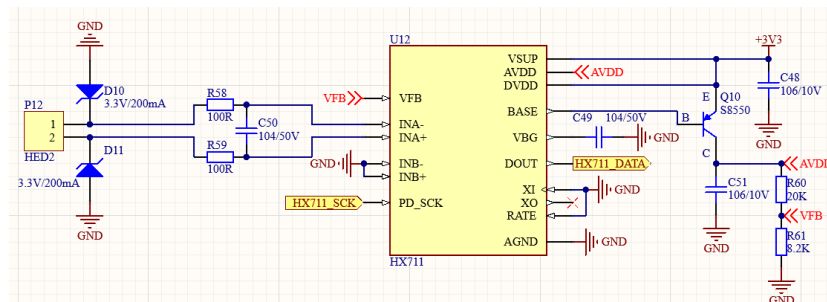
Hình 2.39 Mạch chuyển đổi HX711

HX711 không chỉ có một vài chức năng cơ bản, cũng có tích hợp cao, phản ứng nhanh, khả năng chống nhiễu, và độ tin cậy cao.

Thông số kỹ thuật:

- Điện áp hoạt động : 2.7 - 5V;
- Dòng tiêu thụ : < 1.5 mA;
- Tốc độ lấy mẫu : 10 - 80 SPS (tùy chỉnh);
- Độ phân giải : 24 bit ADC;
- Độ phân giải điện áp : 40mV;
- Kích thước : 38 x 21 x 10 mm.

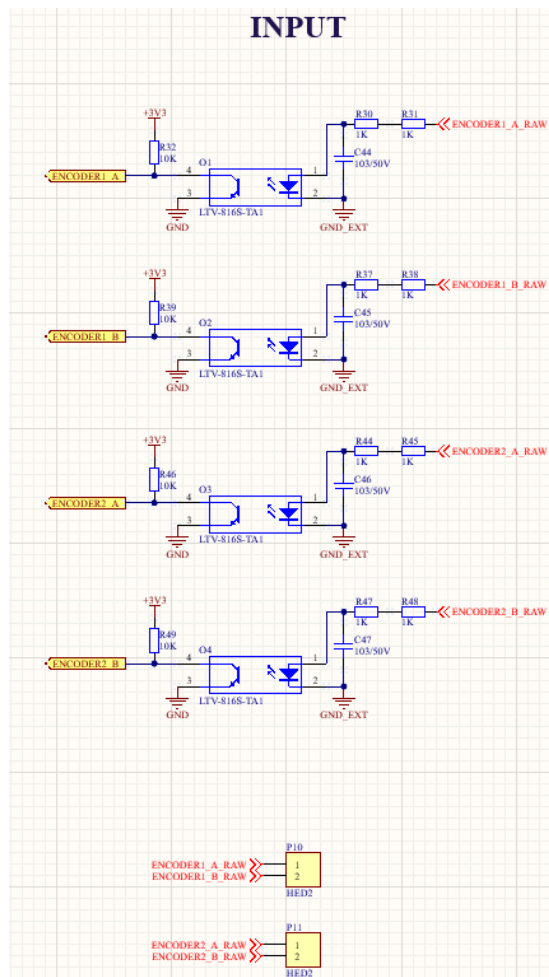
Ở mạch nguyên lí mà nhóm thiết kế (H3.40), HX711 chạy ở mức điện áp 3.3V. Với tín hiệu nhận vào từ chân HX711_SCK và tín hiệu đầu ra HX711_DATA.



Hình 2.40 Mạch nguyên lí khối Analog Converter

e) Khối thiết bị ngoại vi (Peripheral)

Khối thiết bị ngoại vi là khối kết nối với các thiết bị của cơ cấu chấp hành với bộ điều khiển chính. Bao gồm hai hướng tín hiệu chính. Thứ nhất là tín hiệu input. Tín hiệu input là tín hiệu đầu vào, bao gồm một số tín hiệu sau: tín hiệu từ 3 công tắc hành trình, giới hạn không gian chuyển động của rôbốt, tín hiệu của 2 encoder của hai động cơ giảm tốc DC. Bằng cách đọc tín hiệu đầu vào này, ta có thể xác định vị trí và điều khiển tay thao tác một cách chính xác nhất. Các tín hiệu đo được từ encoder và công tắc hành trình được nhận từ các cổng ENCODER1_A_RAW; ENCODER1_B_RAW; ENCODER2_A_RAW; ENCODER2_B_RAW và đưa tới bộ xử lý trung tâm MCU bằng cổng ENCODER1_A; ENCODER1_B; ENCODER2_A; ENCODER2_B như H2.41.



Hình 2.41 Sơ đồ nguyên lý tín hiệu Input

Thứ hai là khối tín hiệu Output, là khối tín hiệu mà khối điều khiển đã xử lý và xuất trả lại cho khối ngoại vi, bao gồm các tín hiệu qua các cổng MCU_OUT1; MCU_OUT2; MCU_OUT3; MCU_OUT4 (H2.42). Ở đây, STM32 sử dụng điện áp 3.3V, nhưng các driver động cơ bước và các động cơ servo SG90 sử dụng điện áp 5V nên ta cần một mạch nâng điện áp. Sử dụng mạch nâng lên 5V bằng nguyên lý Transitor Darlington.

f) Khối Driver

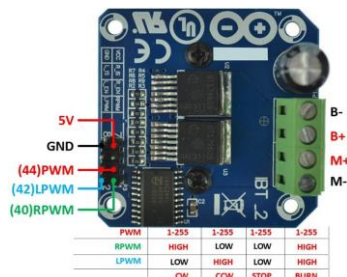
Khởi Driver sinh ra có tác dụng điều khiển động cơ. Tùy loại động cơ mà driver điều khiển lại khác nhau. Với động cơ bước Step57 đã có module điều khiển sẵn, nên nhóm chỉ thiết kế driver điều khiển 2 động cơ giảm tốc DC. Nhóm sử dụng điều khiển 2 động cơ này theo nguyên lý nửa mạch cầu H, như vậy để cần điều khiển 1 động cơ giảm tốc DC cần 2 nửa mạch cầu H, nên để điều khiển 2 động cơ ta cần 4 nửa mạch cầu H. Mạch cầu H - BTS7960 43A (H2.42) dễ dàng giao tiếp với vi điều khiển với driver tích hợp sẵn trong IC với đầy đủ các tính năng current sense (kết hợp với điện trở đo dòng), tạo dead time, chống quá nhiệt, quá áp, quá dòng, sụt áp và ngắn mạch.



Hình 2.42 Module điều khiển động cơ DC BTS 7960 43A

Thông số kỹ thuật:

- Nguồn: 6 ~ 27V.
- Dòng điện tải mạch: 43A (Tải trở) hoặc 15A (Tải cảm).
- Tín hiệu logic điều khiển: 3.3 ~ 5V.
- Tần số điều khiển tối đa: 25KHz.
- Tự động shutdown khi điện áp thấp: để tránh điều khiển động cơ ở mức điện áp thấp thiết bị sẽ tự shutdown. Nếu điện áp < 5.5V, driver sẽ tự ngắt điện và sẽ mở lại sau khi điện áp > 5.5V.
- Bảo vệ quá nhiệt: BTS7960 bảo vệ chống quá nhiệt bằng cảm biến nhiệt tích hợp bên trong. Đầu ra sẽ bị ngắt khi có hiện tượng quá nhiệt.
- Kích thước: 40 x 50 x 12mm.



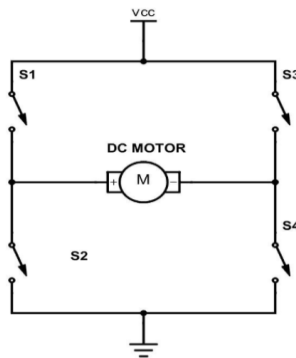
Hình 2.43 BTS 7960 43A Pinout

Sơ đồ chân:

- VCC: Nguồn tạo mức logic điều khiển (5V – 3.3V)
- GND: Chân đất.
- R_EN = 0 Disable nửa cầu H phải. R_EN = 1: Enable nửa cầu H phải.
- L_EN = 0 Disable nửa cầu H trái. L_EN = 1: Enable nửa cầu H trái.
- RPWM và LPWM: chân điều khiển đảo chiều và tốc độ động cơ.
- RPWM = 1 và LPWM = 0: Mô tơ quay thuận.
- RPWM = 0 và LPWM = 1: Mô tơ quay nghịch
- RPWM = 1 và LPWM = 1 hoặc RPWM = 0 và LPWM = 0: Dừng.
- R_IS và L_IS: kết hợp với điện trở để giới hạn dòng qua cầu H.

Nguyên lý hoạt động:

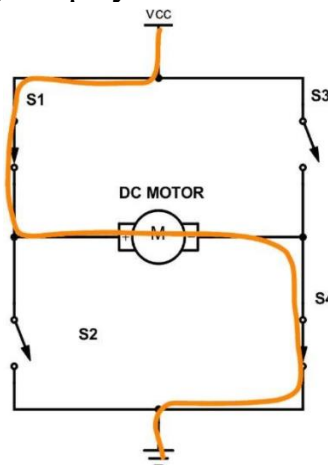
Driver: Mô đun điều khiển động cơ tích hợp 2 mạch nửa cầu H.



Hình 2.44 Mô hình mạch cầu H

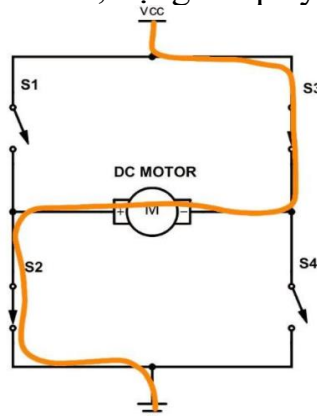
Một động cơ DC có thể quay thuận hoặc quay nghịch tùy thuộc vào cách bạn mắc cực âm và dương cho motor đó. Ví dụ, động cơ DC có hai đầu A và B. Nếu bạn nối A vào cực dương (+) và B vào cực âm (-) của nguồn thì động cơ quay theo chiều thuận (giả sử cùng chiều kim đồng hồ). Bây giờ bạn nối ngược lại, A vào (-) và B vào (+), động cơ sẽ quay nghịch (giả sử ngược chiều kim đồng hồ).

Tương tự, khi ta đóng S1 và S4, ta đã cho A nối với cực dương (+) và B nối với cực âm (-) của nguồn, một dòng điện chạy từ nguồn qua S1 qua động cơ qua S4 về mass làm động cơ quay theo chiều thuận. (H2.45)



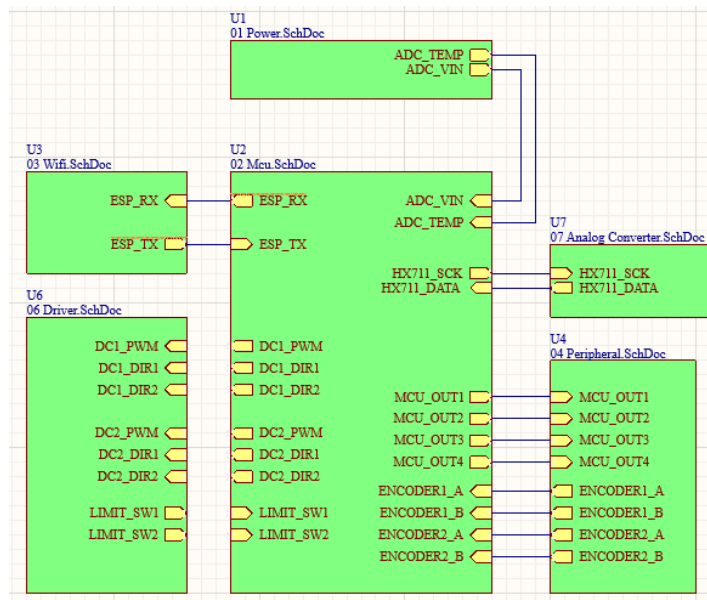
Hình 2.45 Động cơ quay thuận

Ngược lại, khi ta đóng S2 và S3, động cơ quay nghịch.



Hình 2.46 Động cơ quay nghịch

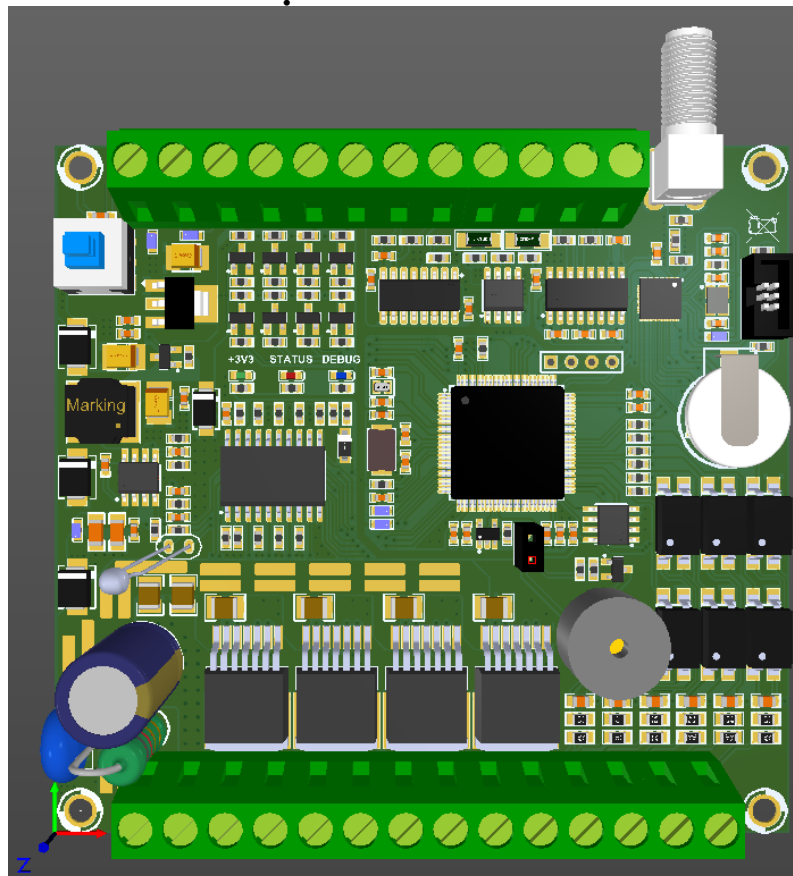
g) Mạch nguyên lí chung



Hình 2.47 Mạch nguyên lý chính của tay thao tác

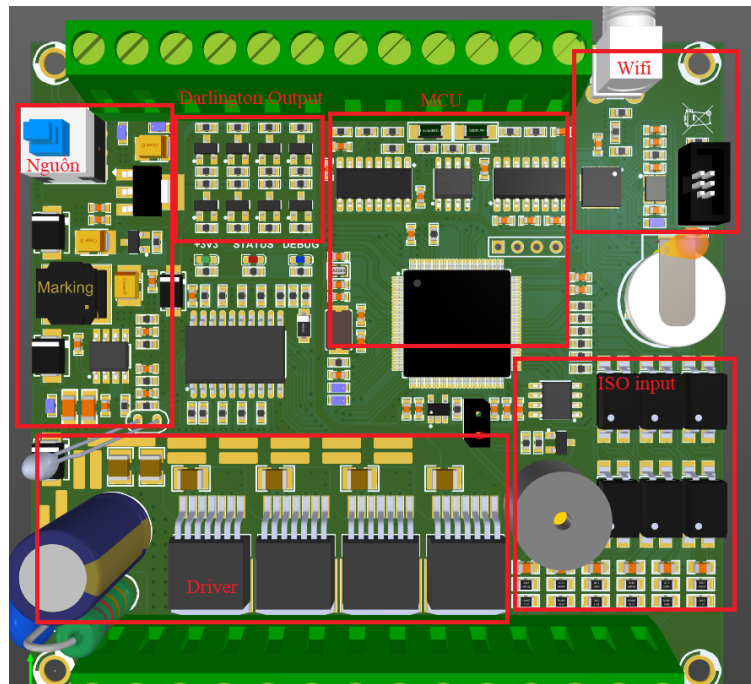
Với mạch nguyên lý này (H2.47), ta kết nối các khối đã nêu trên các mục a,b,c,d,e,f lại với nhau. Các cổng đầu vào đầu ra được thể hiện bằng các kí hiệu màu vàng có mũi tên chỉ hướng.

2.4.2 Quá trình thiết kế mạch PCB



Hình 2.48 Hình ảnh 3D mạch PCB

H2.48 thể hiện mạch 3D PCB của mạch nguyên lý đã thiết kế. Trên đây, nhóm đã hàn và bố trí các khối linh kiện sử dụng theo các vùng khác nhau, thể hiện bằng H2.49 dưới đây.



Hình 2.50 Sơ đồ vị trí các khối nguyên lý

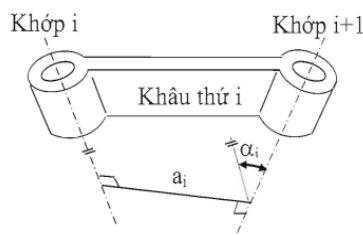
2.4.3 Bài toán động học tay thao tác

Xây dựng hệ tọa độ Denavit Haterberg (D-H):

Một rôbot thường gồm nhiều khâu liên kết nối tiếp với nhau thông qua các khớp động. Góc (khâu cơ bản) của một rôbot được gọi là khâu 0 và không tính vào số các khâu. Khâu 1 nối với khâu cơ bản bằng khớp 1 và không có khớp ở đầu mút khâu cuối cùng, trên khâu cuối cùng có gắn một công cụ. Như vậy các khâu, các khớp được đánh số tăng dần từ khâu cơ bản. Xét một khâu bất kì riêng rẽ thứ i , bất kì khâu nào cũng được đặc trưng bởi hai kích thước:

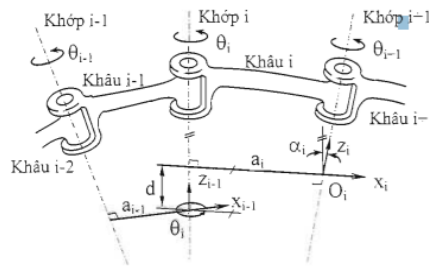
- Độ dài pháp tuyến chung a_i .
- Góc giữa các trục khớp đo trong mặt phẳng vuông góc với a_i . Kí hiệu là α_i .

Thông thường người ta thường gọi a_i là chiều dài khâu, còn α_i là góc xoắn khâu.



Hình 2.51 Chiều dài và góc xoắn khâu thứ i [1]

Khi xét mối quan hệ giữa các khâu với nhau, trong thực tế thường gặp hai khâu liên kết với nhau ở chính trục của khớp (H2.51).



Hình 2.52 Các thông số D-H của khâu [1]

Mỗi trục có hai pháp tuyến, mỗi pháp tuyến dùng cho một khâu (trước và sau khớp có trục đó). Vị trí tương đối của khâu thứ i so với khâu thứ $i-1$ được xác định bởi các thông số:

- d_i là khoảng cách giữa các pháp tuyến đo dọc theo trục khớp thứ i .
- θ_i là góc giữa các pháp tuyến đo trong mặt phẳng vuông góc với trục khớp thứ i .

d_i và θ_i thường được gọi là khoảng cách và góc quay tương đối giữa các khâu.

Bảng thông số D-H thường được biểu diễn dưới dạng sau:

Bảng 2.3 Thông số D-H

Thông số \ Khâu	θ_i	d_i	a_i	α_i
1				
2				
..				
n				

Quy tắc xác định các thông số D-H:

- Từ khâu gốc (đế), đánh số liên tiếp vào các khâu, khớp. Gốc được xem là khâu 0, khâu cuối là khâu tác động cuối. Ngoại trừ gốc và khâu cuối, các khâu còn lại phải bao gồm 2 khớp. Khâu thứ i liên kết với khớp i và khớp $i-1$.

- Dựng các trục z_i , chọn chiều dương. Dựng đường vuông góc chung giữa các trục của hai khớp kề nhau. Ngoại trừ gốc và khâu cuối, trục mỗi khớp (i) đều gắn với 2 đường vuông góc chung với trục khớp động ($i-1$) và khớp ($i+1$).

- Thiết lập hệ tọa độ gốc, ví dụ z_0 dọc theo trục khớp động thứ nhất, x_0 vuông góc với z_0 . Trục y_0 được xác định theo quy tắc bàn tay phải.

- Thiết lập hệ tọa độ bàn kẹp khâu thứ n thỏa mãn x_n vuông góc với trục khớp liền trước z_{n-1} . Trục z_n được chọn song song với trục z_{n-1} hoặc là hướng tiếp cận khâu cuối.

- Gắn tọa độ Đề Các tại khớp cuối của tất cả cá khâu như sau:

Trục z_i được chọn hướng theo trục khớp động thứ $(i+1)$,

Trục x_i được chọn dọc theo đường vuông góc chung giữa hai trục z_{i-1} và z_i ,

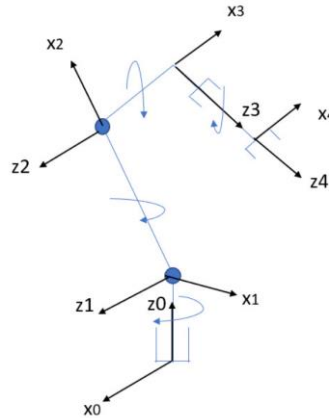
Trục y_i được xác định theo quy tắc hệ tọa độ thuận (quy tắc bàn tay phải),

Nếu các trục z_i và z_{i-1} song song với nhau, x_i có thể chọn bất kỳ đường vuông góc chung của hai trục,

Trong trường hợp hai trục z_{i-1} , z_i cắt nhau, gốc hệ tọa độ được chọn tại giao điểm các trục này và hướng của trục x_i được xác định quá tích có hướng của z_{i-1} và z_i hoặc có hướng ngược lại,

- Xác định các thông số của khâu và các biến khớp θ_i , d_i , a_i , α_i . Lập bảng thông số động học D-H.

Áp dụng vào mô hình robot, ta đặt các trục tọa độ như sau.



Hình 2.53 Các trục tọa độ trên tay thao tác

Trong đó:

- Các trục Z đặt dọc theo trục các khớp.
- Trục X_i đặt vuông góc với trục Z_i .

Bảng 2.4 Bảng D-H tay thao tác 4 bậc tự do

Thông số Khâu	θ_i	d_i	a_i	α_i
1	θ_1	d_1	0	90°
2	θ_2	0	a_2	0°
3	θ_3	0	a_3	90°
4	θ_4	d_4	a_4	0°

Ma trận thuận nhất D-H được thiết lập như sau:

$${}^{i-1}A_i = T(z_{i-1}, \theta_i) * T(z_{i-1}, d_i) * T(x_i, \alpha_i) * T(x_i, a_i)$$

$${}^{i-1}A_i = \begin{bmatrix} \cos\theta_i & -\cos\alpha_i \sin\theta_i & \sin\alpha_i \sin\theta_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\alpha_i \cos\theta_i & -\sin\alpha_i \cos\theta_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Bài toán động học thuận:

Cho biết:

Vị trí và quy luật chuyển động của các khớp. Bao gồm:

- Dạng chuyển động của khớp;
- Kích thước các khâu;
- Giới hạn.

Tìm:

Các quy luật chuyển động của khâu chấp hành cuối (bàn tay kẹp):

- Vị trí;
- Hướng chuyển động.

Từ B2.4, theo phương pháp DH ta có các ma trận biến đổi tọa độ thuần nhất biểu diễn vị trí và hướng tương đối giữa các khâu kế tiếp như sau:

$${}^0A_1 = \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & 0 \\ \sin\theta_1 & 0 & -\cos\theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^1A_2 = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & a_2\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & 0 & a_2\sin\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^2A_3 = \begin{bmatrix} \cos\theta_3 & 0 & \sin\theta_3 & a_3\cos\theta_3 \\ \sin\theta_3 & 0 & -\cos\theta_3 & a_3\sin\theta_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^3A_4 = \begin{bmatrix} \cos\theta_4 & -\sin\theta_4 & 0 & a_4\cos\theta_4 \\ \sin\theta_4 & \cos\theta_4 & 0 & a_4\sin\theta_4 \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Đây lần lượt là các ma trận biểu diễn vị trí và hướng của khâu 1 so với khâu 0; khâu 2 so với khâu 1; khâu 3 so với khâu 2 và khâu 4 so với khâu 3.

Ma trận biểu diễn vị trí và hướng của khâu thao tác so với khâu 0 được tính như sau:

$${}^0A_4 = {}^0A_1 * {}^1A_2 * {}^2A_3 * {}^3A_4 = \begin{bmatrix} c1c23c4 + s1s4 & -c1c23s4 + s1c4 & c1s23 & Px \\ s1c23c4 - c1s4 & -s1c23s4 - c1c4 & s1s23 & Py \\ s23s4 & -s23s4 & -c23 & Pz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Trong đó:

$$\cos\theta_1 = c1, \dots, \cos(\theta_2 + \theta_3) = c23$$

$$\sin\theta_1 = s1, \dots, \sin(\theta_2 + \theta_3) = s23$$

Với:

$$Px = d4c1s23 + a2c1c2 + a4s1s4 + (a4c4 + a3)c1c23$$

$$Py = d4s1s23 + a2s1c2 - a4c1s4 + (a4c4 + a3)s1c23$$

$$Pz = d1 - d4c23 + a3s23 + a2s2 + a4s23c4$$

Giả thiết E và φ lần lượt là vị trí và hướng của khâu thao tác so với khâu 0. Vị trí và hướng khâu thao tác được biểu diễn dạng ma trận như sau:

$$T_E = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 & x_E \\ \sin\varphi & \cos\varphi & 0 & y_E \\ 0 & 0 & 1 & z_E \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ta có: ${}^0A_4 = T_E$

Từ hệ phương trình các tọa độ định vị điểm tác động cuối và hướng của khâu thao được xác định bằng cách so sánh các phần tử ở hai vế của hệ phương trình động học dạng ma trận, suy ra:

$$\begin{cases} x_E = d_4 c_1 s_2 s_3 + a_2 c_1 c_2 + a_4 s_1 s_4 + (a_4 c_4 + a_3) c_1 c_2 s_3 \\ y_E = d_4 s_1 s_2 s_3 + a_2 s_1 c_2 - a_4 c_1 s_4 + (a_4 c_4 + a_3) s_1 c_2 s_3 \\ z_E = d_1 - d_4 c_2 s_3 + a_3 s_2 s_3 + a_2 s_2 + a_4 s_2 s_3 c_4 \end{cases}$$

Bài toán động học ngược:

Cho biết:

Vị trí và quy luật chuyển động khâu chấp hành cuối. Bao gồm:

- Vị trí của bàn tay kẹp;
- Hướng của bàn tay kẹp.

Tìm:

Quy luật chuyển động của các khớp:

- Dạng chuyển động của khớp
- Sự biến thiên của các dạng chuyển động đó.

Đối với cơ cấu robot trong đồ án đang thực hiện có 4 bậc tự do thì các vector chỉ hướng và vector vị trí của khâu thao tác sẽ được cho biết để tìm ra các góc quay $\theta_1, \theta_2, \theta_3, \theta_4$.

Đây là cơ cấu có 4 bậc tự do nên có rất nhiều cách giải để tìm ra các thông số trên. Tuy nhiên trong đồ án này, em sử dụng phương pháp hình học biến đổi giải tích.

Ý nghĩa:

Qua các phép biến đổi hình học giải tích trong bài toán động học ngược, ta đã giải quyết được các yêu cầu về cách tìm dạng chuyển động của các khớp và sự biến thiên của chúng nhằm đưa ra phương án điều khiển hợp lý để robot có thể thực hiện các hành động theo yêu cầu.

a- Tìm θ_2 (θ_2)

Từ việc so sánh 2 ma trận ở bài toán thuận, ta có: $-c_2 s_3 = 1$

Suy ra: $s_2 s_3 = 0$

Thay vào hệ phương trình, ta có:

$$z_E = d_1 - d_4 c_2 s_3 + a_3 s_2 s_3 + a_2 s_2 + a_4 s_2 s_3 c_4 = d_1 - d_4 + a_2 s_2$$

Chuyển về đổi dấu:

$$s_2 = \frac{z_E - (-d_4 + d_1)}{a_2}$$

$$c_2 = \frac{x_E + a_3 c_1}{a_2 c_1}$$

Suy ra:

$$\tan \theta_2 = \frac{s_2}{c_2} = \frac{\frac{z_E - (-d_4 + d_1)}{a_2}}{\frac{x_E + a_3 c_1}{a_2 c_1}}$$

Vậy:

$$\theta_2 = \arctan \frac{s_2}{c_2}$$

b- Tìm theta3 (θ_3)

Do

$$\begin{cases} -c_{23} = 1 \\ s_{23} = 0 \end{cases}$$

Nên $\theta_2 + \theta_3 = 0$

Ta đã tìm được θ_2 .

Vậy $\theta_3 = -\theta_2$

c- Tìm theta4 (θ_4)

Từ hệ phương trình:

$$\begin{cases} x_E = d_4 c_1 s_{23} + a_2 c_1 c_2 + a_4 s_1 s_4 + (a_4 c_4 + a_3) c_1 c_{23} \\ y_E = d_4 s_1 s_{23} + a_2 s_1 c_2 - a_4 c_1 s_4 + (a_4 c_4 + a_3) s_1 c_{23} \\ z_E = d_1 - d_4 c_{23} + a_3 s_{23} + a_2 s_2 + a_4 s_{23} c_4 \end{cases}$$

Thay $-c_{23} = 1$; $s_{23} = 0$, ta có:

$$x_E = a_2 c_1 c_2 + a_4 s_1 s_4 + (a_4 c_4 + a_3) c_1$$

$$y_E = a_2 s_1 c_2 - a_4 c_1 s_4 + (a_4 c_4 + a_3) s_1$$

Đặt $A = a_2 c_2 + a_4 c_4 + a_3$; $B = a_4 s_4$

Suy ra:

$$\begin{cases} x_E = A c_1 + B s_1 \\ y_E = A s_1 - B c_1 \end{cases}$$

Thực hiện phép:

$$x_E^2 + y_E^2 = A^2 + B^2 = (a_2 c_2 + a_4 c_4 + a_3)^2 + (a_4 s_4)^2$$

$$x_E^2 + y_E^2 = P + 2c_4(a_2 c_2 a_4 + a_4 a_3)$$

Với $P = a_2^2 c_2^2 + a_3^2 + a_4^2 + 2a_2 a_3 c_2$

Vậy:

$$c_4 = \frac{x_E^2 + y_E^2 - P}{2(a_2 c_2 a_4 + a_4 a_3)}$$

Suy ra:

$$\theta_4 = \arccos c_4$$

d- Tìm theta4 (θ_1)

Từ việc so sánh 2 ma trận ở bài toán thuận, ta có:

$$-s_1 c_{23} s_4 - c_1 c_4 = \cos \varphi$$

$$-c_1 c_{23} s_4 + s_1 c_4 = -\sin \varphi$$

Trong đó $\cos \varphi$ đã biết.

Tương đương:

$$s_1 s_4 - c_1 c_4 = -\cos(\theta_1 - \theta_4) = \cos(180 - \theta_1 + \theta_4) = \cos \varphi$$

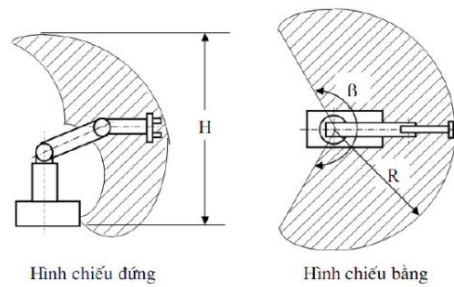
Suy ra: $180 - \theta_1 + \theta_4 = \varphi$

Nên: $\theta_4 = \varphi + \theta_1 - 180$

2.5 Không gian làm việc của tay thao tác

Không gian làm việc của robot là toàn bộ thể tích được quét bởi khâu chấp hành cuối khi robot thực hiện tất cả các chuyển động có thể. Trường công tác bị ràng buộc bởi các thông số hình học của robot cũng như các ràng buộc cơ học của các khớp. Ví dụ, một khớp quay có thể chuyển động

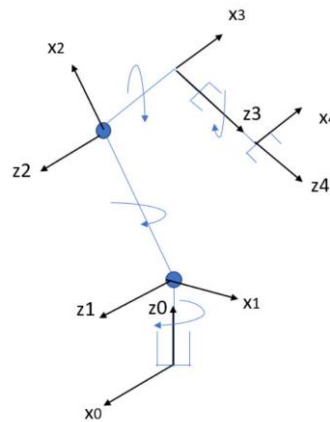
360°. Người ta thường dùng hai hình chiếu để mô tả không gian làm việc của một robot (H2.54)



Hình 2.54 Biểu diễn vùng làm việc của rô-bốt

Bằng phần mềm Matlab, ta biểu diễn được vùng làm việc của tay thao tác bằng cách mô tả tập hợp các điểm nằm trên tâm bàn tay kẹp.

Dưới đây là đoạn code biểu diễn các tọa độ điểm E (tâm bàn tay kẹp) trong tọa độ không gian 3 chiều Oxyz.



Hình 2.55 Tọa độ điểm tác động cuối E trong cơ cấu tay thao tác

Ma trận D-H được thiết lập trong phần mềm Matlab như sau:

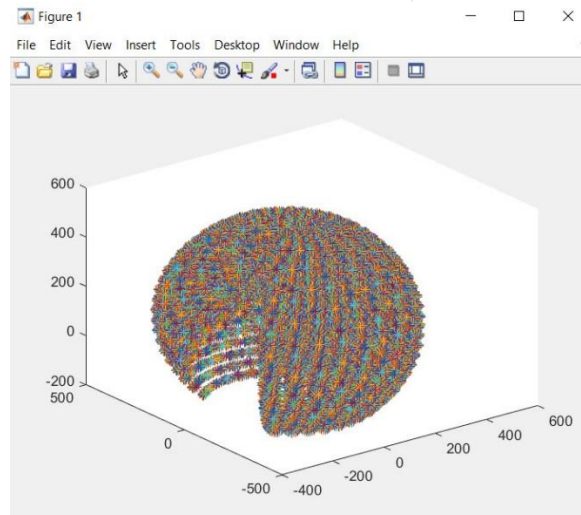
```
%DH parameters
clc
syms a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 t1 t2 t3 t4 t5 t6 t7 d1 d2 d3 d4 x y z s a b phi
n=input('Số lượng hệ trục tọa độ: ');
T=eye(4);
for k=1:n
    fprintf('\n hệ trục số %d \n',k);
    a=input('a : ');
    theta=input('Theta : ');
    d=input('d : ');
    alpha=input('alpha : ');
    A=simplify([cos(theta) -sin(theta)*cos(alpha) sin(theta)*sin(alpha) a*cos(theta);
               sin(theta) cos(theta)*cos(alpha) -cos(theta)*sin(alpha) a*sin(theta);
               0 sin(alpha) cos(alpha) d;
               0 0 0 1]);
    T=T*A;
end
T=simplify(T)
```

Tính toán các ma trận thuận nhân và biểu thức tọa độ điểm tác động cuối E:

```

%Không gian làm việc 4 dof
a2=200;
a3=190;
a4=25;
d1=119;
d4=36;
for t1=-3*pi/4:0.2*pi/4
    for t2=-pi/4:0.2*pi/3
        for t3=0:0.2:3*pi/4
            for t4=-pi/2:0.2*pi/2
                x= d4*(cos(t1)*cos(t2)*sin(t3) + cos(t1)*cos(t3)*sin(t2)) + a2*cos(t1)*cos(t2) + a4*sin(t1)*sin(t4) - a4*cos(t4)
                y= d4*(cos(t2)*sin(t1)*sin(t3) + cos(t3)*sin(t1)*sin(t2)) - a4*cos(t4)*(sin(t1)*sin(t2)*sin(t3) - cos(t2)*cos(t1))
                z= d1 - d4*cos(t2 + t3) + a3*sin(t2 + t3) + a2*sin(t2) + a4*sin(t2 + t3)*cos(t4);
                plot3(x,y,z,'r')
            end
        end
    end
end

```



Hình 2.56 Không gian làm việc của tay thao tác trong hệ $O_0x_0y_0z_0$

CHƯƠNG 3- ĐIỀU KHIỂN ROBOT TỰ HÀNH THÔNG MINH KOBUKI

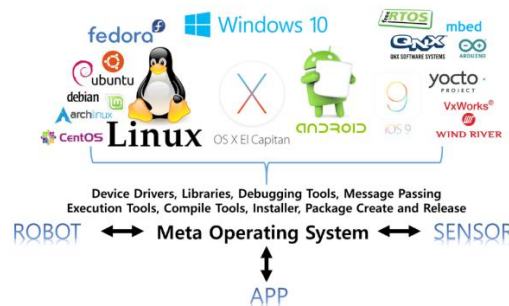
3.1 Tổng quan về hệ điều hành ROS và bài toán SLAM

3.1.1 Hệ điều hành ROS

a) Tổng quan về ROS

Hệ điều hành ROS (Robot Operating System) là một nền tảng phần mềm dùng để phát triển Robot. Nó bao gồm các công cụ được sử dụng để phát triển chương trình trên Robot. ROS là một Meta-Operating System, có thể chạy trên các HĐH hiện có như Linux, Window, Android...

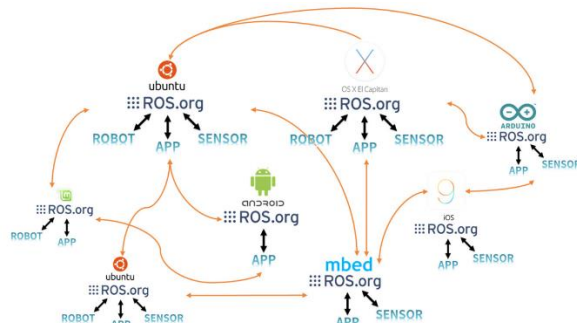
ROS là một hệ thống hỗ trợ để điều khiển Robot, đọc dữ liệu các cảm biến và phát triển ứng dụng Robot trên các hệ điều hành thông thường hiện có (H3.1).



Hình 3.1 ROS là một Meta-Operating System

ROS không phải là một hệ điều hành thông thường nhưng nó lại chạy trên các HĐH thông thường đó. Vì vậy, để sử dụng ROS thì cần phải sử dụng một HĐH khác (thường sử dụng Linux).

ROS cung cấp các chức năng thiết yếu cần thiết cho các chương trình ứng dụng robot như các thư viện, truyền/nhận dữ liệu giữa các phần cứng không đồng nhất và xử lý lỗi. ROS có thể truyền và nhận dữ liệu từ các HĐH khác nhau, các phần cứng và các chương trình khác nhau. Chỉ cần chúng đều sử dụng HĐH ROS thì dù là một chương trình điều khiển trên Android cũng dễ dàng gửi lệnh điều khiển đến ROBOT có một PC sử dụng HĐH Linux. Hay các dữ liệu cảm biến được đọc bằng Arduino cũng có thể gửi về robot dùng Linux mà HĐH.



Hình 3.1 ROS hỗ trợ giao tiếp đa nền tảng

b) Các khái niệm quan trọng

Dưới đây là một số khái niệm quan trọng trong sử dụng HĐH ROS (H3.3):

Client Layer: gồm các thư viện hỗ trợ các ngôn ngữ lập trình khác nhau như: roscpp (hỗ trợ C++), rospy (hỗ trợ Python);

Robotics Application: gồm các ứng dụng cần thiết cho sự hoạt động của robot;

Software Development Tools: các tool hỗ trợ cho quá trình phát triển robot, ví dụ: hiển thị robot, hiển thị bản đồ và các tính hiệu cảm biến trên máy tính trên Rviz;

Simulation: mô phỏng robot và các cảm biến trong môi trường ảo.



Hình 3.3 Các thành phần trong HĐH ROS

ROS gồm có một master và các node: master có vai trò quản lý kết nối của các node trong ROS. Các node đăng ký các thông tin cần thiết với master, và yêu cầu kết nối đến 1 node nào đó với master. Master sẽ gửi yêu cầu kết nối tới node được yêu cầu. Sau đó 2 node sẽ kết nối trực tiếp với nhau để truyền và nhận các thông tin (H3.4).

Master (máy chủ): Là một node đặc biệt quản lý các node khác và quá trình trao đổi thông tin giữa chúng.

Node: Là đơn vị nhỏ nhất của bộ xử lý đang chạy trong ROS, nó như 1 chương trình thực hiện một nhiệm vụ nào đó. Các node được chia làm 2 loại: Subscriber và Publisher.

Message: Là các biến như interger, floating point và Boolean. Nó chính là các thông tin mà các node giao tiếp với nhau.

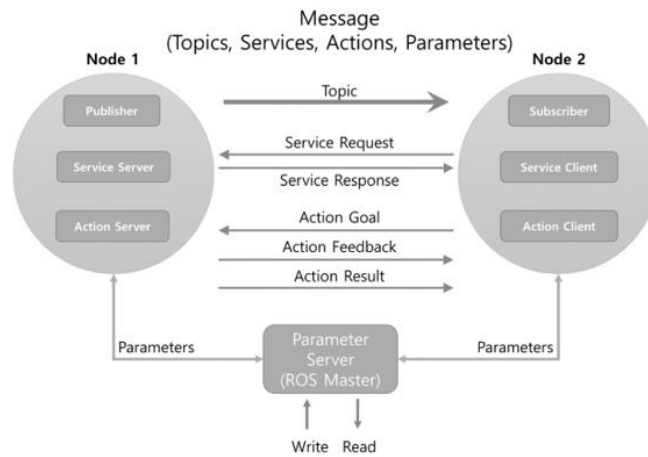
Topic: là một thông tin mà node subscriber đăng ký với master là thông tin mà các node publisher sẽ yêu cầu đến master để kết nối với node publisher.

Node publisher đăng ký thông tin và topic với master, và gửi các message tới các node subscriber được kết nối thông qua topic chung.

Node subscriber đăng ký thông tin và topic với master và nhận thông tin của Publisher, những thông tin được máy chủ publish lên topic liên quan đến subscriber. Dựa trên thông tin nhận được, node subscriber sẽ kết nối trực tiếp với node publisher và nhận các message từ node publisher được kết nối.

Trong ROS có 3 phương thức giao tiếp giữa các node là:

- Topic;
- Service;
- Action.



Hình 3.4 Phương thức giao tiếp giữa các Node
Bảng 3.1 Các phương thức giao tiếp trong ROS

Loại	Đặc tính		Mô tả
Topic	Không đồng bộ	Một chiều	Sử dụng khi trao đổi dữ liệu liên tục.
Service	Đồng bộ	Hai chiều	Sử dụng khi có yêu cầu xử lý vấn đề và phản hồi lại kết quả.
Action	Không đồng bộ	Hai chiều	Sử dụng khi có sự khó khăn khi sử dụng service vì thời gian phản hồi lâu sau khi yêu cầu hoặc cần có giá trị phản hồi trung gian.

Về cơ bản, giao tiếp trong ROS giống với phương thức truyền thông qua mạng MQTT – phương thức ứng dụng phổ biến trong lĩnh vực IOT. Máy chủ (Master) trong ROS giống với một Broker của MQTT, các node giống với các client của MQTT. Nhưng khác với MQTT các client truyền nhận dữ liệu thông qua Broker, còn các node trong ROS chỉ đăng kí thông tin Topic và gửi yêu cầu kết nối với nhau đến máy chủ, sau đó 2 node sẽ kết nối trực tiếp với nhau. Nhưng ROS không phải là một phương thức truyền thông mà ROS là một Meta Operating System. ROS cung cấp một môi trường để lập trình Robot, mô phỏng, và quản lí các file theo các Package, meta package.

c) Phạm vi ứng dụng

Ngày nay rôbot thường được phát triển theo cách kết hợp các module có sẵn để tiết kiệm thời gian tính toán phần cứng, cùng với đó lập trình trên một HĐH Robot. ROS là một HĐH miễn phí và có rất nhiều ưu điểm như:

Khả năng sử dụng lại chương trình

ROS được quản lí theo các package và meta package. Một package có đầy đủ thành phần cần thiết để có thể chạy gói (bao gồm các thư viện, tập hợp các dữ liệu khác nhau). Mỗi package sẽ dùng để xây dựng nên một ứng dụng nào đó trong ROS với một mục đích xác định. Meta package là tập hợp của các package cho 1 ứng dụng lớn hơn. Vì thế khi xây dựng một dự án ROBOT,

ta có thể dễ dàng sử dụng lại những gì đã phát triển, hoặc sử dụng những opensource của các nhóm khác nhau trên thế giới nhờ vào các package. Việc này giúp giảm thời gian phát triển đi đáng kể cũng như giúp tiết kiệm nguồn nhân lực vào việc phát triển Robot từ đầu.

ROS có khả năng thay thế các nền tảng IOT

ROS hoạt động dựa vào việc giao tiếp của các node với nhau, mỗi node đảm nhiệm một chức năng xác định (module hóa). Các node giao tiếp với nhau qua mạng, gần giống như các nền tảng IOT. Thật vậy, khi phát triển một dự án Robot, thông thường sẽ phát triển một chức năng điều khiển từ xa thông qua smartphone hoặc qua website. Nếu sử dụng ROS thì chỉ cần tạo một node trên Android hoặc trên PC và node này sẽ gửi các lệnh điều khiển đến node liên quan để di chuyển trên Robot. Như vậy ROS cơ bản là có thể thay thế được một nền tảng IOT

ROS có cộng đồng phát triển lớn mạnh

ROS có cộng đồng phát triển năng động. Có hơn 5.000 package đã được phát triển và chia sẻ kể từ năm 2017. Các trang Wiki về các package của họ vượt quá 18.000 trang bởi sự đóng góp của những người phát triển cá nhân. Hơn nữa, một phần quan trọng khác của cộng đồng là Q&A đã vượt quá 36.000 bài đăng, tạo ra một cộng đồng phát triển hợp tác. Chúng ta có thể dễ dàng tìm được những package liên quan đến dự án phát triển robot của mình, hay là những vấn đề nan giải gặp phải khi xây dựng và ứng dụng các package lên Robot. Đây cũng là một lợi thế không hề nhỏ của ROS so với các HĐH phát triển Robot tương tự ROS.

ROS hỗ trợ nhiều HĐH, nhiều ngôn ngữ lập trình

ROS hỗ trợ nhiều ngôn ngữ lập trình như C/C++, Python, arduino... điều này làm cho ROS có thể kết nối nhiều thứ với nhau. Ví dụ như bạn có thể sử dụng một mạch Arduino để đọc dữ liệu cảm biến và gửi về robot, Robot sẽ sử dụng những dữ liệu này để bằng node được viết bằng code Python và có thể có thể chạy trên hầu hết các HĐH thông thường như Linux, Android...

Chính vì những ưu điểm trên, hầu hết các rôbot tự hành thông minh đều tích hợp hệ điều hành ROS để tối ưu nhất những hiệu quả của hệ điều hành này. Ứng dụng điển hình của HĐH này ta có thể thấy trong những rôbot như: Robot Kuri (H1.9); TurtleBot;...

3.1.2 Bài toán SLAM

a) Giới thiệu bài toán SLAM

SLAM (Simultaneous Localization and Mapping) là một thuật toán giúp robot thực hiện cả việc khám phá xây dựng bản đồ về môi trường xung quanh và việc định vị robot trong môi trường đó một cách đồng thời. SLAM chính là công nghệ cốt lõi cho các bài toán điều hướng như các công nghệ tự hành, thám hiểm.

Có thể tóm tắt bài toán SLAM như sau:

Các thông số đầu vào:

- Các tín hiệu chuyển trạng thái của robot:

$$U_{1:T} = \{u_1, u_2, \dots, u_T\}$$

- Thông tin đo được từ các cảm biến (lidar, siêu âm, hồng ngoại,...)

$$Z_{1:T} = \{z_1, z_2, \dots z_T\}$$

Cần tìm:

- Bản đồ xung quanh Robot:

$$m = \{m_1, m_2, \dots m_n\}$$
- Đường đi của robot (đã biết điểm bắt đầu x_0):

$$X_{1:T} = \{x_0, x_1, \dots x_T\}$$

SLAM là một bài toán rất khó bởi vì:

- SLAM là một bài toán dạng “con gà và quả trứng”. Để định vị được rôbot trong môi trường thì cần có bản đồ môi trường xung quanh. Nhưng để xác định môi trường xung quanh cần phải biết vị trí hiện tại của rôbot, cùng với khoảng cách đo được để dựng nên bản đồ hiện tại;
- Khi đưa rôbot vào một môi trường mới, rôbot không hề có thông tin gì và môi trường biến đổi liên tục nên việc định vị và xác định trở nên khó hơn nhiều so với một môi trường mô phỏng.
- Sai số của các cảm biến: Cảm biến hiện tại thường có độ chính xác nhất định, cùng với sai số tích lũy trong quá trình di chuyển.
- Tốc độ: Các thuật toán SLAM cần phải đảm bảo tốc độ xử lý, vì nếu gặp vật di chuyển nếu rôbot không xử lý đủ nhanh thì sẽ không thể di chuyển tránh kịp thời.

Các bài toán nhỏ trong bài toán SLAM:

- Tạo bản đồ (Gmapping);
- Định vị (Localization);
- Tìm đường (Path planning);
- Tránh vật cản (Obstacle avoidance).

b) Hướng giải quyết

Bài toán tạo bản đồ:

Tạo bản đồ trong ROS được thực hiện bằng cách chia nhỏ không gian thành các ô lưới có kích thước nhỏ bằng nhau (các Cell). Một ô lưới có thể không có vật cản, hoặc có vật cản. Cuối cùng sau khi tạo bản đồ xong, bản đồ sẽ được lưu lại dưới dạng một bản đồ tĩnh gồm các giá trị của các ô lưới. Trong đó, giá trị ô lưới bằng 0 nếu vị trí của ô lưới đó không có vật cản, bằng 100 nếu có vật cản và bằng -1 nếu chưa khám phá.

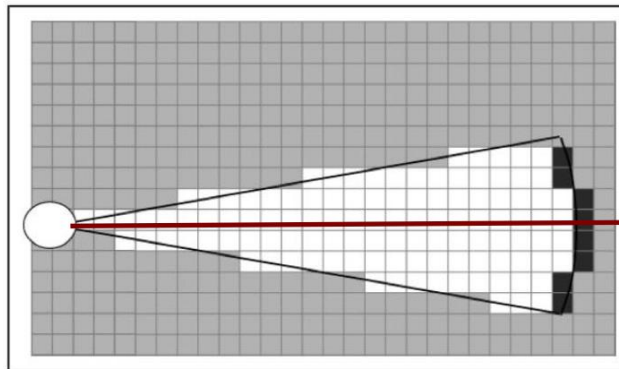
Việc tạo bản đồ chính là việc xác định giá trị của các ô lưới.

Trong thực tế, người ta thường sử dụng cảm biến phạm vi Kinect hoặc quét laser bằng cảm biến laser (lidar). Và việc xác định giá trị của các ô lưới dựa trên nguyên lý xác suất do tín hiệu cảm biến có thể chịu ảnh hưởng nhiều từ môi trường.

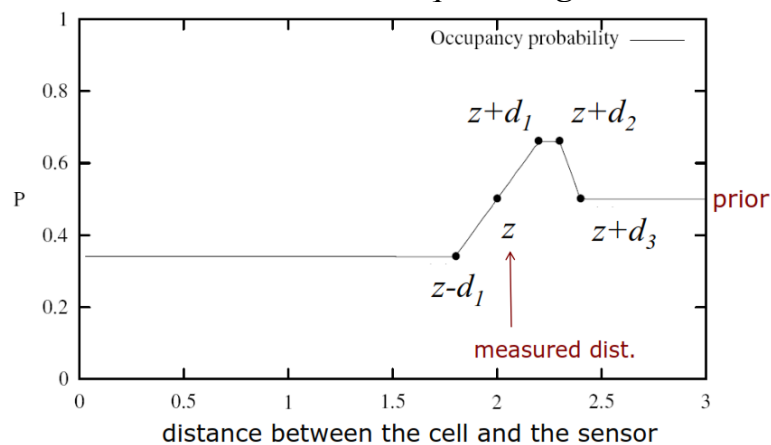
Ví dụ xác định giá trị các ô lưới khi sử dụng cảm biến siêu âm:

Đặc điểm của Sonar sensor là nó sẽ phát sóng siêu âm đi và đọc khoảng cách mà sóng phản xạ lại, nó sẽ không biết chính xác vật cản ở đâu trên cung tròn mà chỉ biết tại khoảng cách đó có vật cản. Nên các ô lưới nằm trên cung tròn chưa điểm phản xạ sẽ được coi là vật cản. Khi đó, các ô lưới có trọng

tâm nằm bên trong hình nón được coi là trống (không có vật cản), các ô lưới có trọng tâm ở ngoài hình nón được coi là chưa khám phá. Cụ thể trong H3.5:



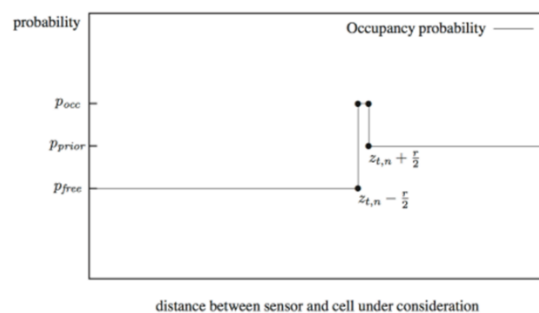
Hình 3.5 Màu của các ô lưới khi quét bằng cảm biến siêu âm



Hình 3.6 Biểu đồ xác suất có vật cản khi sử dụng cảm biến siêu âm

z là kết quả cảm biến đo được. Vì phía trước z là vùng trống, sau z là vật cản nên tại ô lưới ứng với z ta chỉ có xác suất có vật cản là 0,5. Và xác suất sẽ biến đổi một cách tuyến tính nên đồ thị sẽ có dạng H3.6. Vì vật cản có chiều dày nên ta coi các ô lưới từ z đến $(z + d_3)$ là vùng có vật cản. Vì vậy xác suất có vật cản sẽ tăng từ 0.5 tại z đến giá trị xác suất quy định rồi giảm về xác suất 0.5 tại $(z + d_3)$. Vì phía sau khoảng cách đo được ta không có thông tin gì nên xác suất có vật cản sẽ là 0.5. Các tham số d_1, d_2, d_3 được lựa chọn sao cho phù hợp với thực tế nhất.

Inverse Sensor Model for Laser Range Finders



Hình 3.7 Biểu đồ trên thể hiện xác suất theo khoảng cách đối với laser

Tương tự với cảm biến siêu âm, khi cảm biến laser đo được khoảng cách là z thì ô lưới ứng với khoảng cách z sẽ có vật cản (có xác suất là P_{occ}). Nhưng

vì vật cản có độ dày nhất định nên tại ô lưới ứng với khoảng cách nhỏ hơn $(z - \frac{r}{2})$ sẽ được coi là không có vật cản (có xác suất là P_{free}). Và các ô lưới ứng với khoảng cách lớn hơn $(z + \frac{r}{2})$ sẽ là vùng chưa khám phá (có xác suất là P_{prior}). Giá trị r cũng sẽ được lấy thông qua thực nghiệm.

Bài toán định vị:

Particle Filter là một kỹ thuật để dự đoán vị trí robot thông qua mô phỏng dựa trên phương pháp thử sai. Trong phương pháp này, vị trí ban đầu robot được mô tả bởi rất nhiều các điểm gọi là mẫu. Ban đầu vị trí của robot là bất kì trong bản đồ, có thể đúng hoặc sai. Sau đó khi robot di chuyển, các điểm mẫu này cũng dịch chuyển đến một vị trí và hướng mới dựa trên mô hình chuyển động và xác suất của robot. Đồng thời xác suất của mỗi điểm cũng được tính dựa theo giá trị thực đo được. Cuối cùng là loại bỏ các điểm có xác suất thấp để đạt được vị trí robot chính xác nhất. Đối với mobile robot, mỗi điểm được mô tả dưới dạng:

a particle = pose (x,y,i), weight;

x, y: là vị trí và hướng ước tính của robot;

i: là phần trăm khả năng đúng của mỗi điểm mẫu.

Chu trình lọc các điểm mẫu được chạy theo 5 bước. Ngoài bước 1 là khởi tạo, từ bước 2 đến 5 sẽ liên tục lặp lại để ước tính vị trí robot. Nói theo cách khác, đây là phương pháp để ước tính vị trí robot bằng cách liên tục cập nhật mô hình phân bố các điểm mẫu thể hiện vị trí robot trong mặt phẳng.

Bước 1 - Khởi tạo – Initialization:

Bởi vị trí và hướng ban đầu robot là chưa biết, các điểm được sắp xếp ngẫu nhiên trong khu vực mà ở đó vị trí và hướng robot có khả năng là đúng. Ta có N điểm, mỗi điểm có phần trăm đúng là $\frac{1}{N}$, và tổng khả năng là 1. Nếu vị trí ban đầu được cho trước, các điểm này sẽ đặt gần robot.

Bước 2 - Dự đoán – Prediction:

Khi robot di chuyển, cùng lúc các dữ liệu đo lường từ cảm biến, các điểm dự đoán cũng di chuyển theo.

Bước 3 - Cập nhật – Update:

Dựa trên dữ liệu từ cảm biến, xác suất của mỗi điểm được tính và phần trăm khả năng đúng được cập nhật theo.

Bước 4 - Dự đoán vị trí và hướng - Pose Estimation:

Vị trí, hướng và phần trăm khả năng của tất cả các điểm được dùng để tính các giá trị trung bình và lớn nhất cho việc ước tính vị trí robot.

5 - Lấy lại mẫu - resampling

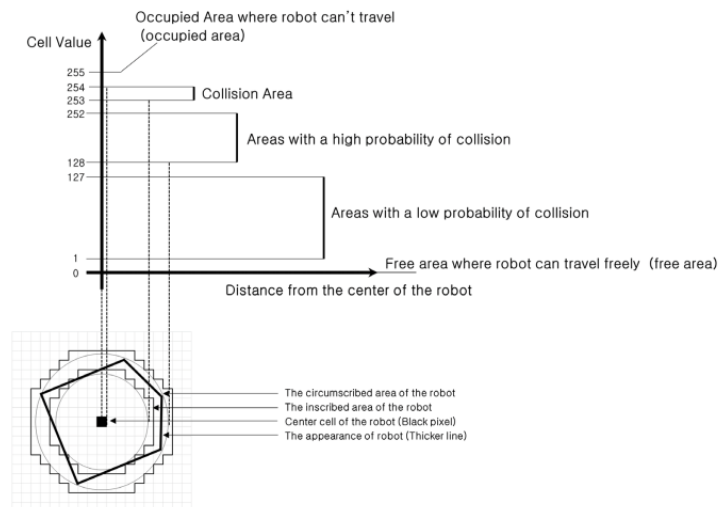
Bước này sẽ loại bỏ những điểm có khả năng chính xác thấp để tạo những điểm mới chính xác hơn. Số lượng N điểm phải được giữ nguyên.

Bài toán tìm đường và tránh vật cản:

Có 3 map mà chúng ta thường sử dụng:

- Static_map: bản đồ tĩnh đã quét được từ trước, trong bước tạo bản đồ.
- Local_cost_map: bản đồ sử dụng để tránh vật cản và tìm quỹ đạo di chuyển trực tiếp cho robot.

- **Global_cost_map**: bản đồ tổng thể dựa trên **static_map**, dùng để tìm quỹ đạo di chuyển từ điểm bắt đầu đến điểm đích.
Costmap là bản đồ trong đó mỗi cell có một giá trị theo sơ đồ ở dưới:



Hình 3.8 Giá trị của các ô lưới trong Costmap

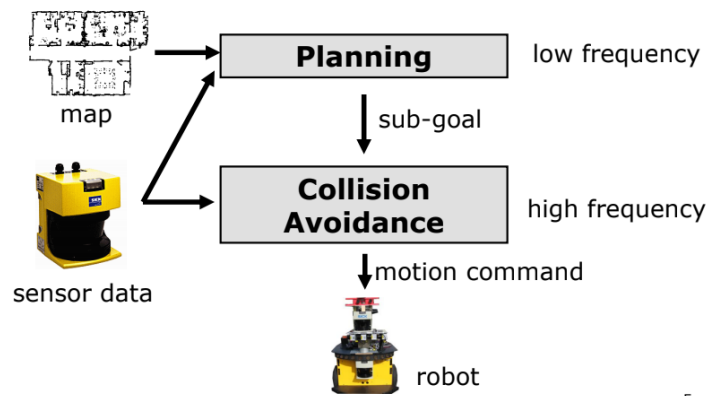
Theo sơ đồ H3.8, các ô lưới trong vùng bị chiếm bởi vật cản sẽ có giá trị là 255 (màu đen). Giá trị của các ô lưới giảm dần xuống 253, 254 là vùng vật cản. Ngoài ra để đảm bảo tránh va chạm vật cản thì robot không nên di chuyển đến vùng gần vật cản quét được, vùng này các ô lưới có giá trị từ 128 đến 252. Các ô lưới có giá trị từ 1 đến 127 tương ứng với vùng có xác suất thấp va chạm với vật cản. Và khi giá trị của ô lưới là 0 tương ứng với vùng không có vật cản.

Dựa vào Costmap, ta sẽ tìm quỹ đạo sao cho số ô lưới trên quỹ đạo và Cost - một giá trị để tính điểm cho quỹ đạo là có giá trị thấp nhất. Khi robot di chuyển từ điểm ban đầu đến điểm đích, giá trị tính điểm cho quỹ đạo được đánh giá qua một số tiêu chí sau:

- Số ô lưới trong quỹ đạo là ít nhất (quãng đường di chuyển ngắn nhất);
- Khoảng cách của quỹ đạo tới vật cản (được chia thành 3: khu vực vật cản – robot không đi vào, nếu vào sẽ báo va chạm. Khu vực gần vật cản – hạn chế di chuyển vào vùng này nhưng khi cần thiết vẫn sẽ đi vào. Khu vực trống – là không gian còn lại)

Sơ đồ nguyên lý tìm đường và tránh vật cản (H3.9):

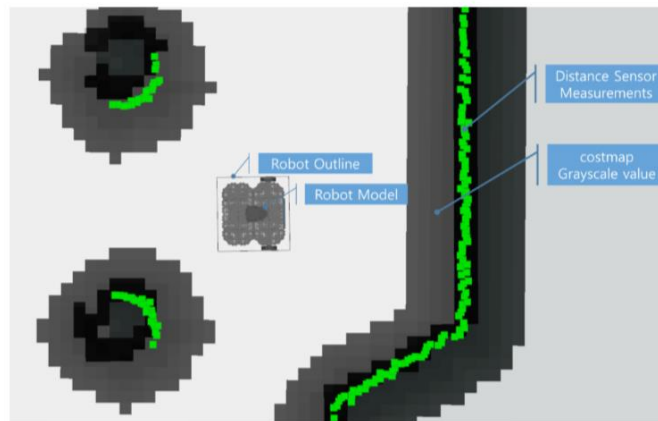
Classic Two-layered Architecture



5

Hình 3.9 Sơ đồ nguyên lý tìm đường và tránh vật cản

Khi chọn một điểm đích, rôbot sẽ tạo ra một quỹ đạo chung đi từ điểm hiện tại đến điểm đích dựa vào bản đồ Global_costmap và các dữ liệu từ cảm biến - Planning. Quỹ đạo này là quỹ đạo có giá trị Cost thấp nhất và được cập nhật với tần số cập nhật thấp. Sau đó Robot sẽ bắt đầu di chuyển dựa theo một quỹ đạo cục bộ là local_planner được tạo dựa vào dữ liệu của local_costmap và dữ liệu từ các cảm biến. Quỹ đạo này sẽ bám theo quỹ đạo chung và tránh các vật cản mới xuất hiện trong local_costmap.



Hình 3.10 Bản đồ Costmap với các vùng có màu tương ứng

Có 2 loại quỹ đạo chính được sử dụng trong ROS là:

- Global_planner: Sử dụng các thuật toán để tính quỹ đạo từ điểm bắt đầu đến điểm đích trên bản đồ.
- Local_planner: Quỹ đạo cục bộ của rôbot và kết hợp với tránh vật cản.

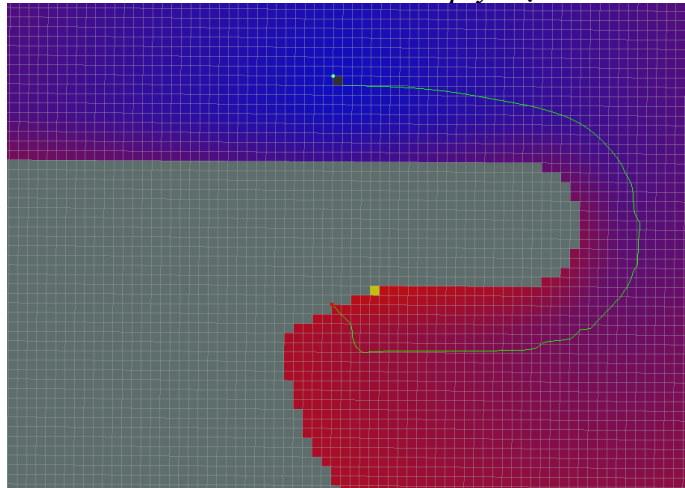
Các tham số trong chương trình tìm Global planner:

```

1
2 GlobalPlanner:
3   old_navfn_behavior: false
4   default false
5   use_quadratic: true
6   calculation, default true
7   use_dijkstra: true
8   use_grid_path: false
9   method, default false
10
11 allow_unknown: true
12
13 costmap_commons_param) to work
14 planner_window_x: 0.0
15 planner_window_y: 0.0
16 default_tolerance: 0.4
17 default 0.0
18
19 publish_scale: 100
20 planner_costmap_publish_frequency: 0.0
21
22 lethal_cost: 253
23 neutral_cost: 66
24 cost_factor: 0.55
25 publish_potential: true
26 default true

```

Hình 3.11 Các tham số của hàm tìm quỹ đạo Global planner



Hình 3.12 Quỹ đạo di chuyển của rôbot khi đặt các tham số theo mặc định

Giải thích các tham số:

old_navfn_behavior:

Một dạng thuật toán cũ dùng để tạo path, tuy nhiên nó thường gặp các vấn đề như là tạo path đi xuyên qua tường, và các nhược điểm khác:

- Vị trí bắt đầu không trùng với vị trí thực tế;
- Quỹ đạo di chuyển khi gần đến đích sẽ đi theo các đường lưới;
- Hệ tọa độ lệch đi nửa cell.

Nếu đặt tham số này bằng “true” Robot sẽ tìm quỹ đạo theo thuật toán cũ này.

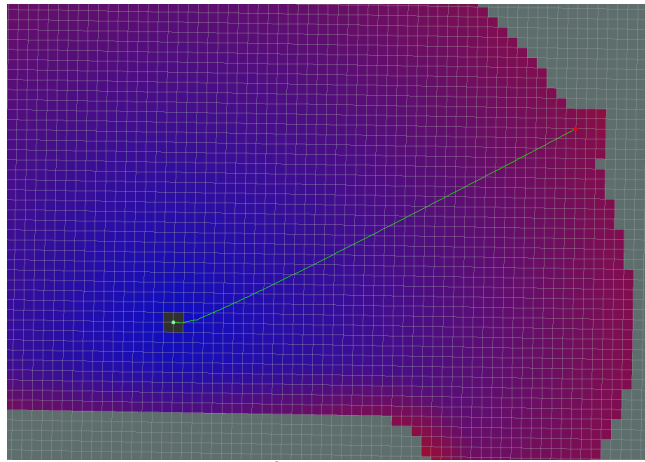
use_quadratic:

Nếu true, sử dụng quadratic approximation of potential;

Nếu false, sử dụng phép tính toán đơn giản hơn.

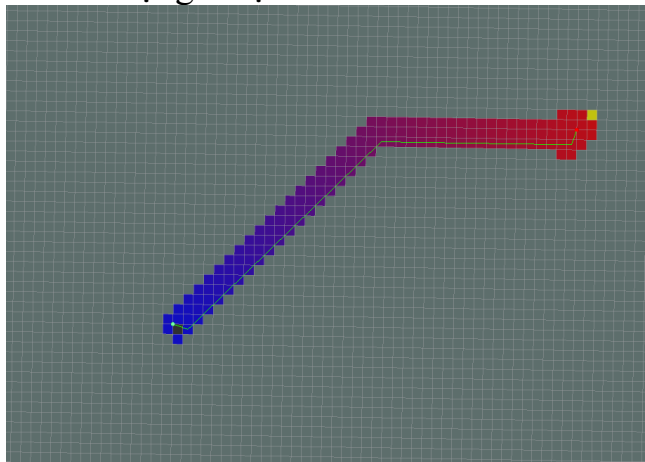
use_dijkstra:

Nếu use_dijkstra = true → sử dụng thuật toán Dijkstra;



Hình 3.13 Quỹ đạo di chuyển của Robot khi sử dụng thuật toán Dijkstra

Nếu set là false → sử dụng thuật toán A*

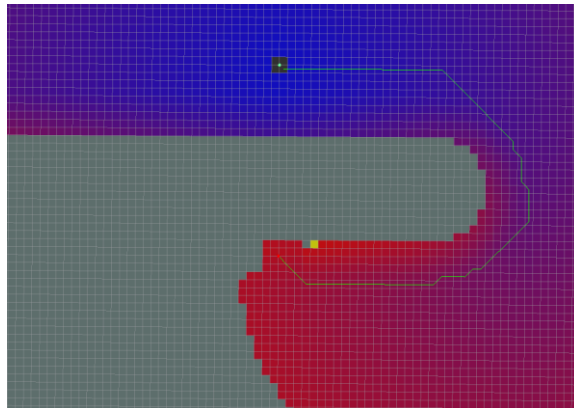


Hình 3.14 Quỹ đạo di chuyển của rôbốt khi sử dụng thuật toán A*

A* nhanh hơn Dijkstra nhưng lại có nhược điểm là tạo ra các path giống nhau không cần thiết. Các potentials được tính toán dựa trên bốn cell gần nhau trong khi tạo path bằng track ngược từ Goal trở về start sử dụng mô hình 8 cell thay vì 4.

use_gridpath:

Nếu true thì path tạo ra sẽ phải đi theo đường lưới hoặc đường chéo. Mặc định để False



Hình 3.15 Quỹ đạo di chuyển của Robot khi sử dụng thuật toán Grid Path

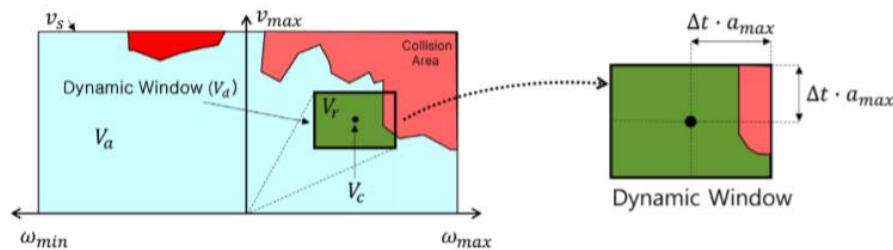
Local planner:

Có hai local_planner được ứng dụng như: base_local_planner & dwa_local_planner. Trong đó, dwa_local_planner hỗ trợ điều khiển theo x, y và trục quay θ , trong khi base_local_planner chỉ hỗ trợ di chuyển theo x và góc θ . Dwa_local_planner được viết lại từ base_local_planner, với hiệu quả di chuyển tốt hơn nên dần được sử dụng nhiều hơn.

DWA – Dynamic Window Approach:

DWA là phương pháp phổ biến trong việc lên quỹ đạo tránh vật cản và tránh vật cản. Phương pháp này lựa chọn một tốc độ mà có thể nhanh chóng tiến đến điểm đích và tránh vật cản mà có thể có khả năng cao va chạm với rôbot. Trong ROS, DWA đang được ứng dụng nhiều bởi tính năng vượt trội.

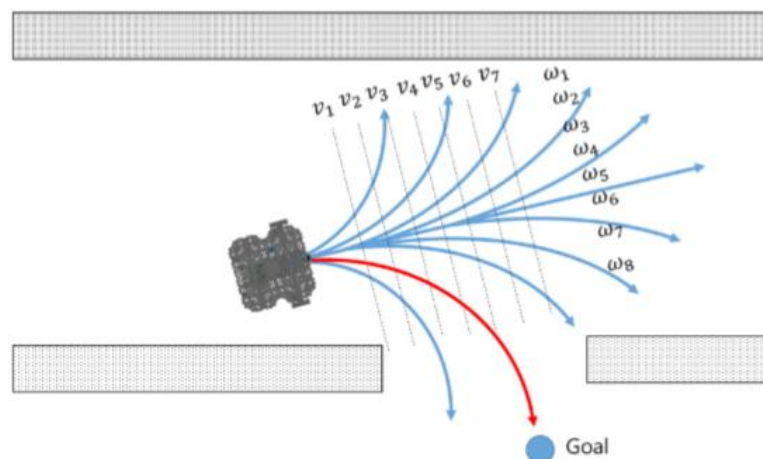
Đầu tiên, rôbot không ở trong hệ tọa độ $x-y$, mà ở trong không gian tìm kiếm tốc độ (velocity search space) với vận tốc thẳng V và vận tốc quay ω . Trong không gian này, rôbot có tốc độ tối đa cho phép bởi phần cứng. Không gian đó được gọi là Dynamic Window.



Hình 3.16 Các không gian vận tốc trong thuật toán DWA

Hình chữ nhật to với $V_{\max}$ và $\omega_{\max}$ tạo nên Velocity Window. Còn hình chữ nhật nhỏ hơn dựa trên gia tốc tối đa của phần cứng rôbot nhân với thời gian ΔT bằng 0.25s, với V_c là giá trị vận tốc hiện tại. Vùng màu xanh nhạt V_d là vùng vận tốc Robot di chuyển tự do không có va chạm, mọi V_c trong vùng này đều thỏa mãn để Robot di chuyển không bị va chạm. Vùng màu đỏ gọi là Collision Area – vùng va chạm. Để xác định vùng này cần sử dụng kết hợp với cả Local_costmap - bản đồ có tần số cập nhật cao.

Trong Dynamic Window, hàm $G(v, \omega)$ được dùng để tính vận tốc thẳng v và vận tốc quay ω . Nếu mô hình hóa lên, ta có thể chọn một hướng tối ưu đến Goal.



Hình 3.17 Lựa chọn quỹ đạo tối ưu tới điểm đích

Search Space – Tìm các vận tốc cho phép:

- Circular Trajectory – Quỹ đạo tròn – DWA chỉ kiểm tra các quỹ đạo tròn được tạo bởi cặp vận tốc thẳng và quay. Kết quả tạo ra không gian vận tốc 2 chiều.
 - Khung hình chữ nhật có trục tung là V_x và trục hoành là V_θ .
- Vận tốc cho phép – Giới hạn các cặp vận tốc đảm bảo an toàn khi di chuyển, sao cho Robot có thể dừng lại trước khi va chạm với vật cản gần nhất.
 - Từ đó tạo ra các vùng va chạm và vùng vận tốc cho phép (permissible velocity area).
- Dynamic Window – Giới hạn các cặp vận tốc sao cho Robot có thể đạt được dựa trên tình trạng hiện tại và giới hạn phần cứng.

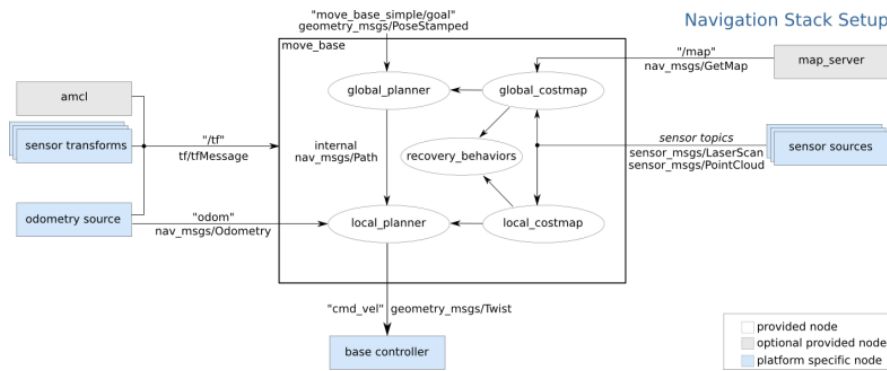
3.1.3 Gói SLAM trong ROS

ROS navigation là một gói tương đối phổ biến, nó có thể sử dụng với hầu hết các loại robot di chuyển. Nhưng cũng có một số yêu cầu:

- Các gói Navigation sẽ làm việc tốt hơn những robot có thể được điều khiển bằng việc gửi lệnh theo dạng: x, y (cặp vận tốc thẳng) và z (vận tốc quay).
- Robot cần có một cảm biến laser phẳng để xây dựng bản đồ và thực hiện bài toán định vị.
- Module để robot nên có dạng hình tròn hoặc hình vuông.

Trình tự robot di chuyển từ điểm hiện tại đến điểm đích:

- **Cảm nhận – sensing:** Trên bản đồ, robot cập nhật dữ liệu từ encoder, cảm biến IMU và đo khoảng cách từ robot đến vật cản nhờ vào cảm biến laser.
- **Định vị - Localization/Pose Estimation:** Dựa trên dữ liệu từ Encoder, IMU và cảm biến laser, gói Navigation sẽ thực hiện ước tính vị trí Robot trên bản đồ có sẵn.
- **Tính quỹ đạo di chuyển – motion planning:** Robot sẽ tạo quỹ đạo từ vị trí hiện tại đến điểm đích. Quỹ đạo bao gồm quỹ đạo tổng thể trên bản đồ chung và quỹ đạo nhỏ xung quanh robot.
- **Di chuyển tránh vật cản – Move/Obstacle Avoidance:** Sau khi có quỹ đạo, robot di chuyển dựa trên quỹ đạo đó. Quá trình tìm quỹ đạo di chuyển của robot cũng được thực hiện liên tục trong suốt quá trình di chuyển tới điểm đích. Các vật cản hoặc vật thể chuyển động đột nhiên xuất hiện sẽ được tránh bởi thuật toán DWA.



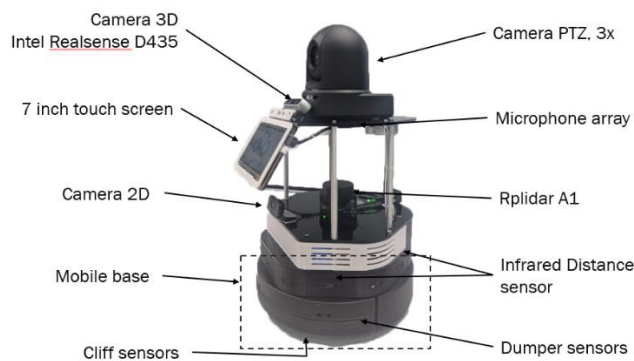
Hình 3.18 Sơ đồ hoạt động của gói Navigation

Giải thích:

- **Odometry source:** thông tin này cho biết vị trí của robot so với điểm trước đó. Dữ liệu về vị trí chủ yếu lấy từ encoder của hai bánh xe, cảm biến gia tốc góc (IMU) và camera 2D/3D (visual Odometry). Giá trị này sẽ được publish dưới dạng một message: `nav_msgs/Odometry`.
- **Sensor source:** dữ liệu cảm biến được sử dụng cho 2 nhiệm vụ là: định vị vị trí của robot trên bản đồ và sử dụng để phát hiện vật cản trong quá trình di chuyển.
- **Sensor transform (/tf):** Dữ liệu từ các cảm biến khác nhau của robot cần được tham chiếu về cùng một hệ tọa độ (thường là hệ tọa độ đặt tại đế robot). Robot publish mối liên hệ giữa hệ tọa độ chính của robot với các hệ tọa độ của các sensor khác nhau vào topic `/tf`.
- **Base_controller:** chức năng chính của `base_controller` là để chuyển hóa tín hiệu ra từ gói Navigation thành dữ liệu điều khiển hai bánh xe của robot.

3.2 Điều khiển và xây dựng giao diện ứng dụng trên Robot Kobuki

3.2.1 Giới thiệu phần cứng



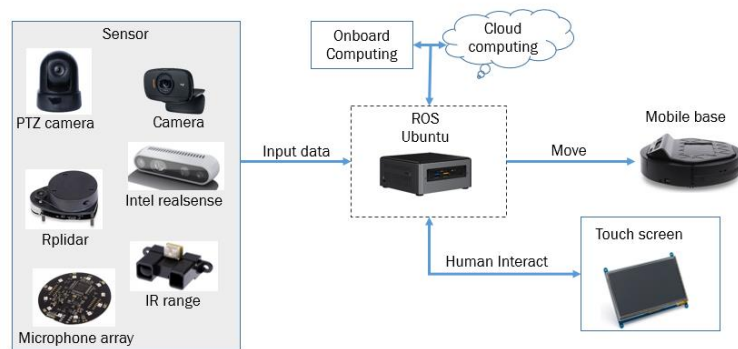
Hình 3.19 Cấu tạo của Robot

Cấu tạo của Robot (H3.19):

- Đế Robot: Đế Kobuki Robot với cảm biến va chạm;
- Các cảm biến đo khoảng cách: cảm biến laser Rplidar A1, và cảm biến hồng ngoại;
- Các camera: camera intel Realsense D435, camera Logitech C525, Camera PTZ 3x;
- Cảm biến âm thanh: Microphone array Respeaker 2.0;

- Màn hình giao tiếp với người dùng: màn hình cảm ứng 7inch;
- Máy tính xử lý trung tâm: PC Intel NUC.

H3.20 là sơ đồ kết nối các phần cứng của rôbốt. Dữ liệu của các cảm biến sẽ được đưa vào máy tính NUC trực tiếp qua các dây kết nối. Máy tính NUC sẽ truyền tín hiệu điều khiển đến để rôbốt cũng thông qua dây kết nối trực tiếp. Đồng thời máy tính NUC cũng sẽ hiển thị giao diện người dung lên màn hình cảm ứng qua cable HDMI.



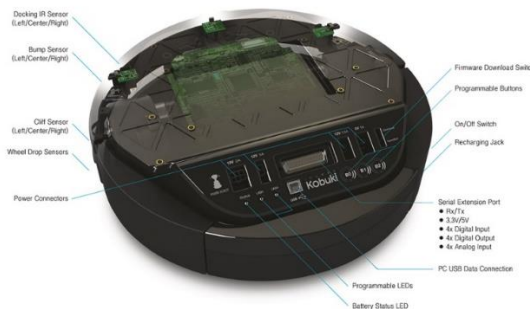
Hình 3.20 Sơ đồ kết nối các phần cứng của Robot

Tuy nhiên, nhóm không sử dụng hết tất cả những thiết bị này trong đồ án, các thiết bị sử dụng được giới thiệu ngắn gọn bằng các đầu mục ngay dưới đây.

a) Đế Kobuki Robot

Module di chuyển của robot tự hành là module quan trọng nhất, nếu không có sẽ không thể được gọi là robot tự hành. Nhưng để xây dựng một module di chuyển hoạt động ổn định, ít xảy ra lỗi sẽ tốn rất nhiều thời gian. Nhưng hiện tại có rất nhiều nhóm phát triển các module di chuyển hoàn chỉnh phục vụ cho học tập và nghiên cứu. Vì thế để giảm thời gian phát triển nên nhóm em quyết định xây dựng robot trên một module di chuyển hoàn chỉnh.

IClebo Kobuki là một cơ sở nghiên cứu mobile robot chi phí thấp được thiết kế cho giáo dục và nghiên cứu về robot. Đế robot Kobuki – module di chuyển của robot có thể cung cấp nguồn điện cho một máy tính bên ngoài cũng như các cảm biến và bộ truyền động bổ sung. Sử dụng con quay hồi chuyển giúp cho đế robot Kobuki có thể đạt độ chính xác cao trong điều hướng di chuyển.



Hình 3.21 Đế Kobuki robot

Khả năng của đế robot Kobuki:

- Vận tốc tịnh tiến tối đa: 70 cm/s;

- Tốc độ quay tối đa: 180 độ/s;
- Tải trọng: 5 kg (sàn cứng), 4 kg (thảm);
- Leo ngưỡng: leo lên ngưỡng 12 mm hoặc thấp hơn;
- Thời gian hoạt động dự kiến: 3/7 giờ (pin nhỏ/lớn);
- Thời gian sạc dự kiến: 1,5 / 2,6 giờ (pin nhỏ/lớn);
- Auto docking: trong phạm vi 2mx5m phía trước trạm sạc.

Đặc điểm kỹ thuật:

- Kết nối PC: USB hoặc qua chân RX / TX trên parallel port;
- Phát hiện quá tải động cơ: vô hiệu hóa khi phát hiện dòng điện cao (> 3A);
- Cảm biến va chạm: trái, giữa, phải;
- Cảm biến thả bánh xe: trái, phải;
- Đầu nối nguồn: 5V/1A, 12V/1.5A, 12V/5A;
- Chân mở rộng: 3.3V/1A, 5V/1A, 4 analog in, 4 digital in, 4 digital out;
- Nút: 3 nút cảm ứng;
- Pin: Lithium-Ion, 14.8V, 2200 mAh (4S1P - nhỏ), 4400 mAh (4S2P - lớn);
- Tốc độ dữ liệu cảm biến: 50Hz;
- Bộ sạc: Đầu vào: 100-240V AC, 50/60Hz, tối đa 1.5A; Đầu ra: 19V DC, 3.16A;
- Docking IR nhận: trái, giữa, phải;
- Đường kính: 351,5mm / Chiều cao: 124,8mm / Trọng lượng: 2,35kg (4S1P - nhỏ).

Các phần mềm:

- Drive C ++ cho linux và windows;
- ROS;
- Mô phỏng Gazebo.

b) PC Intel NUC

Intel PC NUC là một loại mini PC với đầy đủ các loại kết nối thông thường. PC NUC có kích thước nhỏ gọn, có thể dễ dàng bố trí lên Robot mà không tốn nhiều không gian. PC NUC cũng có khả năng hoạt động mạnh hơn so với các máy tính nhúng thông thường, điển hình như Raspberry Pi nên máy tính NUC sẽ phù hợp với những Robot tự hành thông minh ứng dụng các công nghệ cao như công nghệ AI.



Hình 3.22 Intel PC NUC

Các thông số cơ bản:

- Ram 16Gb;

- Processor: intel I5;
- Hard driver: HDD;
- Các cổng kết nối: USB 2.0 và 3.0, HDMI, jack 3.5, ethernet;
- Có hỗ trợ Wireless;
- Kích thước: 115 x 111 x 51mm;
- Nguồn vào: DC 19V.

c) Các loại cảm biến

RpLidar A1M8:

Lidar là một loại cảm biến phổ biến trên robot tự hành nói riêng và các thiết bị di chuyển nói chung. Lidar có khả năng đo kích thước chính xác trong phạm vi rộng gần như liên tục. Điều này vô cùng cần thiết trên một robot tự hành, khi robot phải tự di chuyển ít có sự can thiệp của con người.

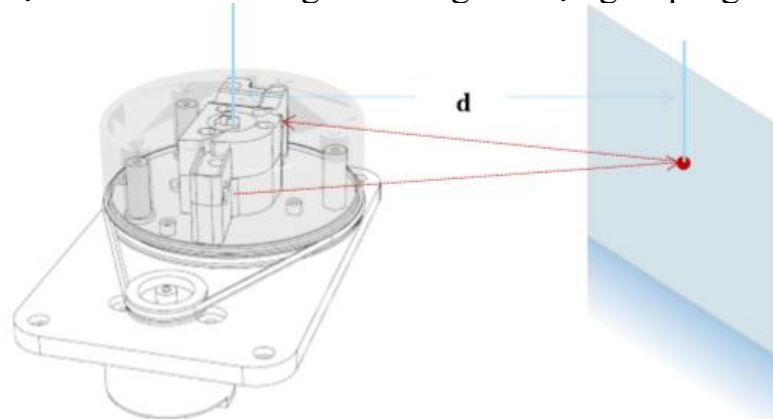
Hệ thống của RPLIDAR A1M8 bao gồm 1 hệ thống quét khoảng cách và một hệ động cơ. Sau khi cấp nguồn cho 2 hệ trên, RPLIDAR A1M8 sẽ bắt đầu quay và quét theo chiều kim đồng hồ. Có thể lấy dữ liệu quét thông qua cổng giao tiếp Serial/USB.



Hình Error! No text of specified style in document..23 Rplidar A1M8

Trong mạch điều khiển có tích hợp phần đo tốc độ và hệ thống tương thích tốc độ quét và tốc độ quay của động cơ một cách tự động. Cũng có thể lấy tốc độ quay thông qua giao tiếp USB.

RPLIDAR phát triển dựa trên nguyên tắc đo khoảng cách bằng tam giác laser và sử dụng hệ thống thị giác máy tốc độ cao và xử lý phần cứng bởi SLAMTEC. RPLIDAR phát ra tín hiệu laser đã được điều chỉnh và nó được phản xạ lại bởi vật thể. Tín hiệu thu về sẽ được xử lý bằng hệ thống nhúng DSP và dữ liệu đầu ra về khoảng cách và góc được gửi qua giao tiếp USB.



Hình 3.24 nguyên tắc đo khoảng cách của Rplidar A1M8

Vấn đề an toàn: Công suất laser nhỏ hơn 5mW. Tia laser phát ra trong 1 khoảng thời gian cực ngắn và có thể đảm bảo tính an toàn tới người và động vật, đạt Class I về tiêu chuẩn an toàn laser.

Các thông số khác:

Item	Unit	Min	Typical	Max	Comments
Band rate	bps	-	115200	-	
Working mode	-	-	8N1	-	8n1
Output high voltage	Volt [V]	2.9	-	3.5	Logic High
Output low voltage	Volt [V]	-	-	0.4	Logic Low
Input high voltage	Volt [V]	1.6*	-	3.5	Logic High
Input low voltage	Volt [V]	-0.3	-	0.4	Logic Low

Hình 3.25 Thông số điện áp tín hiệu giao tiếp Serial UART

Interface	Signal Name	Type	Description	Min	Typical	Max
Motor Interface	VMOTO	Power	Power for RPLIDAR A1 Motor	-	5V	9V
	MOTOCTL	Input	Enable signal for RPLIDAR A1 Motor/PWM Control Signal	0V	-	VMOTO
	GND	Power	GND for RPLIDAR A1 Motor	-	0V	-
Core Interface	VCC_5	Power	Power for RPLIDAR A1 Range Scanner Core	4.9V	5V	6V
	TX	Output	Serial output for Range Scanner Core	0V	-	5V
	RX	Input	Serial input for Range Scanner Core	0V	-	5V
	GND	Power	GND for RPLIDAR A1 Range Scanner Core	-	0V	V5.0

Hình 3.26 Thông số cấp nguồn cho động cơ Rplidar

Item	Unit	Min	Typical	Max	Comments
Scanner voltage	system Volt [V]	4.9	5	5.5	If the voltage exceeds the max value, it may damage the core.
Scanner voltage ripple	system Millivolt[mV]		20	50	High ripple may cause the core working failure.
Scanner system start current	Milliampere (mA)	TBD	500	600	Underpower may cause the startup failure.
Scanner current	system Milliampere (mA)	TBD	80	100	Sleep mode, 5V input
		TBD	300	350	Work mode, 5V input
Motor system voltage	Volt [V]	5	5	10	Adjust voltage according to speed
Motor system current	Milliampere (mA)	TBD	100	TBD	5V input

Hình 3.27 Thông số nguồn Rplidar

d) Các thiết bị khác

Trên robot này có sử dụng một màn hình với mục đích chính là giao tiếp với người dùng. Màn hình có thể hiện các thông tin người dùng cần, ví dụ trong nhà hàng sẽ đưa ra thông tin về menu, trong khách sạn sân bay có thể hiện ra bản đồ, layout cỡ bản của tòa nhà, sân bay,... Màn hình này không chỉ có mục đích như vậy, nó còn giúp dễ dàng đưa code, test chương trình chạy trên robot trong lúc phát triển. Vì thế, một màn hình cảm ứng hỗ trợ nhiều hệ điều hành như “màn hình LCD 7inch HDMI (B)” của hãng Waveshare là thích hợp nhất.

Đặc điểm:

- Kích thước khoảng 165x107mm;
- Độ phân giải 1024x600;
- Màn hình cảm ứng điện dung;
- Hỗ trợ cho nhiều dòng máy tính mini-PC;
- Màn hình có thể làm việc giống như một màn hình máy tính thông thường có cổng kết nối HDMI.



Hình 3.28 màn hình cảm ứng

3.2.2 Điều khiển Kobuki robot trên hệ điều hành ROS

a) Điều khiển Robot

Tải ROS và Package:

ROS là một hệ điều hành Robot miễn phí phù hợp với việc phát triển các dự án Robot theo cách tích hợp các module. ROS hiện tại có thể chạy trên nhiều hệ điều hành, nhưng phù hợp nhất là hệ điều hành Linux. Và với bản ROS Kinetic thì nên cài đặt trên Ubuntu 16.04.

Cách cài đặt ROS khá đơn giản và có hướng dẫn chi tiết trên trang wiki ROS. Các bước cài đặt cơ bản:

- Thiết lập để máy tính cho phép cài phần mềm từ packages.ros.org

```
sudo sh -c 'echo "deb
http://packages.ros.org/ros/ubuntu $(lsb_release -sc)
main" > /etc/apt/sources.list.d/ros-latest.list'
```

- Thiết lập Key của ROS

```
sudo apt-key adv --keyserver
'hkp://keyserver.ubuntu.com:80' --recv-key
C1CF6E31E6B9ADE8868B172B4F42EDFBAB17C654
```

Có thể Key trong báo cáo này không đúng tại thời điểm hiện tại, để đảm bảo không có lỗi thì hãy cập nhật Key mới nhất trên trang Wiki ROS để kiểm tra lại

- Tải và cài đặt ROS

```
sudo apt-get install ros-kinetic-desktop-full
```

- Thiết lập môi trường chạy ROS

```
echo "source /opt/ros/kinetic/setup.bash" >>
~/.bashrc
```

chạy thử ROS bằng lệnh: `roscore`

```
roscore http://192.168.0.104:11311/
zerogzero-X556UR:~$ roscore
... logging to /home/zero/.ros/log/556adda4-bfa3-11ea-b6cf-74c63b87750b/roslaunch-zerogzero-X556UR-5480.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.0.104:45077/
ros_comm version 1.12.14

SUMMARY
=====
PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.14

NODES
auto-starting new master
process[master]: started with pid [5490]
ROS_MASTER_URI=http://192.168.0.104:11311/

setting /run_id to 556adda4-bfa3-11ea-b6cf-74c63b87750b
process[roscout-1]: started with pid [5503]
started core service [/roscout]
```

Hình 3.29 Kết quả chạy lệnh roscore ở terminal trong HĐH Ubuntu
Để chắc chắn đã tải gói Turtlebot – package di chuyển thì cần chạy các lệnh:

```
sudo apt-get install ros-kinetic-turtlebot
ros-kinetic-turtlebot-apps ros-kinetic-turtlebot-
interactions
ros-kinetic-turtlebot-simulator ros-kinetic-kobuki-
ftdi
ros-kinetic-ar-track-alvar-msgs
```

Cài đặt rplidar package:

```
cd ~/catkin_ws/src
git
clone https://github.com/Slamtec/rplidar_ros.git
```

Chú ý cần phải có git để thực hiện git clone:

```
sudo apt update
sudo apt install git
```

Cài package usb_cam – package để sử dụng các camera giao tiếp qua cổng USB:

```
sudo apt-get install ros-kinetic-usb_cam
```

Cài đặt roserial_arduino để giao tiếp với mạch Arduino:

```
sudo apt-get update
```

Tải Arduino IDE tại website của Arduino: <https://www.arduino.cc/>

Cài roserial:

```
sudo apt-get install ros-kinetic-roserial-arduino
sudo apt-get install ros-kinetic-roserial
install ros_lib library
```

Tạo file launch để chạy các gói:

Full_amcl.launch: base + keyboard + gmapping + Lidar + usb_cam

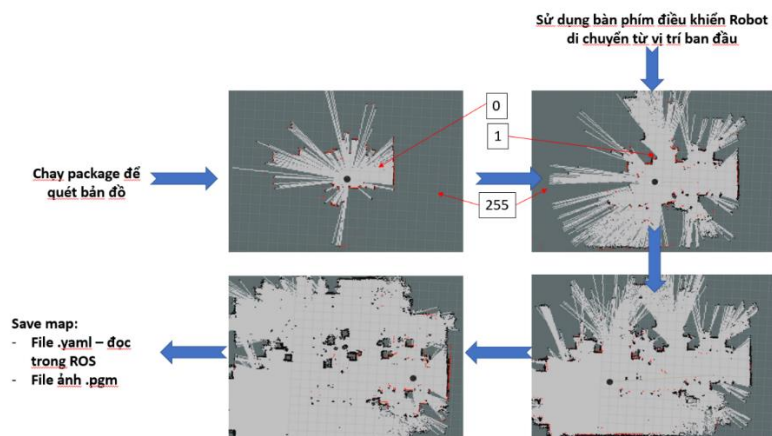
Các bước điều khiển Robot

Bước 1: Quét bản đồ

Để có thể chạy ROS cần phải thực hiện quét bản đồ tĩnh.

Các bước quét bản đồ:

- Chạy package để điều khiển rôbot bằng bàn phím (viết một tệp có định dạng .launch để thực hiện);
- Bật Rviz để có thể quan sát bản đồ trong quá trình di chuyển;
- Sử dụng bàn phím điều khiển rôbot di chuyển quanh phòng đến khi rôbot quét hết toàn bộ phòng;
- Cuối cùng thực hiện lưu bản đồ.



Hình 3.30 Quá trình quét bản đồ tĩnh

Trong quá trình quét bản đồ, có thể gặp các lỗi sau:

- Trong quá trình quét có một vật cản động xuất hiện (ví dụ như người đi vào). Nhưng rôbot lại xác định vật cản đó là vật cản cố định và lưu lại trong bản đồ tĩnh.
- ➔ Khi có người đi qua cần cho rôbot quay 1 vài vòng để rôbot quét lại vị trí có người xuất hiện.
- Khi quét bản đồ, rôbot bị va chạm vào vật cản làm cho bản đồ bị lệch.
- ➔ Cần phải quét lại bản đồ.

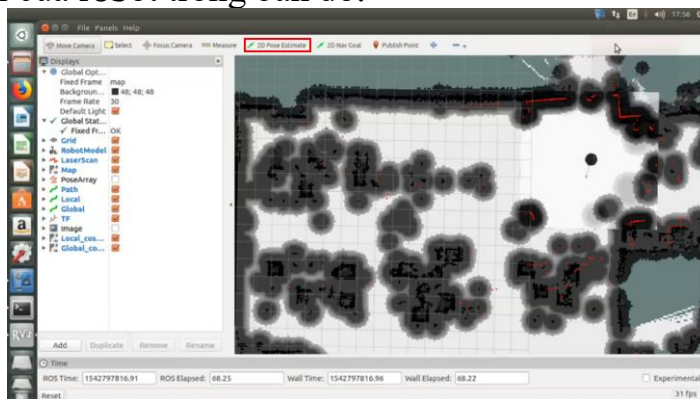
Bước 2: Chọn điểm đích rôbot thực hiện di chuyển

Đầu tiên chạy các gói cần thiết để điều khiển rôbot và các chương trình trong gói Navigation – đã được tích hợp vào file full_amcl.launch.

Các gói sẽ được chạy khi chạy file full_amcl.launch:

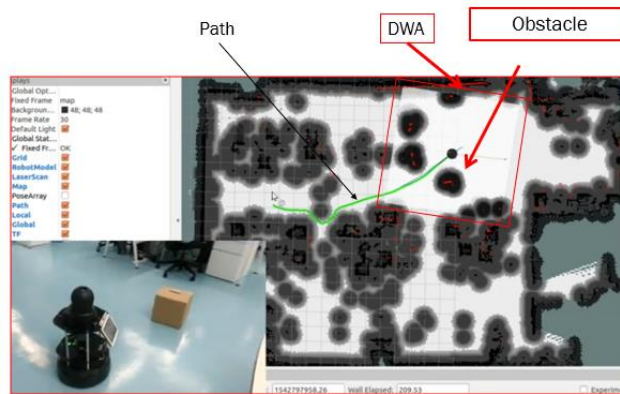
- Gói usb_cam: chạy camera;
- Gói rplidar: chạy rplidar;
- Gói Naviation: gói thực hiện điều khiển rôbot.

Bước tiếp theo là thực hiện chạy Rviz và đặt lại vị trí ban đầu của rôbot cho đúng với vị trí thực tế của rôbot. Sử dụng “2D Pose Estimate” của Rviz để đặt lại vị trí của rôbot trong bản đồ.



Hình 3.31 Rviz

Sau khi đặt đúng vị trí của rôbot so với thực tế, ta sử dụng tính năng “2D Nav Goal” của Rviz để chọn điểm đích. Sau đó rôbot sẽ tự tìm quỹ đạo tới điểm đích và bắt đầu di chuyển:



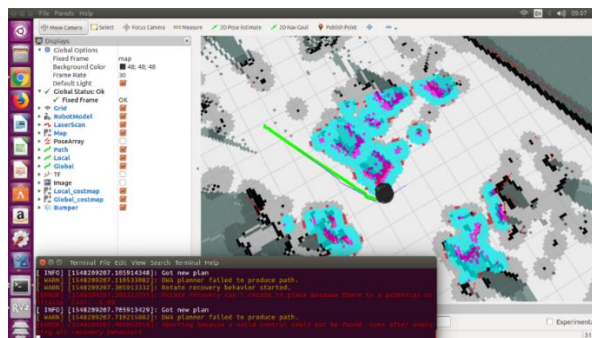
Hình 3.32 Robot di chuyển sử dụng tính năng 2D Nav Goal của Rviz

Tương tự, có thể gửi tọa độ điểm đích và gửi lệnh di chuyển trong một chương trình python hoặc C++, khi đó rôbot cũng sẽ tự tìm đường tới đích và tránh vật cản trong quá trình di chuyển.

b) Các vấn đề di chuyển

Lỗi Robot dễ va chạm khi đi vào đường cong:

- **Mô tả:** khi rẽ trái hoặc rẽ phải, rôbot dễ bị va chạm với vật cản ở góc quay. Do góc rẽ quá nhỏ và rôbot tìm quỹ đạo ngắn nhất dựa trên số lượng ô lưới ít nhất từ điểm bắt đầu tới điểm đích.



Hình 3.33 Lỗi va chạm tại góc rẽ

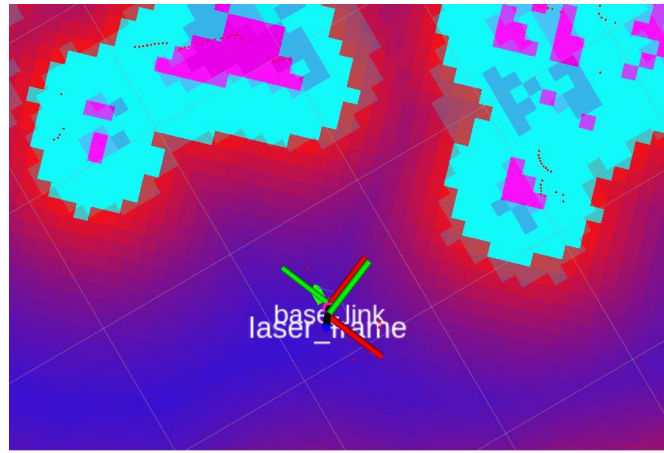
Giải pháp: Điều chỉnh các tham số sau:

Lethal_cost: 253;

Neutral_cost: 66;

Cost_factor: 0.55.

Vùng xung quanh vật cản có giới hạn là 55cm (tham số cost_factor), ngoài vùng này, các ô lưới bất kỳ không nằm trong vùng xung quanh vật cản của bất kỳ vật cản nào sẽ có giá trị là 0 – tức là trống (không có vật cản rôbot có thể di chuyển an toàn). Từ vùng giới hạn này vào vật thể, giá trị đánh giá điểm của quỹ đạo sẽ tăng dần nên rôbot sẽ không ưu tiên đi vào vùng này.



Hình 3.34 Vật cản trong bản đồ costmap

Trong hình trên, các vùng màu xanh dương chính là vùng có giá trị của ô lưới nằm trong khoảng 1-127. Các vùng màu đỏ là vùng có giá trị các ô lưới từ 128 – 252. Planner không ưu tiên đi vào vùng này, nhưng nếu không có đường nào khác thì rôbot sẽ vẫn đi qua khu vực này. Khi đi vào vùng này rôbot sẽ giảm tốc độ để đảm bảo an toàn. Vùng màu hồng vào xanh nhạt là vùng vật cản. Khi rôbot đi vào vùng này, chắc chắn rôbot sẽ không thể di chuyển và báo lỗi.

Lỗi chọn Goal thuộc vùng vật cản:

- **Mô tả:** Ngay khi điểm đích nằm trong vùng vật cản, hoặc có người nào đó đi qua hoặc đứng tại điểm đích thì chương trình DWA sẽ phát thông báo “DWA failed to produce path”.
- **Giải pháp:** Trong package Turtlebot_navigation, trong folder params,– sửa tham số Default_tolerance: 0.0 thành 0.4. trong file navfn_global_planner_param.yaml. Tham số này có ý nghĩa là nếu Goal nằm trong vật cản, planner sẽ tìm một quỹ đạo khác đến gần Goal nhất, trong bán kính được quy định bởi default_tolerance. Tuy nhiên do path được làm mới liên tục nên số lớn dẫn đến điểm mới được chọn cũng thay đổi. Điều này làm chậm quá trình tạo Path hoặc tạo path không thống nhất.

Các tham số qua thực nghiệm trên Robot

Mặc dù rôbot có thể di chuyển nhưng do các tham số được setup sẵn khi tải về sẽ không phù hợp với rôbot hiện tại nên cần phải viết 1 file .launch để chạy Navigation với các tham số sau khi điều chỉnh.

Bảng 3.2 Các tham số điều chỉnh thông qua chạy thực tế

Các thông số cần lưu ý	Giá trị ban đầu	Giá trị thực	Giải thích nguyên nhân sai số
"map_update_interval"	5.0	0.5 giây	0.5(s) cập nhật bản đồ 1 lần
"delta"	0.05	0.1	
"particles"	80	30	Số lượng hạt trong bộ lọc
"maxUrange"	6.0	<12.0: 11.0, 10.0	giá trị này nên để thấp hơn giá trị tối đa của laser
"maxRange"	8.0	12.0	Do laser quét đc 12m
minimumScore	200	0.0	Theo wiki gmapping trên wiki ros
srr	0.01	0.1	Sai số đo lường
srt	0.02	0.2	Sai số đo lường
str	0.01	0.1	Sai số đo lường
stt	0.02	0.2	Sai số đo lường
linearUpdate	0.5	0.05	Thực hiện quét mỗi khi đi được 5 cm
angularUpdate	436	0,0870	Thực hiện quét mỗi khi quay được 1 độ
Xmin	-1.0	-100.0	Kích thước bản đồ ban đầu
Ymin	-1.0	-100.0	Kích thước bản đồ ban đầu
Xmax	1.0	100.0	Kích thước bản đồ ban đầu
Ymax	1.0	100.0	Kích thước bản đồ ban đầu

CHƯƠNG 4- KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN

4.1 Điều khiển tay thao tác và mobile Robot Kobuki

4.1.1 Điều khiển và đánh giá kết quả tay thao tác

Sau quá trình thiết kế và chế tạo, nhóm đã đưa ra được hình ảnh tay thao tác thực tế như H4.1 dưới đây:



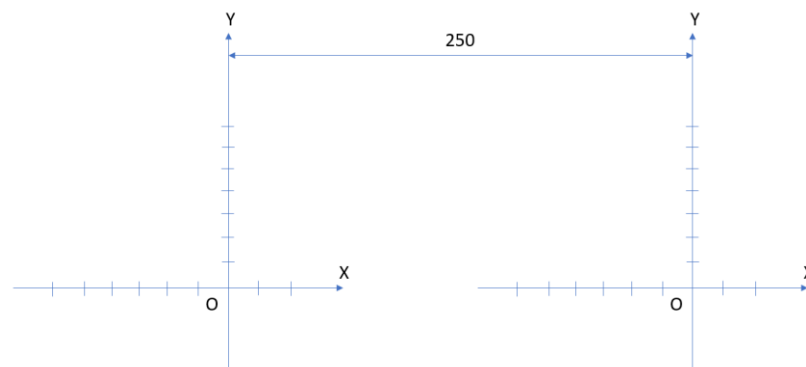
Hình 4.1 Tay thao tác 4 bậc tự do

a. Phương pháp kiểm tra đánh giá độ chính xác của tay thao tác

Cho tay thao tác thực hiện di chuyển theo trình tự được thiết lập từ điểm bắt đầu A (vị trí 1) đến điểm đích B (vị trí 2). Tính toán sai lệch trung bình các điểm di chuyển A và B trong một chu trình lặp lại n lần.

Cách kiểm tra sai lệch vị trí di chuyển A và B:

Gá cố định một đầu bút mực lên phía trên bàn tay gấp. Lần lượt chọn 2 điểm là điểm gốc ban đầu (A) và kết thúc (B) của một chu trình hoạt động. Nhóm xây dựng 2 hệ trục tọa độ Oxy. 2 trục X ngang hàng với nhau, 2 trục Y cách nhau một khoảng là 250mm. Trên các trục X, Y được chia thành các đoạn nhỏ khoảng cách là 10mm (H4.2).

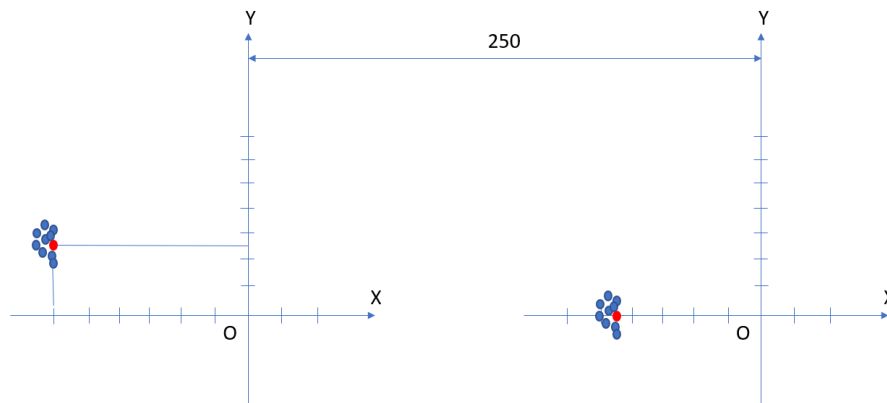


Hình 4.2 Hệ tọa độ đánh giá kết quả của tay thao tác

Để đánh giá có kết quả một cách trực quan nhất, nhóm lựa chọn 2 điểm trên hệ tọa độ Oxy này, với tọa độ 2 điểm lần lượt là: điểm bắt đầu A (-60,25) và điểm kết thúc B(-45,0). Thực hiện quá trình di chuyển giữa 2 điểm này 10 lần với các điều kiện thay đổi như: trường hợp tăng giảm vận tốc thao tác của các khâu, tăng giảm tải trọng thao tác.

Trường hợp 1 (vận tốc chậm, tải nhẹ 50g):

Trong trường hợp này, nhóm lựa chọn tải 50g (khối lượng cây bút dạ). Với vận tốc các động cơ các khớp 1,2,3 lần lượt là: 96(vòng/phút); 6(vòng/phút) và 14.4 (vòng/phút). Dưới đây là hệ tọa độ và bảng số liệu ghi nhận được từ kiểm nghiệm hoạt động của tay thao tác trong trường hợp này.



Hình 4.3 Sơ đồ vị trí các điểm so với điểm gốc TH1 (màu đỏ)

Bảng 4.1 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH1

Thứ tự lần	Tọa độ điểm A _i	Tọa độ điểm B _i
1 (gốc)	(-60,25)	(-45,0)
2	(-61,24)	(-46,0)
3	(-60,24)	(-46,-1)
4	(-60,25)	(-46,-1)
5	(-61,23)	(-47,-2)
6	(-61,26)	(-46,-1)
7	(-62,26)	(-46,0)
8	(-62,26)	(-47,1)
9	(-62,27)	(-47,1)
10	(-63,27)	(-47,-1)

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 1.333 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 1.222 \text{ mm}$$

Sai lệch khoảng cách trung bình của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta_{tb} = \sqrt{\Delta_{xtb}^2 + \Delta_{ytb}^2} = 1.809 \text{ mm}$$

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 1.444 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

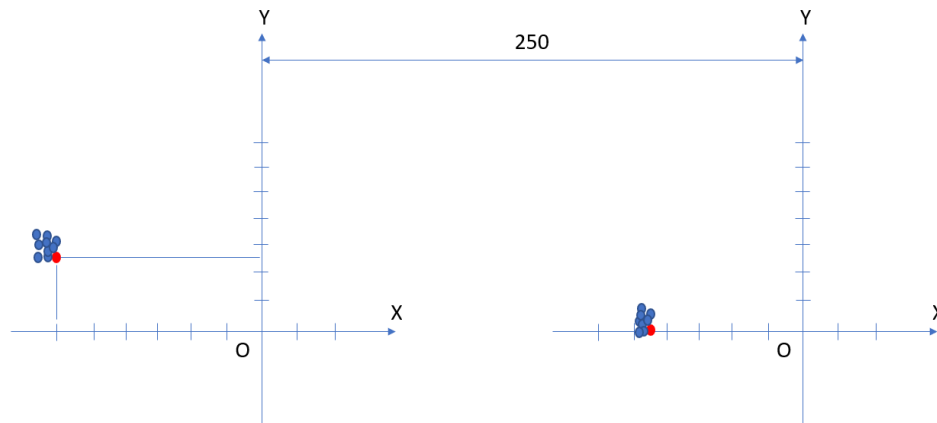
$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 0.888 \text{ mm}$$

Sai lệch khoảng cách trung bình của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta_{tb} = \sqrt{\Delta_{xtb}^2 + \Delta_{ytb}^2} = 1.696 \text{ mm}$$

Trường hợp 2 (vận tốc nhanh, tải nhẹ 50g):

Trong trường hợp này, nhóm lựa chọn tải 50g (khối lượng cây bút dạ). Với vận tốc các động cơ các khớp 1,2,3 lần lượt là: 192(vòng/phút); 12(vòng/phút) và 18(vòng/phút). Dưới đây là hệ tọa độ và bảng số liệu ghi nhận được từ kiểm nghiệm hoạt động của tay thao tác trong trường hợp này.



Hình 4.4 Sơ đồ vị trí các điểm so với điểm gốc TH2 (màu đỏ)

Bảng 4.2 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH2

Thứ tự lần	Tọa độ điểm A _i	Tọa độ điểm B _i
1 (gốc)	(-60,25)	(-45,0)
2	(-61,26)	(-46,0)
3	(-61,26)	(-46,-1)
4	(-62,26)	(-46,-1)
5	(-62,25)	(-46,0)
6	(-62,25)	(-46,1)
7	(-63,26)	(-47,1)
8	(-63,26)	(-47,1)
9	(-63,27)	(-47,2)
10	(-63,27)	(-47,2)

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu A trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 2.222 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu A trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 1 \text{ mm}$$

Sai lệch khoảng cách trung bình của điểm đánh dấu A trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta_{tb} = \sqrt{\Delta_{xtb}^2 + \Delta_{ytb}^2} = 2.436 \text{ mm}$$

. Sai lệch vị trí trung bình theo phương x của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 1.444 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

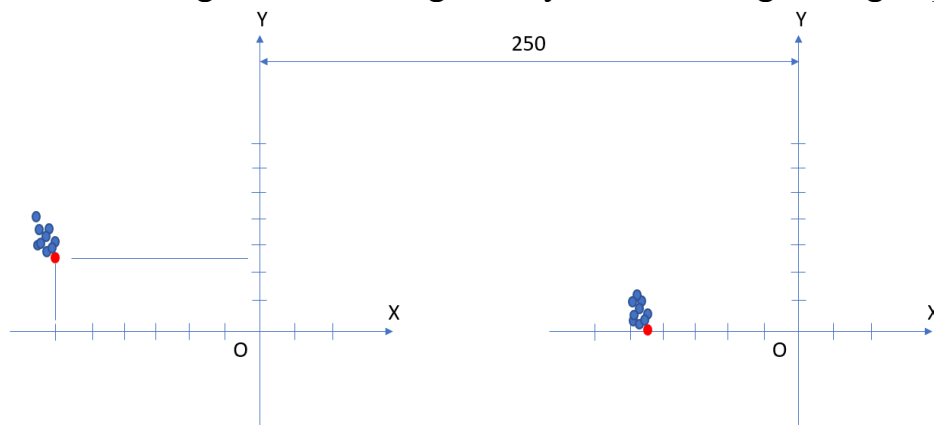
$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 1 \text{ mm}$$

Sai lệch khoảng cách trung bình của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta_{tb} = \sqrt{\Delta_{xtb}^2 + \Delta_{ytb}^2} = 1.756 \text{ mm}$$

Trường hợp 3 (vận tốc chậm, tải nặng 250g):

Trong trường hợp này, nhóm lựa chọn tải 250g (khối lượng cây bút dạ và tải). Với vận tốc các động cơ các khớp 1,2,3 lần lượt là: 96(vòng/phút); 6(vòng/phút) và 14.4 (vòng/phút). Dưới đây là hệ tọa độ và bảng số liệu ghi nhận được từ kiểm nghiệm hoạt động của tay thao tác trong trường hợp này.



Hình 4.5 Sơ đồ vị trí các điểm so với điểm gốc TH3 (màu đỏ)

Bảng 4.3 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH3

Thứ tự lần	Tọa độ điểm A _i	Tọa độ điểm B _i
1 (gốc)	(-60,25)	(-45,0)
2	(-61,26)	(-46,0)
3	(-61,27)	(-46,1)
4	(-62,26)	(-47,1)

5	(-62,26)	(-47,2)
6	(-62,27)	(-47,2)
7	(-63,27)	(-48,2)
8	(-63,28)	(-48,2)
9	(-63,28)	(-48,2)
10	(-64,29)	(-49,3)

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 2.333 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 2.111 \text{ mm}$$

Sai lệch khoảng cách trung bình của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta t_b = \sqrt{\Delta_{x_{tb}}^2 + \Delta_{y_{tb}}^2} = 3.146 \text{ mm}$$

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 2.333 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

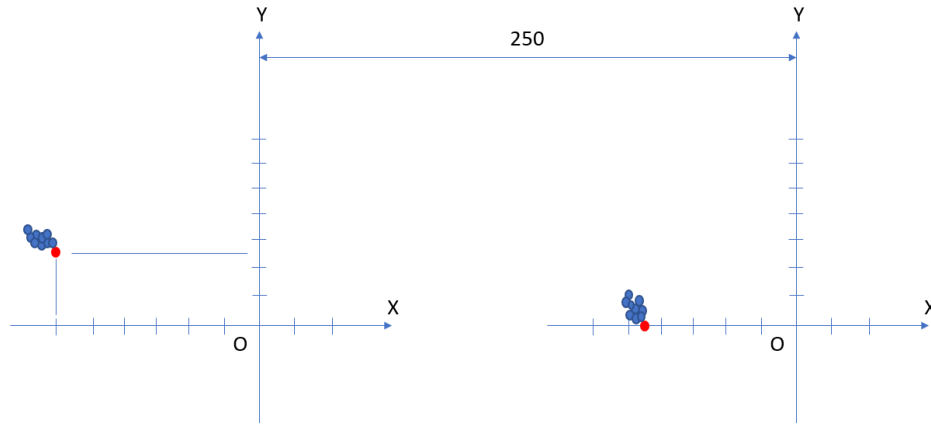
$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 1.667 \text{ mm}$$

Sai lệch khoảng cách trung bình của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta t_b = \sqrt{\Delta_{x_{tb}}^2 + \Delta_{y_{tb}}^2} = 2.867 \text{ mm}$$

Trường hợp 4 (vận tốc nhanh, tải nặng 250g):

Trong trường hợp này, nhóm lựa chọn tải 250g (khối lượng cây bút dạ và tải). Với vận tốc các động cơ các khớp 1,2,3 lần lượt là: 192(vòng/phút); 12(vòng/phút) và 18 (vòng/phút). Dưới đây là hệ tọa độ và bảng số liệu ghi nhận được từ kiểm nghiệm hoạt động của tay thao tác trong trường hợp này.



Hình 4.6 Sơ đồ vị trí các điểm so với điểm gốc TH4 (màu đỏ)
Bảng 4.4 Tọa độ các điểm ghi nhận lại trong 10 lần kiểm tra TH4

Thứ tự lần	Tọa độ điểm A _i	Tọa độ điểm B _i
1 (gốc)	(-60,25)	(-45,0)
2	(-61,26)	(-46,1)
3	(-62,28)	(-47,2)
4	(-62,27)	(-47,2)
5	(-63,27)	(-46,2)
6	(-64,28)	(-48,3)
7	(-65,28)	(-48,3)
8	(-66,29)	(-49,4)
9	(-67,29)	(-50,4)
10	(-68,30)	(-50,-5)

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 4.222 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 3 \text{ mm}$$

Sai lệch khoảng cách trung bình của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta t_b = \sqrt{\Delta_{x_{tb}}^2 + \Delta_{y_{tb}}^2} = 5.180 \text{ mm}$$

Sai lệch vị trí trung bình theo phương x của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^9 |x_{robot,i}|}{9} = 2.889 \text{ mm}$$

Sai lệch vị trí trung bình theo phương y của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^9 |y_{robot,i}|}{9} = 2.889 \text{ mm}$$

Sai lệch khoảng cách trung bình của điểm đánh dấu B trong quá trình di chuyển so với gốc được lưu lại là:

$$\Delta tb = \sqrt{\Delta_{xtb}^2 + \Delta_{ytb}^2} = 4.086 \text{ mm}$$

Nhận xét:

- Trong quá trình di chuyển, tay thao tác không đi đến điểm ban đầu và điểm đích một cách chính xác tuyệt đối, mà thường có sai lệch.

Sai lệch khoảng cách trung bình trong trường hợp 1 (vận tốc chậm, tải nhẹ) là:

$$\Delta tbA = 1.809 \text{ cm}$$

$$\Delta tbB = 1.696 \text{ cm}$$

Sai lệch khoảng cách trung bình trong trường hợp 2 (vận tốc nhanh, tải nhẹ) là:

$$\Delta tbA = 2.436 \text{ cm}$$

$$\Delta tbB = 1.756 \text{ cm}$$

Sai lệch khoảng cách trung bình trong trường hợp 3 (vận tốc chậm, tải nặng) là:

$$\Delta tbA = 3.146 \text{ cm}$$

$$\Delta tbB = 2.867 \text{ cm}$$

Sai lệch khoảng cách trung bình trong trường hợp 4 (vận tốc nhanh, tải nặng) là:

$$\Delta tbA = 5.180 \text{ cm}$$

$$\Delta tbB = 4.086 \text{ cm}$$

Các sai lệch càng lớn khi tốc độ càng cao hoặc tải càng lớn.

Dự đoán nguyên nhân gây ra sai số:

Khi tay thao tác hoạt động, sự không chính xác của các kết cấu cơ khí (độ dơ của hộp giảm tốc động cơ, trượt đai, giãn đai trong quá trình sử dụng) gây nên các ảnh hưởng làm sai lệch khi hoạt động. Ngoài ra, khi có tải lớn và vận tốc hoạt động cao, sự xuất hiện của lực quán tính cũng gây ra các sai số lớn đáng kể. Diễn hình như trong trường hợp số 2 và 4. Đây là bài toán động lực học rôbot phức tạp mà nhóm chưa thể giải quyết được trong phạm vi đồ án này.

Để khắc phục những vấn đề nêu trên, định hướng phát triển của nhóm là lựa chọn những kết cấu chi tiết cơ khí có tính chính xác cao hơn. Thêm vào đó là áp dụng bài toán động lực học ngược để điều khiển một cách chính xác.

4.1.2 Điều khiển và đánh giá kết quả mobile robot Kobuki

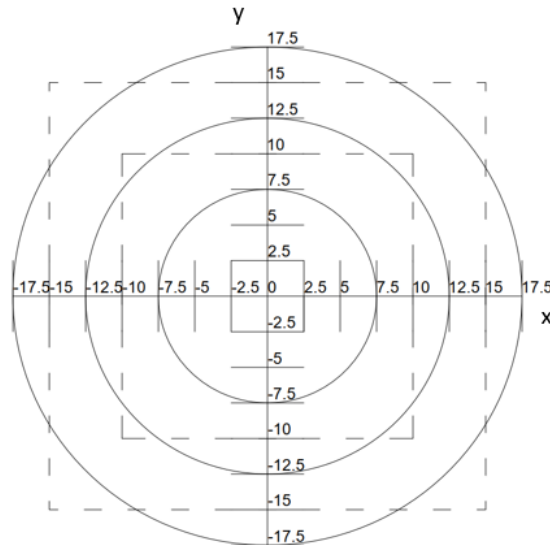
a. Phương pháp kiểm tra độ chính xác của robot tại điểm đích:

Cho robot Kobuki thực hiện di chuyển liên tục từ điểm A (vị trí bắt đầu) đến điểm B (điểm đích). Tính toán sai lệch trung bình của tâm robot so với điểm đích ban đầu được lưu tọa độ trong bản đồ tĩnh để đưa vào chương trình điều khiển robot di chuyển từ A đến B.

Cách kiểm tra sai lệch vị trí tâm của robot tại điểm đích:

Do không nhìn thấy tâm của robot nên khó có thể đo chính xác được sai số của robot so với vị trí ban đầu. Nhưng robot tự hành cũng không quá cần

chính xác trong việc di chuyển tới điểm đích. Vì thế, nhóm quyết định đo tương đối độ sai lệch vị trí bằng cách quan sát đường bao của đế robot có bán kính là 17.75cm. Sử dụng một hệ tọa độ có 2 trục x, y, và một số vòng tròn để quan sát vị trí của đường bao ngoài của đế robot. Vòng ngoài cùng có bán kính 17.5cm xấp xỉ bằng bán kính của đế robot (17,75cm) (H4.7).



Hình 4.7 Hệ tọa độ quan sát sai lệch vị trí của robot ở điểm đích

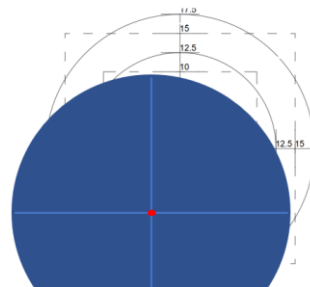
Bước đầu ta sẽ điều khiển robot di chuyển vào chính giữa của hệ tọa độ (sao cho nhìn từ trên xuống sẽ không nhìn thấy đường tròn ngoài cùng có bán kính 17.5cm). Lúc này có thể coi tâm của robot trùng với điểm $O(0;0)$. Sau đó lưu tọa độ của điểm này trên bản đồ tĩnh để đưa vào chương trình điều khiển robot di chuyển.

Khi robot sai lệch xuống dưới, nhìn từ trên xuống ta sẽ nhìn được tọa độ của đường bao ngoài của đế robot là các giá trị x, y dương. Khi đó tọa tâm của robot trong hệ tọa độ kiểm tra sai lệch vị trí là $(X_{robot} ; Y_{robot})$.

$$X_{robot} = X - R_{robot}$$

$$Y_{robot} = Y - R_{robot}$$

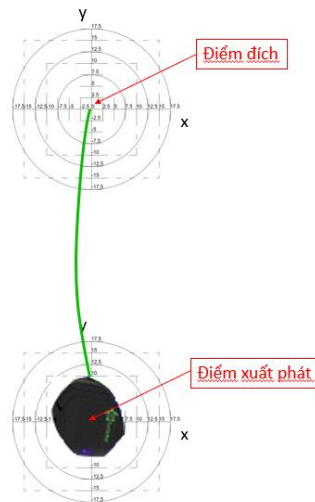
Tương tự với trường hợp robot bị lệch tâm lên trên, thì ta sẽ nhìn thấy tọa độ x, y âm, khi đó tọa độ tâm robot là $(X_{robot} = X + R_{robot} ; Y_{robot} = Y + R_{robot})$.



Hình 4.8 Hình ảnh quan sát từ trên xuống khi tâm robot bị lệch xuống dưới điểm gốc tọa độ

Quá trình thực hiện đánh giá:

- Đánh dấu 2 điểm trên mặt đất bằng hệ tọa độ kiểm tra độ chính xác khi robot đến điểm đích (gọi hệ tọa độ này là hệ tọa độ đo). Hệ tọa độ đo được in trên khổ giấy A2 và dán cố định trên mặt đất.
- Điều khiển robot đến từng điểm đánh dấu sao nhìn từ trên xuống sẽ không thấy đường tròn có bán kính là 17.5cm.
- Lưu tọa độ của từng điểm trong bản đồ tĩnh và đưa vào chương điều khiển robot tự di chuyển qua 2 điểm này.
- Cho robot tự động di chuyển đến 2 điểm liên tục khoảng 10 đến 20 lần, ghi lại giá trị x, y quan sát được vào B4.5.
- Đặt một vật cản gần điểm đích để robot đi theo một quỹ đạo mới, sau đó cũng cho robot di chuyển đến 2 điểm 10 đến 20 lần, ghi lại các giá trị x, y quan sát được vào B4.6.



Hình 4.9 Sự di chuyển của robot kiểm tra độ chính xác robot đến điểm đích

Ghi lại giá trị x, y trên hệ tọa độ của rìa ngoài robot – để xác định tâm chỉ cần trừ đi bán kính của đế robot Kobuki

Bảng 4.5 Bảng giá trị x, y tại điểm đích khi không có vật cản

Lần test	Giá trị quan sát		Tọa độ tâm Robot	
	X1	Y1	X_{robot}	Y_{robot}
1	15	15	-2.5	-2.5
2	15	12.5	-2.5	-5
3	17.5	12.5	0	-5
4	15	7.5	-2.5	-10
5	17.5	10	0	-7.5
6	12.5	10	-5	-7.5
7	12.5	12.5	-5	-5
8	15	7.5	-2.5	-10
9	12.5	12.5	-5	-5
10	10	12.5	-7.5	-5

Bảng 4.6 Bảng giá trị x, y quan sát được tại điểm đích khi có vật cản

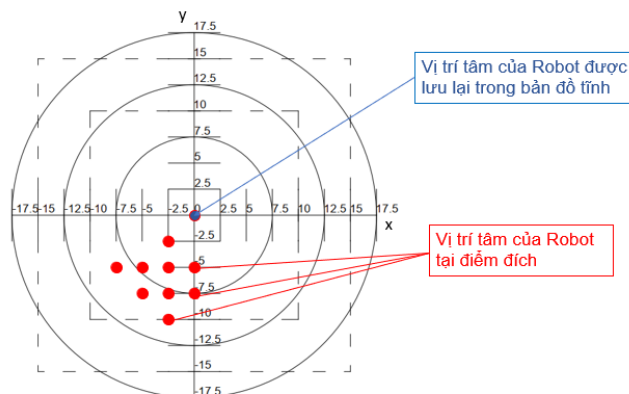
Lần test	Giá trị quan sát	Tọa độ tâm Robot
----------	------------------	------------------

	X1	Y1	X _{robot}	Y _{robot}
1	10	7.5	-7.5	-10
2	-15	10	2.5	-7.5
3	-15	7.5	2.5	-10
4	7.5	5	-10	-12.5
5	-15	12.5	2.5	-5
6	-15	12.5	2.5	-5
7	10	7.5	-7.5	-10
8	-10	12.5	7.5	-5
9	-12.5	12.5	5	-5
10	-12.5	12.5	5	-5

Tính toán kết quả:

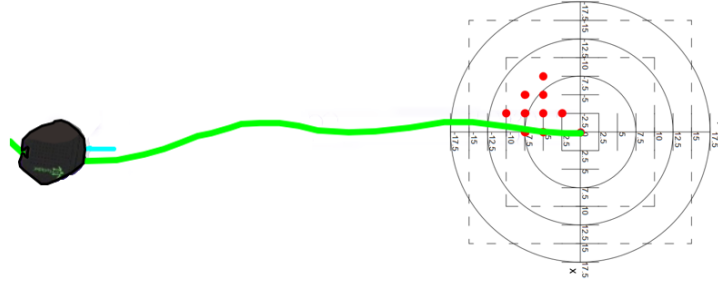
Vì coi ban đầu điểm đích trùng với điểm O(0;0) trên hệ tọa độ đo nên sai lệch vị trí tâm của robot trong quá trình di chuyển với vị trí được lưu lại chính là tọa độ tâm của robot trên hệ tọa độ đo.

Trường hợp không có vật cản.



Hình 4.10 Vị trí tâm của robot trên hệ tọa độ robot dừng lại tại điểm đích khi không có vật cản

Thông qua H4.10 có thể thấy robot không đến chính xác điểm đích được lưu lại vị trí trong bản đồ tĩnh mà có lượng sai số nhất định. Sai lệch lớn nhất theo phương x của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là 7.5cm và theo phương y là 10cm. Tâm của robot trong quá trình di chuyển thực tế chủ yếu nằm trong góc phần tư thứ ba của hệ tọa độ đo, là góc phần tư chứa quỹ đạo di chuyển của robot như H4.11.



Hình 4.11 Quỹ đạo di chuyển của robot khi không có vật cản

Sai lệch vị trí trung bình theo phương x của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^{10} |x_{robot,i}|}{10} = 3.167 \text{ cm}$$

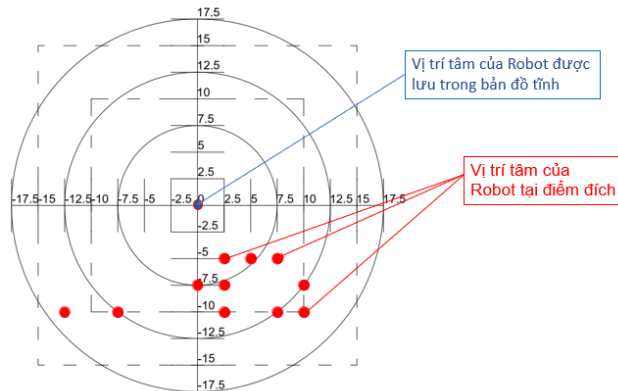
Sai lệch vị trí trung bình theo phương y của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^{10} |y_{robot,i}|}{10} = 6.333 \text{ cm}$$

Sai lệch khoảng cách trung bình của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

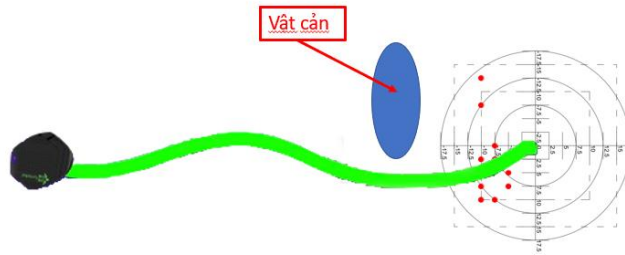
$$\Delta_{tb} = \sqrt{\Delta_{x_{tb}}^2 + \Delta_{y_{tb}}^2} = 7.081 \text{ cm}$$

Trường hợp có vật cản:



Hình 4.12 Tâm của robot trên hệ tọa độ robot dừng lại tại điểm đích khi có vật cản

Tương tự với trường hợp không có vật cản, khi có vật cản robot cũng không đến chính xác điểm đích mà thường có một lượng sai lệch. Sai lệch lớn nhất theo phương x của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là 12.5cm và theo phương y là 10cm. Tâm của robot trong quá trình di chuyển thực tế chủ yếu nằm trong góc phần tư thứ tư của hệ tọa độ đo, là góc phần tư chứa quỹ đạo di chuyển của robot như H4.13.



Hình 4.13 Quỹ đạo di chuyển của robot khi có vật cản

Sai lệch vị trí trung bình theo phương x của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta x_{tb} = \frac{\sum_{i=1}^{10} |x_{robot,i}|}{10} = 5.667 \text{ cm}$$

Sai lệch vị trí trung bình theo phương y của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta y_{tb} = \frac{\sum_{i=1}^{10} |y_{robot,i}|}{10} = 7.333 \text{ cm}$$

Sai lệch khoảng cách trung bình của tâm robot trong quá trình di chuyển với tâm robot được lưu lại trong bản đồ là:

$$\Delta_{tb} = \sqrt{\Delta_{x_{tb}}^2 + \Delta_{y_{tb}}^2} = 9.267 \text{ cm}$$

Nhận xét:

- Trong quá trình di chuyển, robot không đi đến điểm đích một cách chính xác tuyệt đối, mà thường có sai lệch.

Sai lệch khoảng cách trung bình trong trường hợp không có vật cản là:

$$\Delta_{tb} = 7.081 \text{ cm}$$

Sai lệch khoảng cách trung bình trong trường hợp có vật cản là:

$$\Delta_{tb} = 9.267 \text{ cm}$$

- Vị trí tâm của robot khi robot đến điểm đích thường nằm trong góc phần tư của hệ tọa độ đo mà chưa quỹ đạo di chuyển của robot. Có thể dự đoán khi robot cách điểm đích một khoảng nào đó thì chương trình sẽ coi là robot đã tới điểm đích.

Dự đoán nguyên nhân gây ra sai số:

Trong các tham số của package DWA_planner có các thông số dung sai của điểm đích là xy_goal_tolerance và yaw_goal_tolerance. Nhưng do chưa tìm hiểu sâu về thuật toán DWA nên chưa biết 2 thông số này ảnh hưởng như thế nào đến di chuyển. Hiện tại đang để theo mặc định:

$$xy_goal_tolerance = 0.1 \text{ m}$$

$$yaw_goal_tolerance = 0.05 \text{ rad}$$

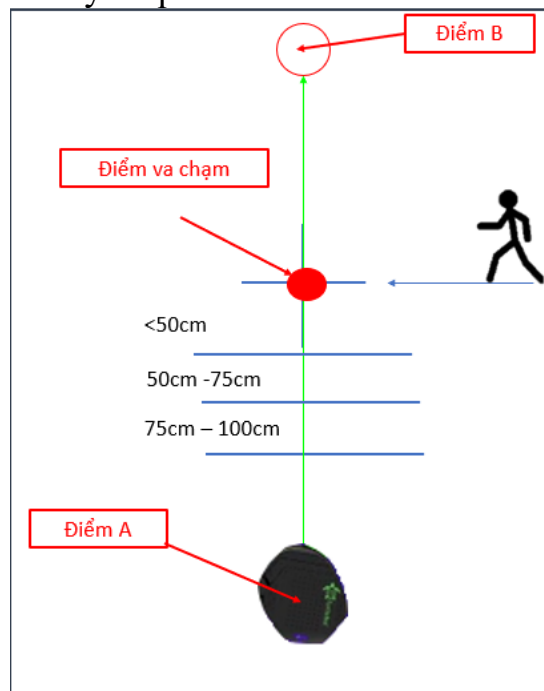
b. Đánh giá khả năng tránh vật cản chuyển động của robot

Phương pháp đánh giá:

Điều khiển robot di chuyển cố định từ điểm A đến điểm B. Trong khi Robot đang di chuyển thì người thực hiện bài đánh giá sẽ đi bộ vào quỹ đạo của Robot và dừng lại tại một điểm đã xác định từ trước. Khi dừng lại sẽ quan sát xem vị trí dừng lại cách robot một khoảng là bao nhiêu.

Quá trình test:

- Chọn 2 điểm A, B bất kì, lưu lại tọa độ trong bản đồ tĩnh của robot. Sau đó viết một chương trình điều khiển robot di chuyển từ điểm A đến điểm B.
- Khi robot di chuyển đánh dấu một vị trí nằm trên quỹ đạo di chuyển của robot (gọi là điểm va chạm). Đánh dấu thêm các vạch cách điểm vừa đánh dấu các khoảng: nhỏ hơn 50cm, lớn hơn 50cm và nhỏ hơn 75cm, lớn hơn 75cm và nhỏ hơn 100cm.
- Khi robot đang di chuyển, người làm bài đánh giá sẽ đi bộ đến điểm va chạm theo hai quỹ đạo. Quỹ đạo thứ nhất vuông góc với đường thẳng nối hai điểm A, B. Quỹ đạo thứ hai song song với đường thẳng AB và có chiều từ B đến A.
- Khi đến điểm va chạm, người thực hiện bài đánh giá sẽ quan sát xem robot đang nằm trong khoảng nào trong các khoảng kích thước đã được vạch từ trước. Ghi lại khoảng kích thước này vào bảng kết quả. Sau đó người thực hiện bài đánh giá tiếp tục đứng tại điểm va chạm đến khi robot di chuyển qua.



Hình 4.14 Sơ đồ đánh giá khả năng tránh vật cản động của Robot

Vận tốc tối đa của robot trong quá trình di chuyển từ điểm A đến điểm B: 40 cm/s. Người thực hiện bài đánh giá sẽ di chuyển đến điểm va chạm theo phương vuông góc với đường thẳng AB với tốc độ đi bộ bình thường là khoảng 100cm/s. Ghi lại các kết quả vào B4.7. Sau đó người thực hiện bài đánh giá sẽ di chuyển đến điểm va chạm theo phương vuông góc với đường thẳng AB với tốc độ đi bộ chậm là khoảng 30cm/s. Ghi lại các kết quả vào B4.8. Cuối cùng người thực hiện bài đánh giá sẽ di chuyển đến điểm va chạm theo phương song song với đường thẳng AB (chiều từ B đến A) với tốc độ đi bộ bình thường. Ghi lại các kết quả vào B4.9.

Bảng kết quả:

Bảng 4.7 Bảng kết quả khi người di chuyển theo quỹ đạo vuông góc với đường thẳng AB với tốc độ khoảng 100cm/s

Lần test	Có bị va chạm không	Khoảng cách của robot đến vật cản
1	không	0.5m – 0.75m
2	có	<0.5m
3	không	0.75m – 1m
4	không	0.5m – 0.75m
5	có	<0.5m
6	không	0.75m -1m
7	không	0.75m -1m
8	không	0.75m -1m
9	có	0.5m - 0.75m
10	không	0.5m - 0.75m

Bảng 4.8 Bảng kết quả khi người di chuyển theo quỹ đạo vuông góc với đường thẳng AB với tốc độ khoảng 30cm/s

Lần test	Có bị va chạm không	Khoảng cách của robot đến vật cản
1	không	<0.5m
2	không	<0.5m
3	không	<0.5m
4	không	<0.5m
5	không	0.5m – 0.75m
6	không	<0.5m
7	không	<0.5m
8	có	<0.5m
9	không	<0.5m
10	không	<0.5m

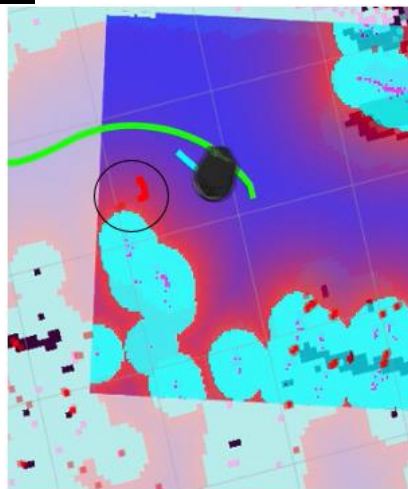
Bảng 4.9 Bảng kết quả khi người di chuyển theo quỹ đạo song song với đường thẳng AB (chiều từ B đến A) với tốc độ đi bộ bình thường khoảng 100cm/s

Lần test	Có bị va chạm không	Khoảng cách của robot đến vật cản
1	không	<0.5m
2	không	<0.5m
3	có	<0.5m
4	có	<0.5m
5	không	0.75m-1m
6	có	<0.5m
7	không	0.75m – 1m
8	không	0.5m -0.75m
9	không	0.5m – 0.75m
10	không	0.5m

Nhận xét kết quả:

- Khi người thực hiện bài đánh giá đi đến điểm va chạm theo quỹ đạo vuông góc với đường thẳng AB với tốc độ đi bộ bình thường thì robot xảy ra va chạm khi robot nằm trong khoảng nhỏ hơn 50cm.
- Khi người thực hiện bài đánh giá đi đến điểm va chạm theo quỹ đạo vuông góc với đường thẳng AB với tốc độ đi bộ chậm thì robot ít xảy ra va chạm hơn so với khi di chuyển với tốc độ đi bộ bình thường.
- Khi người thực hiện bài đánh giá đi đến điểm va chạm theo quỹ đạo song song với đường thẳng AB (chiều từ B đến A) với tốc độ đi bộ bình thường thì robot ít xảy ra va chạm hơn so với khi di chuyển theo phương vuông góc với đường thẳng AB.

Dự đoán nguyên nhân:



Hình 4.15 Vật cản mà cảm biến laser phát hiện nhưng chưa được cập nhật lên bản đồ bản đồ cục bộ

Hiện tại đang đặt tham số update bản đồ: `map_update_interval = 0.5s`. Do đó khi người thực hiện bài đánh giá di chuyển theo phương vuông góc với đường thẳng AB với vận tốc 100cm/s, thì robot sẽ không kịp cập nhật bản đồ như H4.15. Dẫn đến việc robot sẽ không kịp tìm quỹ đạo di chuyển mới để tránh vật cản mà vẫn di chuyển theo quỹ đạo cũ. Vì vậy nên robot có khả năng va chạm khi khoảng cách từ robot đến điểm va chạm nhỏ hơn 50cm.

Còn trường hợp người thực hiện bài đánh giá di chuyển theo phương song song với đường thẳng AB chiều từ B đến A thì vật cản sẽ nằm trên quỹ đạo chính được xác định nhờ vào bản đồ tổng thể. Do đó robot sẽ tìm một quỹ đạo chính khác để tránh vật cản mới xuất hiện này.

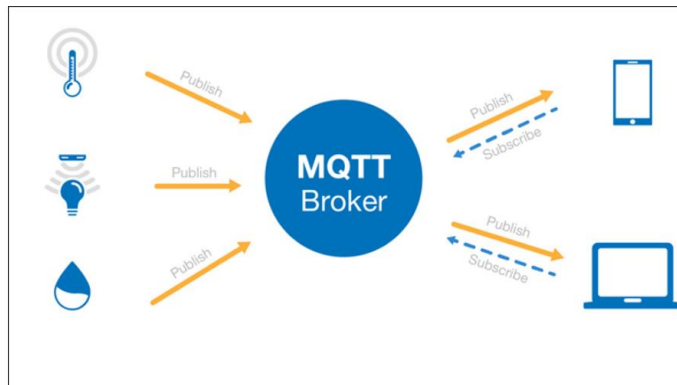
4.1.3 Kết hợp tay thao tác và robot Kobuki

Với mục đích của đề tài là kết hợp giữa tay thao tác cố định và xe tự hành thông minh để có được một sản phẩm tay thao tác di động thực hiện các nhiệm vụ trong y tế như: gấp rác thải y tế độc hại, điều chế các loại dung dịch nguy hiểm, di chuyển dụng cụ y tế trong các phòng bệnh. Tuy nhiên, trong đề tài, nhóm chưa thể hoàn thiện bước đặt trực tiếp mô hình tay thao tác lên bộ robot Kobuki bởi một số khó khăn trong việc mượn dùng thiết bị và thời gian thực hiện tương đối ngắn. Vì vậy, trong đề án này, nhóm sẽ thực

hiện giao tiếp giữa hai bộ phận này qua một giao thức truyền thông nào đó. Nhằm mục đích truyền tải trạng thái hoạt động của cả hai cho nhau và có thể giám sát tình trạng và điều khiển chúng từ xa.

a. Giao thức MQTT

MQTT(Message Queueing Telemetry Transport)là một giao thức mạng kích thước nhỏ (lightweight), hoạt động theo cơ chế publish–subscribe (tạm dịch:xuất bản–đăng ký) theo tiêu chuẩn ISO(ISO/IEC 20922)và OASIS mở để truyền tin nhắn giữa các thiết bị. Giao thức này hoạt động trên nền tảng TCP/IP. MQTT được thiết kế cho các kết nối cho việc truyền tải dữ liệu cho các thiết bị ở xa, các thiết bị hay vi điều khiển nhỏ có tài nguyên hạn chế hoặc trong các ứng dụng có băng thông mạng bị hạn chế.



Hình 4.16 Sơ đồ nguyên lý của giao thức MQTT

MQTT là lựa chọn lý tưởng trong các môi trường như:

- Những nơi mà giá mạng viễn thông đắt đỏ hoặc băng thông thấp hay thiếu tin cậy.
- Khi chạy trên thiết bị nhúng bị giới hạn về tài nguyên tốc độ và bộ nhớ.
- Bởi vì giao thức này sử dụng băng thông thấp trong môi trường có độ trễ cao nên nó là một giao thức lý tưởng cho các ứng dụng M2M(Machine to Machine).

MQTT Broker: hay máy chủ mô giới được coi như trung tâm, nó là điểm giao của tất cả các kết nối đến từ Client (Publisher/Subscriber). Nhiệm vụ chính của Broker là nhận thông điệp (message) từ Publisher, xếp vào hàng đợi rồi chuyển đến một địa điểm cụ thể. Nhiệm vụ phụ của Broker là nó có thể đảm nhận thêm một vài tính năng liên quan tới quá trình truyền thông như: bảo mật message, lưu trữ message, logs, MQTT Broker được cung cấp dưới dạng mã nguồn mở hoặc các phiên bản thương mại giúp người dùng có thể tự cài đặt và tạo broker riêng.

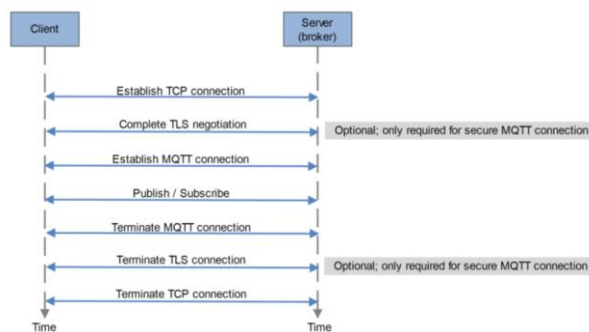
MQTT Client:

Là các thiết bị/ứng dụng Client kết nối đến Broker để thực hiện truyền nhận dữ liệu .Client thì được chia thành hai nhóm là Publisher và Subscriber. Một Client có thể có 1 trong 2 nhiệm vụ hoặc cả 2. Publisher là thiết bị gửi bản tin lên broker. Subscriber là người nhận bản tin mỗi khi có bản tin mới gửi lên Broker.

Cơ chế hoạt động của giao thức MQTT:

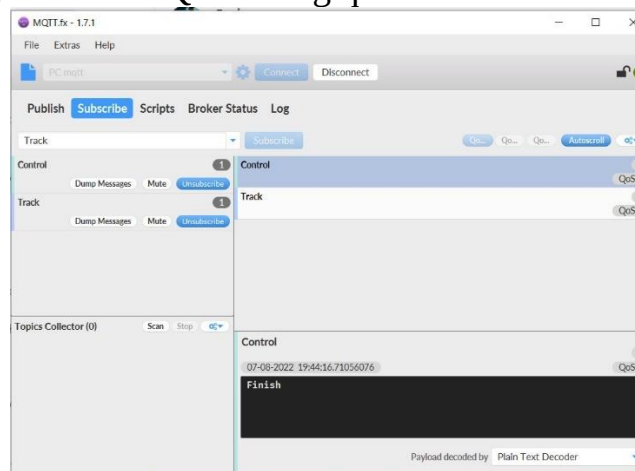
Một phiên MQTT được chia thành bốn giai đoạn: kết nối, xác thực, giao tiếp và kết thúc.

1. Client (máy khách) bắt đầu bằng cách tạo kết nối Transmission Control Protocol/Internet Protocol (TCP/IP) tới broker bằng cách sử dụng cổng tiêu chuẩn hoặc cổng tùy chỉnh được xác định bởi các nhà phát triển broker.
2. Các cổng tiêu chuẩn là 1883 cho giao tiếp không mã hóa và 8883 cho giao tiếp được mã hóa – sử dụng Lớp cổng bảo mật (SSL) / Bảo mật lớp truyền tải (TLS). Trong quá trình giao tiếp SSL/TLS, máy khách cần kiểm chứng và xác thực máy chủ.
3. Sau đó, Client sẽ gửi bản tin lên broker nếu là Publisher hoặc nhận bản tin từ broker về nếu là Subscriber. Quá trình kết nối này sẽ được giữ đến khi Kết thúc kết nối.
4. Sau khi kết thúc để có thể truyền nhận MQTT, chúng ta lại tiếp tục quay lại các bước trên.



Hình 4.17 Cơ chế hoạt động của giao thức MQTT

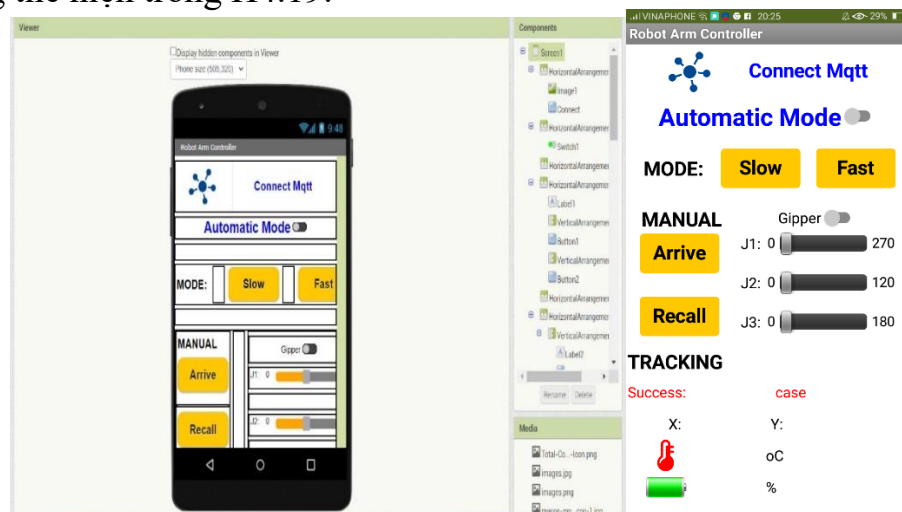
Trong phạm vi đề án mà nhóm thực hiện H4.18: App điều khiển sẽ hiển thị công khai (publish) các thông số điều khiển lên server vào topic "Control" và subscribe topic "Track" để hiển thị các thông số của robot lên app. Các dữ kiện mà nhóm lựa chọn hiển thị và đẩy lên Server bao gồm: góc quay các khớp, nhiệt độ và phần trăm pin. Cánh tay và phần thân dưới robot sẽ lấy dữ liệu từ topic "Control" để hành động và đẩy các thông số qua topic "Track". Kết nối sử dụng giao thức MQTT thông qua broker online hivemq.



Hình 4.18 Giao diện Server

b. Giao diện app điều khiển

Để có thể thực hiện điều khiển rôbot hoạt động, nhóm sẽ xây dựng một app điều khiển. Nhóm sử dụng App Inventor là một ứng dụng web mã nguồn mở được cung cấp bởi Google từ tháng 7 năm 2010. Sau này, App Inventor được quản lý bởi Viện Công nghệ Massachusetts hay còn gọi là MIT. Đó cũng là lý do tại sao nó hay được gọi là MIT App Inventor. Về cơ bản, App Inventor sẽ hoạt động dựa trên nền tảng di động Android. Tức là các thành phẩm được tạo ra từ App Inventor sẽ chỉ hoạt động được trên Android. Giao diện của App Inventor bao gồm các khối hộp, bên trong là các đoạn mã. Khi sử dụng, người dùng sẽ kéo thả các khối này vào bảng mã để tiến hành lắp ghép thành một ứng dụng hoàn chỉnh. Sơ đồ các khối bố trí của app mà nhóm xây dựng thể hiện trong H4.19.



Hình 4.19 Giao diện xây dựng app điều khiển tay thao tác

Giao diện app điều khiển bao gồm các chế độ Auto/Manual, các chế độ hoạt động ở vận tốc cao/thấp (đã được trình bày trong mục 4.1), các thanh điều khiển góc quay các khâu và hiển thị nhiệt độ và phần trăm pin còn lại. Ở chế độ "Manual" robot sẽ được điều khiển hoàn toàn thông qua app. Ở chế độ Auto, thân dưới sẽ chạy đến đích sau đó đẩy thông tin đã đến đích lên server, sau đó cánh tay sẽ nhận biết được và tiến hành gấp vật.

4.2 Hướng phát triển

Dựa vào các kết quả đánh giá trong mục 4.1.1 và 4.1.2, nhóm đã đưa ra một số ý tưởng cải tiến cho mô hình để có thể hoàn thiện hơn nữa và có thể áp dụng vào môi trường đời sống thực tế cũng như môi trường công nghiệp nói riêng. Sau đây là một số hướng phát triển mà nhóm đưa ra.

Hướng phát triển 1:

Hướng phát triển thứ nhất là hướng phát triển mà nhóm đưa ra nhằm giải quyết các vấn đề còn đang gặp phải của mô hình cơ khí tay thao tác. Chính vì những sai số của các chi tiết gây nên sai số cho toàn bộ hệ thống. Vì vậy, việc lựa chọn những chi tiết có độ chính xác cao hơn, phần nào đó sẽ giảm bớt những sai số không đáng có này, ví dụ như: lựa chọn các động cơ có độ chính xác cao, độ dơ nhỏ (như các động cơ servo tải trọng lớn); lựa chọn bộ truyền thích hợp, giảm thiểu tối đa các tác động do chùng đai, giãn đai (lựa chọn các loại đai có vật liệu tốt hơn); thực hiện các phương pháp gia công

chính xác như phay CNC, tiện CNC thay vì các phương pháp gia công hiện tại như in 3D, tiện thủ công bằng máy vạn năng. Ngoài ra, phần rôbot tự hành thông minh cũng có thể cải tiến để nó có thể di chuyển trên các địa hình phức tạp như bậc thang hay địa hình không bằng phẳng.

Hướng phát triển thứ 2:

Hướng phát triển thứ hai mà nhóm hướng tới là tích hợp thêm các công nghệ cao để mô hình có thể đảm nhận những vai trò và nhiệm vụ khó hơn. Có thể gắn thêm các camera giám sát, thu thập, nhận diện và phân tích các hình ảnh thu được trong quá trình di chuyển. Từ đó mà đưa ra những phương án giải quyết thích hợp để đáp ứng nhu cầu đặt ra. Đây hiện tại đang là xu thế của hướng phát triển các rôbot đa năng hiện nay.

KẾT LUẬN

Trên đây là bản báo cáo đồ án tốt nghiệp **Thiết kế, chế tạo và điều khiển tay thao tác di động ứng dụng trong y tế** của nhóm chúng em. Đây là một đề tài có tính thực tế cao, trong đời sống ngày càng phát triển sự cạnh tranh không ngừng đòi hỏi năng suất và chất lượng phải được cải thiện nhờ đây chuyên máy móc hiện đại thay thế lao động thủ công của con người trong những hoạt động có sự nguy hiểm và độc hại lớn.

Như vậy trong đồ án tốt nghiệp em đã được tìm hiểu được cách xây dựng một mô hình robot với một số yêu cầu đề bài dưới đây: rôbốt 4 bậc tự do, tải trọng 0.5kg tầm với 350mm, vận tốc bàn tay kẹp 0.25m/s, nhóm đã đạt được một số kết quả về tính toán hệ thống cơ khí, lập trình, mô phỏng hoạt động, thiết kế, chế tạo, xây dựng mô hình thực tế và đánh giá các kết quả thu được. Công việc hoàn thành bao gồm:

- Tổng quan về tay thao tác, rôbốt tự hành thông minh;
- Tính toán các bài toán động học của tay thao tác;
- Xây dựng kết cấu 2D cho tay thao tác bằng phần mềm AutoCad;
- Dựng mô hình 3D bằng Solidworks;
- Lựa chọn, động cơ, kích thước và độ bền các khâu;
- Xây dựng mạch điều khiển PCB bằng phần mềm Altium
- Xây dựng mô hình tay thao tác thực tế;
- Xây dựng giải thuật và code.
- Mô phỏng không gian chuyển động của tay thao tác;
- Tìm hiểu về hệ điều hành ROS, các bài toán di chuyển có liên quan;
- Khảo sát và đánh giá các kết quả từ các bài toán kiểm nghiệm để tìm ra những nguyên nhân và phương pháp khắc phục.

Qua đề tài trên nhóm chúng em đã biết cách vận dụng những kiến thức chuyên môn được đào tạo ở Đại học Bách Khoa Hà Nội trong thời gian qua vào với thực tế cuộc sống nhất là với đời sống y tế nói riêng. Sản phẩm sẽ phần nào hỗ trợ cũng như giúp đỡ các y bác sĩ trong quá trình làm việc và nghiên cứu. Không chỉ vậy qua đồ án này chúng em cũng học được rất nhiều như kỹ năng làm việc nhóm, giải quyết vấn đề, tìm tài liệu, viết báo cáo... rất có ích cho sau này. Một lần nữa nhóm xin chân thành cảm ơn sự hướng dẫn tận tình của TS Nguyễn Xuân Hạ cùng các thầy cô trong bộ môn đã giúp em hoàn thành đề tài này.

Do giới hạn về thời gian, kiến thức và chất lượng các linh kiện sử dụng gây nhiều khó khăn trong quá trình thực hiện trong đồ án này; nhóm em đã giải quyết một số vấn đề cơ bản trong việc thiết kế một tay thao tác di động. Tuy nhiên còn rất nhiều vấn đề cần phải giải quyết để có một sản phẩm hoàn thiện đạt độ chính xác cao vì vậy em rất mong quý thầy cô và các bạn đóng góp ý kiến để đề tài này hoàn thiện hơn nữa.

Nhóm chúng em xin chân thành cảm ơn!

TÀI LIỆU THAM KHẢO

Sách giáo trình, bài giảng

- [1]- **Nguyễn Thiện Phúc**, *Robot công nghiệp*, Nhà xuất bản khoa học và kỹ thuật Hà Nội, 2004.
- [2]- **PGS.TS. Phan Bùi Khôi**, *Bài giảng Tính toán thiết kế robot*, ĐH Bách khoa Hà Nội.
- [3]- **PGS.TS. Phan Bùi Khôi**, *Bài giảng Robotics*, ĐH Bách khoa Hà Nội.
- [4]- **GS.TSKH. Nguyễn Văn Khang**, *Cơ sở Robot công nghiệp*.
- [5]- **GS.TSKH. Nguyễn Thiện Phúc**, *Robot công nghiệp*.
- [6]- **Nguyễn Doãn Phước**, *Lý thuyết điều khiển tự động*.
- [7]- **Vương Sỹ Cường**, *Điều khiển Robot tự hành thông minh Kobuki dựa trên hệ điều hành ROS*.
- [8] *Xu thế phát triển của rô bốt trên thế giới và tình hình nghiên cứu phát triển rô bốt ở Việt Nam hiện nay*, Nguồn: <http://www.vast.ac.vn>
- [9] *Ifr Releases Its Annual World Robotics Report*.
Nguồn: <https://www.universal-robots.com>
- [10] **YoonSeok Pyo; HanCheol Cho; RyuWoon Jung; TaeHoon Lim**, *ROS Robot Programming*, ROBOTIS Co.,Ltd., 2017.
- [11] *explore Kuri*. Nguồn: <https://www.heykuri.com/explore-kuri/>
- [12] *Turtlebot3*. Nguồn: <https://robots.ros.org/turtlebot3/>
- [13] **Sebastian Thrun; Wolfram Burgard; Dieter Fox**, *Probabilistic Robotic*, Cambridge, Massachusetts: The MIT Press, 2006
- [14] *Kobuki Robot*. Nguồn: <http://kobuki.yujinrobot.com/about2/>

PHỤ LỤC 1

(Code điều khiển cánh tay thao tác)

```
#include "Motor_Driver.h"
#include "math.h"

void Mobile_Robot_v1_Setup(){

}

void Motor_Init(Motor *motor_x) {

    //set up Travel Switch 1
    motor_x->GPIO_PIN_TRSW1.Mode=GPIO_MODE_INPUT;
    motor_x->GPIO_PIN_TRSW1.Pull=GPIO_NOPULL;
    HAL_GPIO_Init(motor_x -> GPIO_PORT_TRSW1, &motor_x ->
GPIO_PIN_TRSW1);
    //set up Direction 1
    motor_x -> GPIO_PIN_DIR1.Mode = GPIO_MODE_OUTPUT_PP;
    motor_x -> GPIO_PIN_DIR1.Pull = GPIO_NOPULL;
    motor_x -> GPIO_PIN_DIR1.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(motor_x -> GPIO_PORT_DIR1, &motor_x ->
GPIO_PIN_DIR1);
    //set up Direction 2
    motor_x -> GPIO_PIN_DIR2.Mode = GPIO_MODE_OUTPUT_PP;
    motor_x -> GPIO_PIN_DIR2.Pull = GPIO_NOPULL;
    motor_x -> GPIO_PIN_DIR2.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(motor_x -> GPIO_PORT_DIR2, &motor_x ->
GPIO_PIN_DIR2);
    //set up encoder phase A
    motor_x->ENCODER_PIN_PHASEA.Mode = GPIO_MODE_IT_FALLING;
    motor_x->ENCODER_PIN_PHASEA.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(motor_x->ENCODER_PORT_PHASEA,
&motor_x->ENCODER_PIN_PHASEA);
```

```

    HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
    //set up encoder phase B
    motor_x->ENCODER_PIN_PHASEB.Mode=GPIO_MODE_INPUT;
    motor_x->ENCODER_PIN_PHASEB.Pull=GPIO_NOPULL;
    HAL_GPIO_Init(motor_x -> ENCODER_PORT_PHASEB, &motor_x ->
ENCODER_PIN_PHASEB);
    //set up PWM
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};
    motor_x->TIM_PWM.Instance = TIM2;
    motor_x->TIM_PWM.Init.Prescaler = PRESCALER_DEFAULT;
    motor_x->TIM_PWM.Init.CounterMode = TIM_COUNTERMODE_UP;
    motor_x->TIM_PWM.Init.Period = COUNTER_DEFAULT;
    motor_x->TIM_PWM.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    motor_x->TIM_PWM.Init.AutoReloadPreload
                                                    =
TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&motor_x->TIM_PWM) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if
        (HAL_TIM_ConfigClockSource(&motor_x->TIM_PWM,
&sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_Init(&motor_x->TIM_PWM) != HAL_OK)
    {
        Error_Handler();
    }

```

```

sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&motor_x->TIM_PWM,
&sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
sConfigOC.OCMode = TIM_OCMode_PWM1;
sConfigOC.Pulse = 0;
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&motor_x->TIM_PWM, &sConfigOC,
motor_x->CHANNEL_PWM) != HAL_OK)
{
    Error_Handler();
}
HAL_TIM_MspPostInit(&motor_x->TIM_PWM);
    HAL_TIM_PWM_Start(&motor_x->TIM_PWM,motor_x->CHANNEL_
PWM);

}

void Motor_Rotate_Free(Motor *motor_x,uint8_t dir,uint16_t speed){
    if(dir==CW){
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,0);
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,1);
    }
    else if(dir==CCW){
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,1);

```

```

        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,0);
    }
    else if(dir==STOP){
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,0);
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,0);
    }
    else if(dir==FIX){
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,1);
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,1);
    }
    __HAL_TIM_SET_COMPARE(&motor_x->TIM_PWM,motor_x->CHA
NNEL_PWM,speed);
}

void Motor_Homing(Motor *motor_x,uint16_t home_vel,long *count){
    //home_vel=(home_vel<COUNTER_DEFAULT)?home_vel:COUNTER_
DEFAULT;
    motor_x->Encoder_dir=1;
    if(HAL_GPIO_ReadPin(motor_x->GPIO_PORT_TRSW1,motor_x->GPIO_PIN
_TRSW1.Pin)==0){
        *count=0;
        while(HAL_GPIO_ReadPin(motor_x->GPIO_PORT_TRSW1,motor_x->
GPIO_PIN_TRSW1.Pin)==0){
            Motor_Rotate_Free(motor_x,motor_x->POS_DIR,home_vel);
        }
        if(*count<0)
            motor_x->Encoder_dir=-1;
        Motor_Homing(motor_x,home_vel,count);
    }
}

```

```

}
else{
*count=0;
while(HAL_GPIO_ReadPin(motor_x->GPIO_PORT_TRSW1,motor_x->GPIO_
PIN_TRSW1.Pin)==1){
uint8_t NEG_DIR=(motor_x->POS_DIR==CW)?CCW:CW;
Motor_Rotate_Free(motor_x,NEG_DIR,home_vel);
}
if(*count>0)
    motor_x->Encoder_dir=-1;
Motor_Rotate_Free(motor_x,4,0);
}
*count=0;
}

void Motor_RotatePID_2angle(Motor *motor_x,long *count,uint16_t
max_speed){
motor_x->er=(motor_x->Position)*(motor_x->pulse_1degree)-*count;
float a_g=150*cos((( *count)*3.14)/((motor_x->pulse_1degree)*180));
    float PID_value;
    if(((motor_x->er)<(motor_x->pulse_1degree)*4)&&((motor_x->er)>-
(motor_x->pulse_1degree)*4))
        PID_value=0;
    else
        PID_value=(motor_x->Kp)*(motor_x->er)+(motor_x->Kd)*((motor_x->er)-
(motor_x->pre_er))+a_g;
    uint16_t vel;
    (motor_x->pre_er)=(motor_x->er);
    if(PID_value>0){
        vel=(uint16_t)PID_value;
    }
    if(vel>max_speed)
        vel=max_speed;
    //caculate direction

```

```

if(motor_x->POS_DIR==CW){
    HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,0);
    HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,1);
}
else {
    HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,1);
    HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,0);
}
Motor_Rotate_Free(motor_x,motor_x->POS_DIR,vel);
}
else {
    vel=-(uint16_t)PID_value;
    if(vel>max_speed)
        vel=max_speed;
    //caculate direction
    if(motor_x->POS_DIR==CW){
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,1);
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,0);
    }
    else {
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR1,motor_x->GPIO_P
IN_DIR1.Pin,0);
        HAL_GPIO_WritePin(motor_x->GPIO_PORT_DIR2,motor_x->GPIO_P
IN_DIR2.Pin,1);
    }
    uint8_t NEG_DIR=(motor_x->POS_DIR==CW)?CCW:CW;

```

```
Motor_Rotate_Free(motor_x,NEG_DIR,vel);  
}  
}
```